LDImmersiveEngineering

The Complete Manual

A private fork of Immersive Engineering for Minecraft 1.12.2

August 20, 2026

Abstract

This is the whole of Immersive Engineering as this fork ships it: every block, every machine, every system, and the four large additions that are not in the mod upstream — the virtual power grid, the petroleum chain, the virtual fluid network, and conduits — together with the city-mode performance work that runs underneath all of it.

Where this fork differs from stock Immersive Engineering, the difference is called out in an orange box. Where a number appears, it was read out of the source rather than remembered.

Contents

Part I

The mod

CHAPTER 1

What this is

Immersive Engineering is a technology mod built around the idea that machinery should look like machinery. Its power is carried on wires strung between poles, its ore is crushed by a drum you can watch turn, and the large machines are structures you assemble block by block rather than single cubes that happen to contain a factory.

This manual documents **LDImmersiveEngineering**, a private fork of that mod for Minecraft 1.12.2. Everything the original does, this does. On top of it sit four large additions and one performance regime, none of which exist upstream:

The virtual power grid Moves Immersive Flux between places with no wire between them. chapter 10.
Petroleum A complete oil chain: prospect, drill, refine, store, burn, sell. chapter 11.
The virtual fluid network The same idea as the grid, for fluid. chapter 12.
Conduits Surface-mounted bundled wiring for the inside of buildings. chapter 13.
City mode Trades simulation detail for server tick time, subsystem by subsystem. chapter 16.

1.1 How to read this

Part II is Immersive Engineering as it ships. Part III is what the fork adds. Part IV is generated straight from the source tree — every block, every recipe, every configuration option — and is the place to look when you want a number rather than an explanation.

Three kinds of callout appear throughout:

Tip

Something that makes the mod easier, or a habit worth forming early.

Careful

Something that will cost you a build, a machine or an afternoon if you get it wrong.

This fork

Behaviour that differs from stock Immersive Engineering. If you already know the mod, these boxes are the parts you have to read.

1.2 Where the numbers come from

Every figure in this manual was read out of the source. Where a number is configurable — and most are — the value quoted is the shipped default, and chapter 19 lists the option that changes

it.

Careful

A modpack may have changed any of them. If a machine in your world does not match a number here, check the config before assuming the manual is wrong.

CHAPTER 2

Getting started

Immersive Engineering does not begin with a machine. It begins with a hammer and a fire.

2.1 The Engineer's Hammer

Three iron ingots and two sticks. It is the mod's universal tool and you will never stop carrying one:

- **Assembles multiblocks.** Every structure in chapter 7 is built by placing its blocks and then striking one of them.
- **Rotates blocks.** Right-click most machines to turn them.
- **Configures sides.** Sneak-right-click a machine face to cycle what that side does.
- **Dismantles.** Takes a multiblock apart into its parts.

2.2 The Engineer's Manual

One book and one lever. It is the in-game version of this document, and it updates itself as you obtain things. chapter 20 reproduces it in full.

2.3 Treated wood

Creosote oil in a barrel or a Bottling Machine turns planks into **treated wood**, which resists weather and is the base of almost everything early. You get creosote from the coke oven, so the coke oven comes first.

2.4 The Coke Oven

A $3 \times 3 \times 3$ of Coke Bricks. Turns coal into **coal coke** and **creosote oil**, both of which you need continuously:

- Coal coke burns longer than coal and is the fuel for the Blast Furnace.
- Creosote oil treats wood and, later, is a Squeezer input.

Tip

Build two. The oven is slow, both outputs are consumed constantly, and the second one costs almost nothing next to the time it saves.

2.5 The Blast Furnace

A $3 \times 3 \times 3$ of Blast Bricks. Iron plus coal coke gives **steel**, which is the material the entire middle of the mod is built from. It also produces **slag**, a Blast Brick ingredient, so the furnace partly pays for its own expansion.

The **Advanced Blast Furnace** is the upgrade: faster, and it accepts external heating from a Blast Furnace Preheater.

2.6 Your first power

Immersive Engineering measures power in **Immersive Flux (IF)**, which is Forge Energy under another name — it interoperates with RF and FE at one to one.

The cheapest start is a **Water Wheel** or a **Windmill**: both are built from treated wood, need no fuel and produce a trickle. Put a **Dynamo** behind the wheel, an **LV Connector** on the dynamo, and run copper wire to whatever needs it.

Careful

Wires are not decorative. Walking into an energised HV wire will hurt you, and connecting a wire to a connector of the wrong tier will destroy it. See chapter 4.

2.7 The order most people build in

1. Hammer, manual, coke oven.
2. Treated wood, then a water wheel or windmill for a first trickle of power.
3. Blast furnace, then steel.
4. Crusher, for doubled ore.
5. Metal Press, for cheap plates and rods — this is the point the mod starts paying you back.
6. Diesel generator or one of the fork's power plants (chapter 11) for real output.

Part II

Immersive Engineering

CHAPTER 3

Materials

3.1 Ores

Immersive Engineering adds **copper**, **aluminium** (bauxite), **lead**, **silver**, **nickel** and **uranium**. All generate in ordinary stone; uranium is deep and sparse.

Every one of them can be doubled in a Crusher and, with the Arc Furnace, taken further still.

3.2 Alloys

Made in the Alloy Smelter, or in an Arc Furnace once you have one:

Constantan Copper + nickel. Used in the Thermoelectric Generator and MV components.
Electrum Silver + gold. The MV wire metal.
Steel Iron + coal coke, in the Blast Furnace rather than the Alloy Smelter. The HV wire metal and the backbone of everything.

3.3 Plates, rods and wire

Three shapes, and the machines that make them cheaply are the reason to build those machines:

Plates Hammered from ingots by hand, or pressed far more cheaply in a **Metal Press** with a plate mould.
Rods (sticks) Same, with a rod mould.
Wire Same, with a wire mould. Wire is what wire coils are made of.

Tip

Hand-hammering plates works and is miserable at volume. The Metal Press pays for itself within one build, and its moulds are the cheapest thing in the mod to duplicate.

3.4 Sheet metal

Decorative and structural blocks in every metal the mod knows. They are the visual vocabulary of an IE build — most large multiblocks are faced in them — and several structures require them outright.

3.5 Concrete

Sand, gravel, clay and water give **concrete**, which sets from a liquid. It is blast resistant, comes in several finishes, and is what most players end up building the actual buildings from.

3.6 Hemp

A crop grown from Hemp Seeds. Two blocks tall, harvested from the top. Gives **plant fibre**, which becomes **fabric** and then the cloth used in Balloons, Shaders and the Powerpack.

3.7 Coal coke and creosote

Covered in chapter 2, and worth repeating because they never stop being needed: coke fuels the Blast Furnace and creosote treats wood.

3.8 Fluids

Stock Immersive Engineering ships creosote, plant oil, ethanol, biodiesel and concrete.

This fork

This fork adds the entire petroleum family — crude, the refined fractions, and the products of cracking. chapter 11.

CHAPTER 4

Wiring

Power in Immersive Engineering travels along wires strung between connectors. The wires sag, they have length limits, they lose energy over distance, and at high voltage they will kill you. That is the whole character of the mod in one system.

4.1 The three tiers

Tier	Wire	Max transfer	Loss	Max length
LV	Copper	2 048 IF/t	5 %	16 blocks
MV	Electrum	8 192 IF/t	2.5 %	16 blocks
HV	Steel	32 768 IF/t	2.5 %	32 blocks

Loss is proportional to how much of the wire's maximum length the run actually uses, so a short HV run loses very little and a run at the limit loses the full percentage.

Tip

Insulated copper and insulated electrum carry the same power as their bare versions and do not shock anything that touches them. They cost more. On a build people walk through, pay it.

4.2 Connectors and relays

A **connector** is where a wire meets a machine: it takes power from the block it is attached to, or gives power to it. A **relay** only passes wire to wire — it is a pole, not a socket — and relays are how you get a line across a distance without a machine at every corner.

Careful

A connector accepts only its own tier's wire. Attaching the wrong one destroys the connector.

4.3 Transformers

Step between tiers. An **LV/MV Transformer** joins a low-voltage side to a medium-voltage one; an **MV/HV Transformer** does the same higher up. Wire the correct tier to each side — they are not symmetrical, and the block's model tells you which side is which.

4.4 Wire coils and the wirecutter

Wire is placed by holding a **coil** and right-clicking one connector, then the other. The **Engineer's Wirecutter** removes a wire and returns the coil.

4.5 Posts and arms

Wooden and **Steel Posts** are what you actually string wire between across country. Both accept arms, which give you somewhere to mount connectors and transformers off the pole itself.

4.6 Breaker switches and meters

Breaker Switch A manual on/off in a wire run, also switchable by redstone.
Redstone Breaker The same, driven by redstone alone.
Energy Meter Reports throughput on the line passing through it.
Current Transformer Emits a redstone signal proportional to the power flowing.

4.7 Feedthroughs

An insulator that carries a wire through a wall without breaking the run, so a line can enter a building without a connector on each side.

4.8 Wire damage

An energised HV wire damages anything that touches it, and an overloaded wire burns out and drops as scrap.

This fork

City mode changes both. Wire burnout is disabled outright, and shock damage becomes local: rather than every powered connector broadcasting to the whole network every tick on the chance somebody is touching a wire, a node that needs the information asks for it. That single broadcast was the most expensive thing the mod did on a profiled server. chapter 16.

This fork

This fork also offers two alternatives to wire entirely: the virtual grid (chapter 10) for distance, and conduits (chapter 13) for indoors.

CHAPTER 5

Generators

5.1 Water Wheel

Treated wood. Mounted on a **Dynamo**, turned by flowing water down one side. Output scales with how much of the wheel is being driven, and three wheels can share one dynamo.

Cheap, silent, needs no fuel, produces very little. It is the right first generator and never the last.

5.2 Windmill

Treated wood on a dynamo again, mounted high and in the open. Output rises with altitude and falls if obstructed. The **Improved Windmill** adds iron sheet metal for a substantial increase.

Tip

Windmills need clear air around them, not merely above them. A windmill in a valley is a decoration.

5.3 Thermoelectric Generator

Two blocks of differing temperature on opposite sides — classically lava and packed ice, or a blaze-adjacent block against a cold one — and it produces a steady trickle with no moving parts and no fuel logistics. The default output multiplier is 1.

Its virtue is that it never stops. Its vice is that it is slow.

5.4 Diesel Generator

The first serious generator: a multiblock burning biodiesel for **4 096 IF/t** by default. Needs a fluid supply, and rewards a real one.

This fork

The petroleum chain supplies it far better than the biodiesel loop does, and adds several larger plants above it, ending at the Gas Turbine. chapter 11.

5.5 Lightning Rod

A tall multiblock that captures strikes for **16 000 000 IF** apiece by default. Entirely weather-dependent, spectacular, and useless as a base load — pair it with storage large enough to hold a

strike or most of it is wasted.

5.6 Storage

LV / MV / HV Capacitor Buffers of increasing size. Each face can be configured to input, output or nothing with a hammer.

Capacitor Backup Keeps a capacitor's contents through a chunk unload.

Careful

A capacitor with every face set to output will happily drain itself into whatever is adjacent. Configure the faces deliberately.

This fork

City mode makes fuel cosmetic for generators: a presence check and a token sip instead of a per-tick burn rate and tank drain. A generator with fuel in it runs; how much is left stops being tracked.

CHAPTER 6

Machines

The single-block devices. Everything here is placed, powered and used on its own; the assembled structures are in chapter 7.

6.1 Powered

Sample Drill Takes a core sample of the chunk, telling you what mineral vein is beneath it. The prerequisite for an Excavator.
Charging Station Charges tools, armour and any item that holds flux.
Fluid Pump Moves fluid through pipes, and is what makes a pipe network do anything.
Fluid Placer Places fluid into the world as a source block.
Garden Cloche (Belljar) Grows a crop in a glass jar from water and fertiliser. Slow, tidy, and entirely automatic.
Floodlight A directional lamp. Expensive to run and the brightest light in the mod.
Turret (Gun / Chemical) An automated defence, configurable by owner and target type.
Tesla Coil Damages entities in a radius. Genuinely dangerous to its owner.
Electric Lantern A small lamp. Cheap enough to use in quantity.

This fork

City mode caps how many light blocks a Floodlight may place and stops it re-tracing its beams on a timer — it re-traces only when the light switches or a neighbour changes. Floodlights are usually the most expensive block in a city build.

6.2 Unpowered

Wooden Barrel Holds fluid, and is where creosote treats wood by hand.
Wooden Crate Storage that keeps its contents when broken.
Reinforced Crate The same, lockable.
Workbench Where blueprints are used and tools are modified.
Item Router Sorts items by filter into up to six outputs.
Fluid Sorter The same for fluid.
Redstone Wire Connector Carries a redstone signal down a wire rather than along the ground.

6.3 Fluid pipes

Steel pipes carrying fluid between machines. They connect automatically, can be covered with sheet metal to hide them, and need a Fluid Pump to move anything.

This fork

Two changes here. The pipe network's flood fill was rewritten — it used a linear scan for its closed set and removed from the head of a list for its open set, which is quadratic on a large tank farm — and city mode lets a pipe hand its fluid to endpoints in order rather than simulating a fill against every one of them and splitting in proportion. Nothing is created or destroyed; only which of several tanks fills first changes.

CHAPTER 7

Multiblocks

Every structure here is built by placing its blocks in the right shape and then striking it with an Engineer's Hammer. The in-game manual renders each one as a rotating diagram; chapter 20 carries the text, and the exact block lists are in chapter 17.

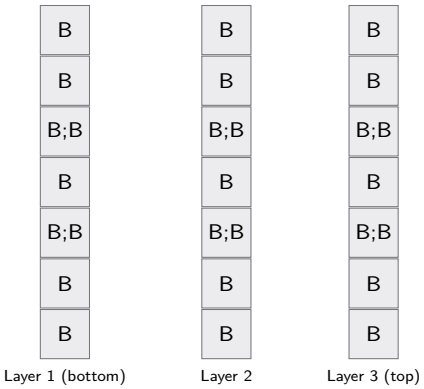
Tip

Build multiblocks with the hammer already in your hand and the manual page open. Almost every failed assembly is one block in the wrong place, and the diagram will show you which.

7.1 Reading a layer plan

Every plan in this chapter is drawn bottom layer first, left to right, as you would build it. A dot is empty space. The highlighted cell is where you strike with the hammer.

Coke Oven and Blast Furnace — both are a solid $3 \times 3 \times 3$



B = Coke Bricks for the oven, Blast Bricks for the furnace. Strike any face.

Tip

The two simplest structures in the mod are the same shape, which is why they are shown together. Once you can build one you can build the other, and both need no power at all.

7.2 Processing

Coke Oven $3 \times 3 \times 3$. Coal to coal coke and creosote oil. Needs no power.
Blast Furnace $3 \times 3 \times 3$. Iron to steel, with slag. Needs no power.

Advanced Blast Furnace Faster, and accepts a Preheater for more speed still.
Alloy Smelter Combines two metals. Needs no power.
Crusher Doubles ore, and grinds a great many other things. The first machine most players build that visibly pays for itself.
Arc Furnace Recycles, alloys and processes ores further than the Crusher can. Consumes graphite electrodes, which wear out.
Metal Press Ingots to plates, rods and wire with a mould. Cheap to run and enormously cheaper than hammering.
Squeezer Presses seeds to plant oil, and other things besides.
Fermenter Ferments crops to ethanol.
Refinery Plant oil and ethanol to biodiesel.
Mixer Combines fluids with items — concrete, most importantly.
Bottling Machine Fills containers with a fluid.
Auto Workbench Automates blueprint crafting.
Assembler Automates ordinary crafting from up to three recipes at once, pulling from a shared inventory.

7.3 Mining

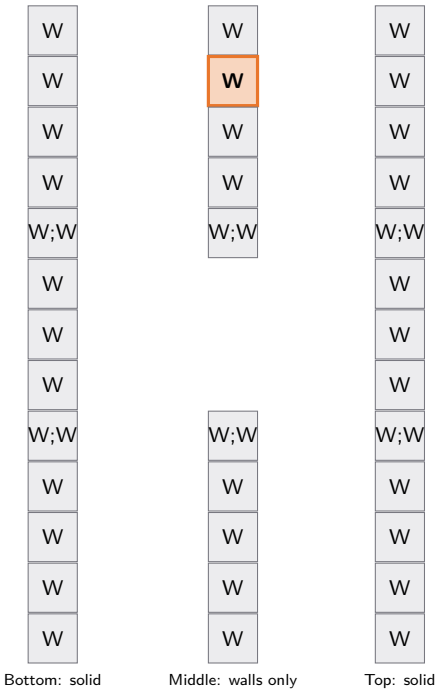
Excavator The largest structure in the mod. Digs a mineral vein located by Sample Drill, producing ore in quantity until the vein depletes. Consumes **4 096 IF/t** by default.
Bucket Wheel The visible half of the Excavator, and the reason it is worth building where people can see it.

7.4 Plans for the fork's structures

Grid Management Console — a 2×2 panel on a wall

H = Console Housing. Strike the front face, not the top.

Storage Tank — any hollow box from 3^3 to 16^3, shown at $5 \times 5 \times 3$



W = Storage Tank Wall. The inside must be empty. Strike any wall block.

Careful

A Storage Tank with anything inside the shell will not form. That is deliberate — a solid cube would otherwise be given the capacity of a box it does not actually enclose.

7.5 Storage and power

Sheetmetal Tank A $5 \times 5 \times 5$ vessel holding 512 000 mB.
Silo The same idea for solid items.
Diesel Generator chapter 5.
Lightning Rod chapter 5.

This fork

The fork adds several: the Grid Management Console (chapter 10), the Fluid Control Console (chapter 12), and the whole petroleum chain — Derrick, Cracking Unit, Loading Gantry, the buried tanks and the free-form Storage Tank (chapter 11).

This fork

City mode stops idle multiblocks re-scanning the recipe list every tick and widens the scan interval for the rest. A room full of idle machines is close to free.

CHAPTER 8

Logistics

8.1 Conveyor belts

The mod's item transport, and one of the things it is known for: items ride visibly on top rather than vanishing into a pipe.

Conveyor Belt Moves items along. Right-click with a hammer to change direction.
Uncontrolled Does not stop items at the end — they fall off.
Dropper Drops items through the block beneath.
Vertical Carries items upward.
Extract / Insert Pull from and push into adjacent inventories.
Splitter Alternates output between two directions.
Chute A vertical shaft. Fast, and made of any sheet metal.

Tip

Belts can be covered with any block, so a line can run through a wall or under a floor and still be watched where you want it seen.

8.2 Sorting

Item Router Six outputs, each with its own filter. The workhorse.
Fluid Sorter The same, for fluid.

8.3 Storage

Wooden Crate Keeps its contents when broken, so it doubles as a moving box.
Reinforced Crate Lockable to its owner.
Toolbox A portable container for tools, food and ammunition.

8.4 Posts and scaffolding

Wooden and **Steel Scaffolding** climb like ladders and are the mod's structural vocabulary.
Posts carry wire across country and take arms for mounting hardware.

8.5 Redstone

Redstone Wire Connector Carries a redstone signal along a wire.
Breaker Switch A lever in a power line.

Current Transformer A redstone signal proportional to power flowing.

This fork

Conduits carry redstone too, on any of their sixteen channels, alongside power in the same bundle. chapter 13.

CHAPTER 9

Tools and equipment

9.1 The essentials

Engineer's Hammer Assembles, rotates, configures and dismantles. Never stop carrying one.
Engineer's Wirecutter Removes wire and returns the coil.
Engineer's Voltmeter Reads a machine's stored energy and throughput.
Engineer's Manual The in-game book.

This fork

The Voltmeter gained a second job: sneak-click a grid box to pick up its segment, sneak-click others to assign them, sneak-click the air to clear. There is no Engineer's Screwdriver in 1.12 — it is a later item — so the quick-assign gesture lives here instead.

This fork

Two more fork tools, for the virtual networks:

Grid Linker Right-click a grid box: a short list of segments appears, with colours and what each is doing. Pick one and it links that box and loads the tool; every right-click after that links the box you clicked, with no window in the way. Sneak-right-click a box to choose again, or the air to empty the tool.
Fluid Linker The same thing for fluid mains and their fittings.
Engineer's Network Terminal Both networks in a pocket, read-only. Changes are still made at a console.

The linkers add to the console and the per-box panel rather than replacing either, and they respect locks: a tool in hand is not a way around one.

9.2 Powered tools

Engineer's Drill A mining tool taking a drill head, which wears out and can be replaced. Mines in a 3×3 with the right upgrade and is the fastest miner in the mod.
Buzzsaw Fells trees, and cuts wood into planks as it goes.
Chemical Thrower Sprays a fluid — extinguishing, igniting or poisoning depending on what you load.
Railgun Fires rods at very high velocity for very high damage.
Revolver The mod's sidearm, with a wide range of ammunition and cosmetic parts. Extensively upgradeable at a Workbench.

9.3 Upgrades

Most powered tools take upgrades at a Workbench: capacity, speed, extra barrels, larger tanks, scopes. The upgrade is an item and can be removed again.

9.4 Armour and equipment

Steel Armour A full set, better than iron.
Faraday Suit Protects against electricity — including the mod's own wires and Tesla Coils.
Powerpack A back-worn battery charging held tools as you use them.
Earmuffs Silence machinery near you.
Shield Blocks, and takes upgrades of its own.
Climbing Rope, Skyhook Movement. The Skyhook rides along wire runs, which is the most enjoyable transport in the mod once you have a wire network worth riding.

9.5 Shaders

Purely cosmetic skins for tools, armour and some machines, obtained from Shader Bags. They do not change behaviour and there are a great many of them.

This fork

One held item is new: the Engineer's Network Terminal, which reads the power grid and the fluid network from anywhere and changes neither. chapter 10.

Part III

This fork

CHAPTER 10

The Virtual Power Grid

Running catenary wire across a city is honest work and is not always practical. The **virtual power grid** moves Immersive Flux between places that are not connected by any wire at all — across a mountain, across a chunk border, across dimensions.

This fork

Not in stock Immersive Engineering. It takes the idea Flux Networks made popular and rebuilds it in IE's idiom: sheet-metal service boxes, a console you walk to, and named segments rather than per-player networks.

10.1 Feed Units and Service Units

Power still has to enter and leave somewhere. It enters at a **Grid Feed Unit** and leaves at a **Grid Service Unit**. Both bolt to any solid surface, including engineering posts.

These boxes exchange power with the block they are bolted to, exactly as a machine does — and they take a **wire directly**. String a coil at the terminal post on the front of a Feed or Service Unit and it connects as any connector would: no relay block in between, and any of LV, MV or HV. The wire sets the tier and the loss, exactly as it does everywhere else. A Feed Unit takes power out of the world; a Service Unit puts it back.

A Service Unit powers what it touches first and the wire second, so bolting one onto a machine still powers that machine before the run leaves the building. A Signal Unit takes no wire at all: it moves no flux, and a wire on it would be a lie.

Tip

A wire connector *beside* a Service Unit is fed as well as one bolted to it, whichever way the connector faces. IE connectors normally take power on one side only — the block they are mounted on — and a connector on the wall next to a unit would otherwise sit there touching live hardware and doing nothing.

10.2 Segments

The grid is not one pool. It is divided into **segments**: named groups of boxes, each with its own switch, its own transfer limits and its own statistics. A segment is the thing you switch off when you want the east side of town dark; the boxes on it are the sockets.

A box that has not been assigned to a segment sits with an unlit lamp doing nothing. Assigning it is the whole configuration step.

Each segment carries a **priority** order and a **transfer cap** per device, so you decide what is served first when there is not enough to go round. Anything marked **critical** is served before everything else — the pumps, the blast furnace, the hospital.

10.3 The Grid Management Console

A 2×2 wall of four different blocks, struck anywhere on its front face with an Engineer's Hammer:

	Left	Right
Upper	Console Housing (the terminal)	Redstone Engineering Block
Lower	Light Engineering Block	Heavy Engineering Block

The terminal is the screen, the redstone block is the instrument rack beside it, and the light and heavy blocks are the desk and the power cabinet under them. The finished console is drawn as one piece — one desk, one monitor spanning both blocks — and breaking it returns the four components.

Six tabs: an overview of every segment, the segment editor, the device list, the failover editor, live statistics, and settings. Right-clicking any box also opens its own small panel, so a freshly placed unit can be assigned without walking back.

10.4 Failover

A segment may name another as its **backup**. If the first cannot meet its load, the second covers it. With *top-up* enabled a backup also covers ordinary shortfalls, not only outages; with it off, backups engage on outage or trip alone.

10.5 Breakers and schedules

If breakers are enabled, a segment held at its output ceiling long enough will **trip** and latch off until somebody flips it back on. That is a real switch on the Segments tab.

A segment may also carry a **schedule**: two times of day between which it may run. The schedule is a gate, never a second switch — it can hold a segment down but never raise one a player switched off, because otherwise the console toggle and the clock fight every dusk and whichever ran last wins.

10.6 The Grid Signal Unit

Bridges a segment to redstone in either direction. In *input* mode a signal holds its segment off, which makes any lever, daylight sensor or logic circuit an external kill switch; inverted, it demands a keep-alive signal instead, which is a dead-man's switch. In *output* mode it reports whether its segment is up — or, inverted, whether it is down, which is a fault lamp.

10.7 The Grid Linker

Wiring a street one panel at a time is the part of this feature that was tedious. The **Grid Linker** answers it: right-click any grid box and a short list of segments appears, with each one's colour and what it is doing. Pick one and it links that box *and* loads the tool; every right-click after that links the box you clicked, with no window in the way.

Sneak-right-click a box to choose a different segment, and sneak-right-click the air to empty the tool. What it is holding is on its tooltip.

Tip

The linker adds to the console and the box's own panel rather than replacing either, and it can do exactly one thing: choose which segment a box belongs to. Segments are still created, named, priced and deleted at a console — and a locked segment stays locked, on both ends of every move.

10.8 The Engineer's Network Terminal

A held item showing every segment and every fluid main, from anywhere, and changing neither.

Tip

The terminal is read-only by construction rather than by convention — there is no path from it to a change, whatever the client sends. Changes are made at a console.

10.9 Chunk loading

A device may be marked to keep its chunk loaded, within a server-wide budget. Fittings sharing a chunk cost one between them. A disabled device stops holding its chunk.

10.10 City mode

See chapter 16. In short: segments stop accounting for flux and switch to presence. A segment is energised or it is not, and its Service Units deliver freely.

CHAPTER 11

Petroleum

A complete oil chain: find it, drill it, refine it, store it, burn it, and sell it at a forecourt.

This fork

Entirely this fork, and the largest single addition in it.

11.1 Prospecting

Oil sits in **reservoirs** beneath the world, which have to be found before they can be drilled. The survey kit reports bearings and ranges rather than handing you a coordinate.

11.2 Drilling

A **Derrick** sunk over a reservoir brings crude to the surface through a **Wellhead**. A reservoir is finite: its flow decays as it empties, and it can be blown out by mishandling.

11.3 Refining

Crude is separated into fractions, and a **Cracking Unit** converts between them. The cracker has no temperature dial — the dial is the firebox. Run it hot on heavy fuel oil and it yields gasoline; run it cool on gas and it yields diesel. A knob built out of blocks.

11.4 Storage

Three **buried tanks** — Domestic, Commercial and Bulk — are invisible except for a surface fill cap carrying the gauge. Build the shell, cover the roof, and hammer the cap.

The **Storage Tank** is the other kind: a hollow box of any rectangular shape from $3 \times 3 \times 3$ to $16 \times 16 \times 16$, holding sixteen buckets per enclosed block. Build the shell you want and strike any wall with a hammer. Pipe into any wall block, not just one fitting.

Tip

Buried tanks are for supply you do not want to look at — under a forecourt, under a yard. The Storage Tank is for a tank farm you do.

For propane specifically there are three single-block vessels: the **Propane Cylinder** at four buckets, the **Upright Propane Tank** at twelve, and the **Torpedo Propane Tank** at twenty-four. They differ only in size and shape — all three take propane and nothing else, all

three keep their contents when broken, and all three fill from a bucket or a pipe. Pick the one that suits what you are building: a barbecue, a workshop wall, or the end of a garden.

11.5 Burning it

A ladder of power plants ending at the Gas Turbine, plus the **Portable Generator** for a build site.

11.6 The forecourt

A **Gas Station Pump**, a **Forecourt Price Sign** and a nozzle that binds to a pump within eight blocks. Fuel is dispensed to a person rather than to a machine.

Careful

Nothing past the Gas Turbine has been extensively playtested. The arithmetic is unit-tested and the server starts clean, but the balance of the later chain is still unproven.

CHAPTER 12

The Virtual Fluid Network

What the virtual grid is for power, this is for fluid: it moves a fluid between places with no pipe between them.

This fork

Not in stock Immersive Engineering. It is deliberately built as a mirror of the power grid rather than as a generalisation of it — the two have separate save data, so a mistake in one cannot cost you the other.

12.1 Mains

The fluid network is divided into **mains**, which are what segments are to the grid: named groups of fittings with their own switch, limits and statistics.

A main carries **one fluid**. The first inlet to deliver decides which; after that the main refuses anything else until it drains. That is not a limitation to work around — it is what makes a main something you can name "diesel" and trust.

12.2 Fittings

Fluid Inlet Takes fluid from an adjacent tank or pipe and puts it into its main.

Fluid Outlet Draws from its main and fills whatever sits against it. One marked **critical** keeps its supply when the main runs short.

Main Valve Shuts a main by hand or on a redstone signal, and reports whether it is pressurised.

Fluid Control Console Where the network is managed. A separate structure from the power console, because the two networks fail apart.

12.3 The Fluid Control Console

A 2×2 wall of four different blocks, struck anywhere on its front face with an Engineer's Hammer:

	Left	Right
Upper	Fluid Console Housing (the terminal)	Redstone Engineering Block
Lower	Light Engineering Block	Heavy Engineering Block

The same kit of parts, in the same places, as the Grid Management Console: the terminal is the screen, the redstone block is the instrument rack beside it, and the light and heavy blocks are the desk and the pump cabinet under them. The finished console is drawn as one piece —

one desk, one monitor spanning both blocks, its gauges and mains running edge to edge — and breaking it returns the four components.

Four tabs: an overview, the mains themselves, the fittings on them, and a minute of throughput history. It runs on a trickle of Flux; losing power darkens the screen and makes the window read-only rather than refusing to open.

Tip

A console built before this fork's console rework was laid out the other way round and only half of it comes apart. Break it and rebuild it once; there is nothing to migrate but the four blocks.

12.4 The Fluid Linker

The Grid Linker's twin, and the same tool pointed at this network. Right-click a fitting: a short list of mains appears, with colours and what each is carrying. Pick one and it links that fitting and loads the tool; every right-click after that links the fitting you clicked. Sneak-right-click a fitting to choose again, or the air to empty the tool.

Tip

A Fluid Inlet has always taken a pipe run straight into any of its faces, which is why this network never needed the power side's second half of this change. What it did lack was a way to assign a hundred fittings without a hundred walks to a console.

12.5 Failover and schedules

As the grid: a main may name a backup, and may carry a schedule that can hold it closed but never open one somebody shut.

12.6 City mode

Mains stop accounting for millibuckets and switch to presence — a main is pressurised or it is not.

CHAPTER 13

Conduits

Wire sags. Strung across a valley that is exactly right; strung along a ceiling it is exactly wrong. **Conduit** is the indoor answer: thin tubing that clips flat against whatever face you place it on and runs in that face, turning in right angles.

This fork

Not in stock Immersive Engineering. It takes the bundled-cable idea from SimpleLogic and puts it in IE's idiom — surface conduit, junction boxes, and ordinary IE connectors at the breakouts.

One length of conduit carries **sixteen** independent conductors, named after the sixteen dyes.

13.1 Runs

Place conduit against a wall, floor or ceiling and it clips to whatever you clicked. Lengths joined along the same surface make a **run**.

To continue a run, click the end of it. A conduit placed against another conduit joins that conduit's surface rather than clipping to the conduit itself, so you can lay a corridor by clicking along the run instead of hunting for the bare wall between lengths. The first length off a junction box does the same thing — it looks for the surface behind it rather than clipping to the box.

A run wraps around corners. It reaches a wall and climbs it; it reaches the end of the beam it is running along and follows that beam's end face down. Neither needs a junction box, and both happen between two lengths of conduit.

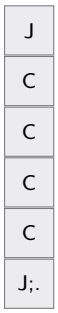
An **inner** corner — the floor meeting the wall — is turned inside the lower length's own cell, so it needs a block in the corner to turn around. A doorway at the foot of the wall leaves nothing to turn around, and the run stops there. An **outer** corner — the end of the beam — needs nothing: the two lengths are diagonal neighbours bolted to two faces of the same block.

Click anywhere sensible and you get the piece that turns the corner: the wall the run is heading for, the far side of the edge it is running along, or the exposed face of the last length.

A run still arrives at a junction box *face on* rather than around a corner. A box is a cube in the middle of its cell, so put the box **at** the corner instead — it sits in the plane of whichever run reaches it, and both runs meet it flush.

Runs are discovered, not drawn. Laying conduit between two boxes creates the connection; pulling a length out of the middle breaks it. There is no coil and no linking tool — the run is the thing you built.

A run seen from above, on a floor



C = conduit, J = junction box. The run turns in the plane of the surface it is clipped to, and wraps onto the next surface where that one stops.

13.2 The Junction Box

A patch panel with six faces. Each face is one of three things:

- **Sneak and hit a face with an Engineer's Hammer**: steps that face to the next conductor, and after the last one takes the breakout away entirely. Colours already spent on another face of the same box are stepped over — the same conductor arriving at two connectors is a short, not a circuit.
- Right-click with a **dye**: that face breaks out the conductor of that colour, in one hit rather than by cycling to it.
- Right-click a patched face with **redstone dust**: cycles it between power, reading redstone, and emitting redstone.
- Sneak and right-click empty-handed: unpatches the face.

A breakout reaches the block edge, wearing its conductor's colour at the mouth — so a box tells you how it is wired from across the room, and whatever is bolted against that face meets it flush rather than standing three pixels clear of it. A box grows the same stub, uncoloured, toward anything else set down against it, so one sunk into a wall reads as part of the wall.

String an LV, MV or HV wire straight to a *bare* face — or put a Connector of that tier, or a Grid Feed or Service Unit, against one — and the box breaks a conductor out to it by itself. That is all most circuits need; the dye and the hammer are how you choose *which* conductor rather than whether there is one.

Which colour it picks is fixed, not arbitrary. Looking at a box: **red** on the left, **blue** in the middle — on top, for a box on a pole — and **green** on the right. West and north take red, east and south take green, up and down take blue. Two boxes wired the same way therefore

come out the same, which is what makes a row of poles readable from the ground. A fourth breakout, or one whose colour is already spent elsewhere on that box, falls back to the lowest free conductor.

Whatever is on a power breakout picks up that conductor and nothing else. One bundle down a corridor can feed a lighting circuit, a workshop and a furnace, each on its own colour.

13.2.1 Wires straight to a face

A junction box is a wire endpoint in its own right: point a coil at the face you want and the wire attaches there, with no connector bolted to the wall beside it.

- **One wire per face.** A face is one breakout on one conductor, so six faces are six wires and six separate circuits.
- **The face is the face you clicked.** Nothing has to be configured afterwards — the gesture already said which circuit you meant.
- **Not the face the box is bolted to.** The housing lies flush against that one. A box no run reaches stands on the floor of its cell, so that is the face it refuses.
- **LV, MV and HV only.** Structural cable holds things up and redstone wire carries no flux; neither has anything to do on a conductor.
- **Wirecutters clicked on a face cut that face's wire**, and leave the other five alone. The face is then free for another wire, and the breakout stays patched where it was.

Runs are never cut this way. A run is made by laying conduit and removed by breaking it, which is the whole of how one is edited.

Tip

The connectors have not gone anywhere. Three of them round a box, with the runs buried, is what an underground feeder looks like — and a face happily serves a wire and an adjacent connector at once, since the two are the same conductor arriving by two routes. What the neighbour takes is gone before the wire is offered anything, so it is never a second helping.

Tip

Auto-patching only answers to wiring hardware — a connector or a grid box. A junction box dropped beside a capacitor bank to turn a corner will not quietly start draining the run into it, and a box whose sixteen conductors are all spoken for hands out nothing rather than stealing one from a working circuit.

Tip

There is no tier setting. The tier of a circuit is whatever hardware is on its face — the wire you string to it, or the connector you hang on it — because IE's wires and connectors already cap throughput by tier. An HV wire on one face and an LV wire on another are two circuits of two tiers coming out of the same bundle.

A colour lives on one face at a time — patching it somewhere new takes it off wherever it was, because the same conductor arriving at two connectors is a short.

A box passes the whole bundle through whatever is patched, so one dropped in to turn a corner or change surface needs no configuration at all.

13.3 The Ground Feeder

A junction box is the right way to leave a surface indoors, where it reads as hardware bolted to a wall. Out in a field it is a steel box sitting in a lawn, and there is no surface to change *to* until the run is already underground.

The **Ground Feeder** is the other answer: a whole cube that fills a hole in a floor, a wall or the ground, passes a bundle straight through along one axis, and **draws itself as whatever is around it**.

It looks at what is visible rather than at what is merely nearby. A feeder set into a grass field has eight grass blocks around it and nine dirt blocks underneath, so counting neighbours would put dirt in the middle of a lawn. Instead a block scores triple if any face of it can be seen at all, and double if it shares a face with the feeder rather than meeting it at a corner. Turf wins on a lawn; stone wins in a cellar wall, because down there the stone is what is exposed to the shaft.

Right-click a feeder with any full block to pin that block instead. Sneak and right-click it empty-handed to hand it back to the survey.

Tip

It can only wear full, opaque, ordinary cubes. A feeder is a cube and always will be, so wearing a fence would draw a fence with a solid hitbox and wearing a chest would draw a chest whose lid never opens — both read as a bug rather than as a disguise.

A run passes through along **one axis**, which an Engineer's Hammer turns. A feeder laid across the grain of a run stops it exactly as a stone block would, and stops it visibly: the conduit does not draw its arm into one it cannot enter.

Crossing a feeder changes surface without a corner. A run turns corners on its own, but a corner is two faces of one block; a feeder is what lets a run come out of a floor on a surface with no relation to the one it went in on. The conduit on the far side still has to have the crossing in its own plane — the same thing a junction box asks — so a run can come down a north wall and continue down an east one, but it cannot continue into conduit lying flat on the floor of the shaft.

A feeder is also *passed through* rather than climbed. It is a solid cube, so a run reaching one would otherwise take it for a wall to turn up.

A run leaving a building, seen from the side

C = conduit on a wall, G = ground feeder, J = junction box. The feeder fills the floor; the run crosses it and carries on below.

A feeder **splits nothing** — it carries the whole bundle. A breakout is a thing you should be able to see, and a patched box wearing coloured plates is exactly that; hiding one would leave a base whose wiring could only be found by digging.

Tip

A feeder is scenery, not hardware. It is never a node in the wire graph, holds no energy and costs nothing per tick — a run crossing four of them is still one connection between the same two boxes, four blocks longer.

13.4 Redstone channels

A face set to **read** puts the signal arriving at it onto its conductor; a face set to **emit** puts its conductor's signal back out. The signal reaches every box on the run, so a lever at one end of a corridor throws a lamp at the other along the same bundle already carrying the lighting.

A face does one of the three and never two. Reading and emitting on one face is how a redstone network latches itself on and never lets go.

13.5 Capacity and cost

Each conductor carries what a conductor carries. A bundle is not a shared pipe with an allowance to divide up: sixteen circuits down one corridor get sixteen circuits' worth of throughput.

Sixteen conductors are *one* connection in the wire graph rather than sixteen, and a junction box carrying nothing costs one integer comparison per tick. Replacing sixteen catenaries with one conduit costs nothing but the crafting.

Energy moves as a bucket brigade: each box hands half the difference to whichever neighbour holds less, and drains in full into any connector on the matching breakout. The honest cost of that is one tick per hop, and boxes exist only at corners and ends.

CHAPTER 14

Pole Signage

A hundred poles across a map are a hundred identical poles. The **Utility Pole Sign** is the tag that tells them apart: which station feeds this one, which feeder it is on, who last climbed it.

This fork

Not in stock Immersive Engineering, and not invented either. All thirteen kinds are signs that hang on real poles — the shapes, the colours and what each one means are the *Los Angeles Department of Water and Power*'s and *Southern California Edison*'s. They are told apart the way the real ones are: by shape and colour, long before you are close enough to read the number.

One item, one recipe, thirteen plates. Which plate a sign is lives on the sign, not on the item, so there is nothing to hunt for in a creative tab and nothing to craft twice.

14.1 Hanging one

Place it against the **side** of whatever holds the wires up — a pole, a crossarm, a wall. The plate bolts flat to that face and reads from the outside. There is no top or bottom placement: a tag lying on a floor is not a thing that exists, and a sign whose support is broken comes down with it, the way a torch does.

Signs have no collision box. Walking into a pole covered in them is walking into the pole.

14.2 Choosing the plate

Hit a sign with an Engineer's Hammer and it steps to the next of the thirteen. Keep hitting and it walks the whole set and comes back round — one gesture, no menu, and it works from a ladder.

Sneak and hit it with the hammer and the plate opens for writing:

- The **arrows** pick the kind, the same set the hammer walks.
- One **text box** per line that kind carries — one, two or three, or none at all for the line crossing diamond, which is a symbol rather than a label.
- The **preview** shows the real plate with the real lettering on it, laid out by exactly the arithmetic the world uses. What is in that window is what ends up on the pole.

Enter and Tab step between the lines. Done and Escape both keep what was typed.

Tip

Text is **printed to fill the plate** rather than typeset at a fixed size. A short number comes out big and a long one comes out small, which is what the real ones do and why a pole number and a station name can share a plate the size of a hand. The vertical strips read downwards, because a strip six pixels wide holds a pole number no other way.

A sign taken down keeps its plate and its text, so hammering thirteen times to find the one you meant and then mining it by accident costs nothing.

14.3 What the kinds mean

Parallel Generation The horizontal red strip. Several sources are feeding the distribution station this pole hangs off, or power is running in a loop across the area. Mainly a LADWP sign, and the only one in the set that carries white text. Two lines.

Pole Number The yellow vertical strip: general pole identification, and the one every utility uses. One line, read downwards.

Street Light The white vertical strip. SCE hangs these for street lighting, and so do privately owned poles. One line, read downwards.

Old Pattern The bare metal vertical strip — mostly replaced by something more legible, and still on plenty of poles. One line, read downwards, in grey.

Series Light Oval The painted white oval a series-wired street light wears: the series number on top and the pole number underneath.

Feeder Tag The horizontal yellow strip. For the LADWP this is the distribution station, the feeder off it, and how many conduit or transformer bank connections are on that connection; on a light pole it is the capacitor bank or the phase cutoff. Everybody else uses it for pole numbers.

Fibre Run The orange strip, horizontal or vertical, marking fibre and cable runs. Almost every utility uses these.

Pole Inspection Tag The round bolt-on tag: who inspected the pole on top, the year underneath. It also does duty as the tag saying where a wooden pole was logged.

Tower Number The plain yellow diamond the LADWP numbers transmission towers with, roughly every fourth tower. No border — the number is painted straight onto it.

Line Crossing The same diamond with a black outline and a black cross, marking where a line crosses another, or a span nobody wants to walk: a valley, a ravine. It carries no text.

Tower Identifier The tower run, in two forms. The vertical one reads downwards in three parts — the initials of the plant the run starts at, the tower's number, then, under a rule printed on the plate, the initials of the station it ends at. The horizontal one is one line, and is what the DC towers of the Pacific Intertie carry.

14.4 What a sign costs to look at

Nothing you will notice. The plate is an ordinary block model baked into the chunk mesh like any other — one of fifty-two, thirteen plates times four facings — so a pole line of them costs what a pole line of blocks costs.

Only the *lettering* is drawn per frame, and only within forty-eight blocks. Past that the plate is a pixel or two across and the number was never legible anyway, so nothing is drawn at all. A blank sign draws nothing at any distance.

CHAPTER 15

The Hydraulic Crawler

A tracked excavator you climb into and drive, with three attachments and a demolition tool on the end of the arm.

This fork

Entirely this fork, and its first vehicle.

Tip

This is *not* the Excavator in chapter 7. That one is a bucket-wheel multiblock that mines ore from a mineral vein and does not move. This one is a machine you sit in.

15.1 Getting one

Craft the Crawler and place it on level ground with about five blocks clear — it is a large machine and it will refuse to go anywhere it does not fit, saying so rather than silently doing nothing. Right-click to climb in. Sneak and strike it with an Engineer's Hammer to take it away again.

15.2 Driving

- **W** and **S** drive; **A** and **D** steer the tracks, which turn on the spot the way tracks do.
- **Looking around slews the house** — the cab, the counterweight and the arm all turn with your head, at the speed a slew ring turns rather than instantly. You cannot look around independently of the machine, exactly as you cannot in the real thing.
- **R** and **C** raise and lower the arm.
- **X** and **Z** reach out and draw back in.

It is a heavy machine and it is meant to feel like one. The throttle takes about a second to reach full speed and rather less than that to stop; it brakes harder than it accelerates, so it does not coast about after you have let go. It steers tightest standing still and describes an arc at speed, which is what a skid steer does. A one-block ledge is climbed rather than bumped into, at the cost of some of the speed you arrived with.

The arm is *aimed* rather than jointed: one pair of keys says which way it points, the other how far along that line the bucket sits, and the boom and stick work out their own angles. Between them they cover a band from about two blocks out to nearly six, and from five blocks up to nearly three below the tracks.

Tip

The panel in the corner shows the arm's height, its reach, the fuel, and the list of controls.

15.3 Attachments

G changes the tool on the end of the arm, **V** works it.

- The **Bucket** digs and keeps what it digs, in nine slots inside the machine. Sneak and press **V** to tip it out; a pipe can also unload it.
- The **Grapple** picks up one block — or one animal — and puts it down again. A block goes back exactly as it came up, facing and all.
- The **Breaker** destroys whatever it touches and leaves it on the ground.

Careful

The Breaker is a demolition tool and behaves like one. It respects land claims and protected areas exactly as your own pickaxe does — every block it takes is on your account — but within your own build it will happily take a wall down faster than you can change your mind.

15.4 Fuel

The Crawler runs on **diesel** and holds eight buckets. Refuel it by hand with any container, or drive it up to a Gas Station Pump.

Idling costs a little and working costs a lot, so a tank goes a long way if you are only driving. When the fuel runs low the *attachment* stops but the *tracks keep turning* — that last reserve is deliberately kept back so you can always drive home rather than being stranded next to a half-demolished building.

CHAPTER 16

City mode

City mode trades Immersive Engineering's simulation detail for server tick time. It is aimed at city and roleplay packs where the mod's machinery is set dressing rather than an engineering puzzle: you keep the entire look of the build and give up the physics behind it.

This fork

Entirely this fork. **It is off by default**, and off means byte-identical to stock.

It covers eight subsystems, each with its own switch. A subsystem is simplified only when the master switch is on *and* that subsystem has not been individually turned back off, so switching the master off is always sufficient to restore stock behaviour.

Wires One lossless push per connector instead of loss, distance weighting, a proportional split and a network-wide broadcast.

Fluid pipes A pipe hands its fluid to the endpoints in order rather than simulating a fill against each and splitting in proportion. Which of several tanks fills first is the only observable difference.

Floodlights Beams re-trace only when the light switches or a neighbour changes, never on a timer, and the light blocks one lamp may place are capped.

Generators Fuel becomes cosmetic — a presence check and a token sip.

Machines Idle multiblocks stop re-scanning the recipe list every tick.

Virtual grid Segments switch from accounting to presence.

Conduits Bundles switch from accounting to presence.

Tanks A tank holding any fluid at all becomes an unlimited source of it. Empty is still empty, and filling is untouched.

16.1 What you give up

The numbers stop being conservation. A segment can deliver more than its feeds collected, because nothing is being collected — the feed is only being checked. The console's Stats tab says so directly rather than letting the graphs be misread.

Careful

City mode is a decision about what your server is for. If you want the arithmetic to be honest, leave the individual subsystem off while the master switch is on; they are independent.

16.2 The performance history

Roughly half of everything the mod did on a profiled server turned out to be a single per-tick network broadcast feeding wire-shock damage. That is fixed for everyone, in both modes. The full account, with measurements, is in `docs/CITY_MODE_AND_PERF.md`.

Part IV

Reference

CHAPTER 17

Every block

Generated from the block metadata enums in the source tree. Metadata order is save-file order: a block's number here is the number in the world.

There are 195 block states across 28 block registrations.

17.0.1 ClothDevice

Registry name `cloth_device`. 4 metadata values.

Meta	Registry	Name	Notes
0	`cushion`	Jump Cushion	—
1	`balloon`	Balloon	—
2	`stripcurtain`	Strip Curtain	—
3	`shader_banner`	*unnamed*	—

17.0.2 Connector

Registry name `connector`. 15 metadata values.

Meta	Registry	Name	Notes
0	`connector_lv`	LV Wire Connector	—
1	`relay_lv`	LV Wire Relay	—
2	`connector_mv`	MV Wire Connector	—
3	`relay_mv`	MV Wire Relay	—
4	`connector_hv`	HV Wire Connector	—
5	`relay_hv`	HV Wire Relay	—
6	`connector_structural`	Structural Cable Connector	—
7	`transformer`	Transformer	—
8	`transformer_hv`	HV Transformer	—
9	`breakerswitch`	Breaker Switch	—
10	`redstone_breaker`	Redstone Breaker	—
11	`energy_meter`	Current Transformer	—
12	`connector_redstone`	Redstone Wire Connector	—
13	`connector_probe`	Redstone Probe Connector	—
14	`feedthrough`	Feedthrough Insulator	—

17.0.3 FakeLight

Registry name `fake_light`. 1 metadata value.

Meta	Registry	Name	Notes
0	`air`	*unnamed*	—

17.0.4 FluidNetDevice

Registry name `fluid_net_device`. 4 metadata values.

Meta	Registry	Name	Notes
0	`fluid_inlet`	*unnamed*	—
1	`fluid_outlet`	*unnamed*	—
2	`main_valve`	*unnamed*	—
3	`console_housing`	*unnamed*	—

17.0.5 FluidNetMultiblock

Registry name `fluid_net_multiblock`. 1 metadata value.

Meta	Registry	Name	Notes
0	`fluid_console`	*unnamed*	—

17.0.6 GridDevice

Registry name `grid_device`. 2 metadata values.

Meta	Registry	Name	Notes
0	`feed_unit`	Grid Feed Unit	Hands Immersive Flux to a grid segment. String a wire straight to the terminal post on its front, put a cable or connector against it, or bolt it onto a capacitor or generator; the…
1	`service_unit`	Grid Service Unit	Draws Immersive Flux out of a grid segment and pushes it into whatever sits against it, and out along any wire strung to its terminal post. Assign it to a segment from a Grid Manag…

17.0.7 GridMultiblock

Registry name `grid_multiblock`. 1 metadata value.

Meta	Registry	Name	Notes
0	`grid_console`	Grid Management Console	Where the power grid is managed: segments are created, named, coloured and priced here, feed and service units are assigned to them, and schedules and failover are set up. Every ch…

17.0.8 Hemp

Registry name `hemp`. 6 metadata values.

Meta	Registry	Name	Notes
0	`bottom0`	*unnamed*	—
1	`bottom1`	*unnamed*	—
2	`bottom2`	*unnamed*	—
3	`bottom3`	*unnamed*	—
4	`bottom4`	*unnamed*	—
5	`top0`	*unnamed*	—

17.0.9 MetalDecoration0

Registry name `metal_decoration0`. 8 metadata values.

Meta	Registry	Name	Notes
0	`coil_lv`	Copper Coil Block	—
1	`coil_mv`	Electrum Coil Block	—
2	`coil_hv`	High-Voltage Coil Block	—
3	`rs_engineering`	Redstone Engineering Block	—
4	`light_engineering`	Light Engineering Block	—
5	`heavy_engineering`	Heavy Engineering Block	—
6	`generator`	Generator Block	—
7	`radiator`	Radiator Block	—

17.0.10 MetalDecoration1

Registry name `metal_decoration1`. 8 metadata values.

Meta	Registry	Name	Notes
0	`steel_fence`	Steel Fence	—
1	`steel_scaffolding_0`	Steel Scaffolding	—
2	`steel_scaffolding_1`	Steel Scaffolding	—
3	`steel_scaffolding_2`	Steel Scaffolding	—
4	`aluminum_fence`	Aluminium Fence	—
5	`aluminum_scaffolding_0`	Aluminium Scaffolding	—
6	`aluminum_scaffolding_1`	Aluminium Scaffolding	—
7	`aluminum_scaffolding_2`	Aluminium Scaffolding	—

17.0.11 MetalDecoration2

Registry name `metal_decoration2`. 9 metadata values.

Meta	Registry	Name	Notes
0	`steel_post`	Steel Post	—
1	`steel_wallmount`	Steel Wallmount	—
2	`aluminum_post`	Aluminium Post	—
3	`aluminum_wallmount`	Aluminium Wallmount	—
4	`lantern`	Lantern	—
5	`razor_wire`	Razor Wire	—
6	`toolbox`	Engineer's Toolbox	—
7	`steel_slope`	Steel Structural Arm	—
8	`alu_slope`	Aluminium Structural Arm	—

17.0.12 MetalDevice0

Registry name `metal_device0`. 7 metadata values.

Meta	Registry	Name	Notes
0	`capacitor_lv`	LV Capacitor	—
1	`capacitor_mv`	MV Capacitor	—
2	`capacitor_hv`	HV Capacitor	—
3	`capacitor_creative`	Creative Capacitor	—
4	`barrel`	Metal Barrel	—
5	`fluid_pump`	Fluid Pump	—
6	`fluid_placer`	Fluid Outlet	—

17.0.13 MetalDevice1

Registry name `metal_device1`. 14 metadata values.

Meta	Registry	Name	Notes
0	`blast_furnace_preheater`	Blast Furnace Preheater	—
1	`furnace_heater`	External Heater	—
2	`dynamo`	Kinetic Dynamo	—
3	`thermoelectric_gen`	Thermoelectric Generator	—
4	`electric_lantern`	Powered Lantern	—
5	`charging_station`	Charging Station	—
6	`fluid_pipe`	Fluid Pipe	—
7	`sample_drill`	Core Sample Drill	—
8	`tesla_coil`	Tesla Coil	—
9	`floodlight`	Floodlight	—
10	`turret_chem`	Chemthrower Turret	—
11	`turret_gun`	Gun Turret	—
12	`turret_rail`	*unnamed*	—
13	`belljar`	Garden Cloche	—

17.0.14 MetalLadder

Registry name `metal_ladder`. 3 metadata values.

Meta	Registry	Name	Notes
0	`normal`	Metal Ladder	—
1	`covered_steel`	Scaffold Covered Ladder	—
2	`covered_aluminum`	Scaffold Covered Ladder	—

17.0.15 MetalMultiblock

Registry name `metal_multiblock`. 16 metadata values.

Meta	Registry	Name	Notes
0	`metal_press`	Metal Press	—
1	`crusher`	Crusher	—
2	`tank`	Fluid Tank	—
3	`silo`	Item Silo	—
4	`assembler`	Assembler	—
5	`auto_workbench`	Automated Engineer's Workbench	—

Meta	Registry	Name	Notes
6	`bottling_machine`	Bottling Machine	—
7	`squeezer`	Industrial Squeezer	—
8	`fermenter`	Industrial Fermenter	—
9	`refinery`	Refinery	—
10	`diesel_generator`	Diesel Generator	—
11	`excavator`	Excavator	—
12	`bucket_wheel`	Bucket Wheel	—
13	`arc_furnace`	Arc Furnace	—
14	`lightningrod`	Lightning Rod	—
15	`mixer`	Mixer	—

17.0.16 MetalsAll

Registry name `metals_all`. 11 metadata values.

Meta	Registry	Name	Notes
0	`copper`	*unnamed*	—
1	`aluminum`	*unnamed*	—
2	`lead`	*unnamed*	—
3	`silver`	*unnamed*	—
4	`nickel`	*unnamed*	—
5	`uranium`	*unnamed*	—
6	`constantan`	*unnamed*	—
7	`electrum`	*unnamed*	—
8	`steel`	*unnamed*	—
9	`iron`	*unnamed*	—
10	`gold`	*unnamed*	—

17.0.17 MetalsIE

Registry name `metals_i_e`. 9 metadata values.

Meta	Registry	Name	Notes
0	`copper`	*unnamed*	—
1	`aluminum`	*unnamed*	—
2	`lead`	*unnamed*	—
3	`silver`	*unnamed*	—
4	`nickel`	*unnamed*	—
5	`uranium`	*unnamed*	—
6	`constantan`	*unnamed*	—
7	`electrum`	*unnamed*	—
8	`steel`	*unnamed*	—

17.0.18 Ore

Registry name `ore`. 6 metadata values.

Meta	Registry	Name	Notes
0	`copper`	Copper Ore	—
1	`aluminum`	Bauxite Ore	—
2	`lead`	Lead Ore	—
3	`silver`	Silver Ore	—
4	`nickel`	Nickel Ore	—

Meta	Registry	Name	Notes
5	`uranium`	Uranium Ore	—

17.0.19 PetroleumDecoration

Registry name `petroleum_decoration`. 4 metadata values.

Meta	Registry	Name	Notes
0	`asphalt`	Asphalt	Road. Put 250 mB of bitumen through a Mixer with sand and gravel for 500 mB of wet asphalt, then pour it – it flows, finds its own level and sets a moment later into a full block....
1	`asphalt_tile`	Asphalt Tile	A tiled variant of asphalt, for forecourts and yards rather than open road. Cosmetic.
2	`asphalt_marked`	Asphalt with Markings	Asphalt with lane markings painted on it. Cosmetic.
3	`forecourt_canopy`	Forecourt Canopy	The roof a filling station stands under. Cosmetic.

17.0.20 PetroleumDevice

Registry name `petroleum_device`. 13 metadata values.

Meta	Registry	Name	Notes
0	`wellhead`	Wellhead	What a Drilling Derrick leaves standing over a finished bore. From then on it is the whole of the well. It buffers a couple of seconds of production and pushes it into anyth...
1	`oilfield_frame`	Oilfield Frame	The structural part every oilfield rig is built out of, and the stack a finished rig packs itself back into. None of the oilfield machinery is crafted as a block and placed....
2	`flare_stack`	Flare Stack	Stood against a Wellhead, it burns off the gas nothing else wants. It takes gases only – it will not accept crude, so it is no way to make a spill disappear – destroys wha...
3	`lubrication_manifold`	Lubrication Manifold	Where lubricant goes. Bolt it against any of the mod's larger machines, fill it, and that machine costs noticeably less to run. It only draws while its host actually has som...
4	`propane_cylinder`	Propane Cylinder	A bottle of propane you fill somewhere and carry to wherever it is needed. Holds 4,000 mB. Right-click it with a fluid container to fill or empty it. The Upright and Torpedo...

Meta	Registry	Name	Notes
5	`tank_fill_cap`	Tank Fill Cap	The only part of a buried tank that shows above ground. A pipe connects to it, a comparator reads it, and its gauge is the whole of the tank's readout. Dig the hole, line it...
6	`gas_pump`	Gas Station Pump	Hands fuel to a person rather than to a machine. Stack two of them and strike one with an Engineer's Hammer: they become a single bowser two blocks tall, with the price on its own ...
7	`forecourt_sign`	Forecourt Price Sign	A price board for a forecourt. It has no price of its own – it finds the nearest Gas Station Pump and shows that one's.
8	`portable_generator`	Portable Generator	One block, 256 Flux a tick, four buckets of gasoline or ethanol, about five minutes of running. It is what gasoline is for: a Diesel Generator will not touch gasoline, becau...
9	`reinjection_well`	Re-injection Well	Puts water or gas back downhole and gets a second tranche out of a tiring field. The thinking player's answer to a Flare Stack. Water is cheap and poor; scrubbed natural gas...
10	`storage_tank_wall`	Storage Tank Wall	Build a hollow box of these – any rectangular shape from 3x3x3 up to 16x16x16 – and strike any wall with an Engineer's Hammer. It becomes one tank holding 16 buckets per enclosed...
11	`propane_tank_upright`	Upright Propane Tank	A standing propane tank holding 12,000 mB – three cylinders' worth in one block. Right-click it with a fluid container to fill or empty it.
12	`propane_tank_torpedo`	Torpedo Propane Tank	The largest propane body: a horizontal torpedo tank holding 24,000 mB. It stops deliberately short of the buried Domestic Tank's 32,000: above this size, fuel belongs underg...

17.0.21 PetroleumMultiblock

Registry name `petroleum_multiblock`. 15 metadata values.

Meta	Registry	Name	Notes
0	`derrick`	Drilling Derrick	Sinks the bore: a 3x3 lattice nine blocks tall, hammered together like any other structure. Power goes into the drill floor – the whole bottom course takes a connector on a...
1	`pumpjack`	Pumpjack	What a tired well needs. Build it so its well bay lands over an existing Wellhead – the bay is not a part you supply, and forming the machine will never swallow the well it exists...

Meta	Registry	Name	Notes
2	`distillation_tower`	Distillation Tower	Where crude becomes worth something: a 2x2 steel column twelve blocks tall on a scaffolding deck, with nozzles out of the shell in pairs at seven heights. The nozzle heights…
3	`industrial_burner`	Industrial Burner	A 3x3 firebox of blast brick under a steel sheetmetal crown, with a burner head where the fuel line lands. It makes no Flux at all, and that is the point of it. It makes hea…
4	`gas_scrubber`	Gas Scrubber	Turns flared sour gas into two products: twin vessels six blocks tall on a 3x3 skid, with an alley between them to stand and plumb in. Height is the interface. Sour gas ente…
5	`gas_turbine`	Gas Turbine	Burns natural gas or propane for Flux: three blocks high and six long, filter house to nacelle to alternator to exhaust stack. Gas goes in anywhere along the bottom course; …
6	`fuel_oil_boiler`	Fuel Oil Boiler	The furnace half of a power station. It makes no Flux at all – it makes steam. That is what finally gives heavy fuel oil a job at scale. Crude and diesel burn in it too and…
7	`hrsg`	Heat Recovery Steam Generator	Burns nothing whatsoever. Butt it against a Gas Turbine's exhaust end and it makes steam out of heat that was going up the stack regardless. One HRSG claims one turbine's ex…
8	`steam_turbine_hall`	Steam Turbine Hall	The largest thing in the chapter and the most efficient. Steam in at the condenser end, Flux out at the switchyard. It is slow to start and it hates being cycled. Feed it st…
9	`engine_bank`	Reciprocating Engine Bank	The honest workhorse: diesel, biodiesel or gas, instant response, mediocre efficiency. It is the only plant here that scales by building another one. Put a second bank along…
10	`domestic_tank`	Domestic Tank	2x2x2 of iron sheetmetal under the garden, holding thirty-two buckets. The cheap one on purpose: the first thing to build once you have a fuel worth keeping, rather than the…
11	`commercial_tank`	Commercial Tank	A 5x5 hole four deep, lined in steel, holding exactly what a Sheetmetal Tank holds. Matching that is the point. This is not a bigger store – it is the same store out of sig…
12	`bulk_depot`	Bulk Depot	Nine by nine and six deep, holding four million millibuckets: a refinery's own stock, or a town's. Nearly eight times the commercial tank for three and a half times the bloc…

Meta	Registry	Name	Notes
13	`loading_gantry`	Fluid Loading Gantry	Filling one jerrycan by hand is fine. Filling eleven is not. It takes empty containers out of the inventory on one side, fills them, and puts them into the inventory on the ...
14	`cracking_unit`	Cracking Unit	Breaks the heavy cuts into the light ones. Heavy fuel oil or naphtha goes in at the deck; gasoline leaves along the front of the head and diesel along the back, and the row between...

17.0.22 Signage

Registry name `signage`. 1 metadata value.

Meta	Registry	Name	Notes
0	`utility_sign`	Utility Pole Sign	A tag bolted flat to a pole, in thirteen kinds – the ones the LADWP, SCE and their neighbours really hang. Place it against the side of whatever holds the wires up. Hit it with an...

17.0.23 StoneDecoration

Registry name `stone_decoration`. 11 metadata values.

Meta	Registry	Name	Notes
0	`cokebrick`	Coke Brick	—
1	`blastbrick`	Blast Brick	—
2	`blastbrick_reinforced`	Reinforced Blast Brick	—
3	`coke`	Block of Coal Coke	—
4	`hempcrete`	Hempcrete	—
5	`concrete`	Concrete	—
6	`concrete_tile`	Concrete Tile	—
7	`concrete_leaded`	Leaded Concrete	—
8	`insulating_glass`	Insulating Glass	—
9	`concrete_sprayed`	Quickdry Concrete	—
10	`alloybrick`	Kiln Brick	—

17.0.24 StoneDevices

Registry name `stone_devices`. 8 metadata values.

Meta	Registry	Name	Notes
0	`coke_oven`	*unnamed*	—
1	`blast_furnace`	*unnamed*	—
2	`blast_furnace_advanced`	*unnamed*	—
3	`concrete_sheet`	*unnamed*	—
4	`concrete_quarter`	*unnamed*	—
5	`concrete_threequarter`	*unnamed*	—
6	`coresample`	*unnamed*	—
7	`alloy_smelter`	*unnamed*	—

17.0.25 TreatedWood

Registry name `treated_wood`. 3 metadata values.

Meta	Registry	Name	Notes
0	`horizontal`	Treated Wood Planks	—
1	`vertical`	Treated Wood Planks	—
2	`packaged`	Treated Wood Planks	—

17.0.26 WoodenDecoration

Registry name `wooden_decoration`. 2 metadata values.

Meta	Registry	Name	Notes
0	`fence`	Treated Wood Fence	—
1	`scaffolding`	Treated Wood Scaffolding	—

17.0.27 WoodenDevice0

Registry name `wooden_device0`. 8 metadata values.

Meta	Registry	Name	Notes
0	`crate`	Wooden Storage Crate	—
1	`barrel`	Wooden Barrel	—
2	`workbench`	Engineer's Workbench	—
3	`sorter`	Item Router	—
4	`gunpowder_barrel`	Gunpowder Barrel	—
5	`reinforced_crate`	Reinforced Storage Crate	—
6	`turntable`	Turntable	—
7	`fluid_sorter`	Fluid Router	—

17.0.28 WoodenDevice1

Registry name `wooden_device1`. 5 metadata values.

Meta	Registry	Name	Notes
0	`watermill`	Water Wheel	—
1	`windmill`	Windmill	—
2	`windmill_advanced`	Improved Windmill	—
3	`post`	Wooden Post	—
4	`wallmount`	Wooden Wallmount	—

CHAPTER 18

Every recipe

Generated from the recipe JSON. Entries beginning with a lower-case word rather than an item name are ore-dictionary entries, which accept any mod's equivalent.

416 crafting recipes across 23 groups.

18.0.1 armor

Output	Qty	Recipe
`earmuffs`	1	S S S W W *S = stickIron, W = wool/32767*
`faraday_suit_chest`	1	A A AAA AAA *A = plateAluminum*
`faraday_suit_feet`	1	A A A A *A = plateAluminum*
`faraday_suit_head`	1	AAA A A *A = plateAluminum*
`faraday_suit_legs`	1	AAA A A A A *A = plateAluminum*
`powerpack`	1	LBL WCW *B = metal_device0/0, C = connector/0, L = leather, W = wirecoil/0*
`steel_armor_chest`	1	A A AAA AAA *A = plateSteel*
`steel_armor_feet`	1	A A A A *A = plateSteel*
`steel_armor_head`	1	AAA A A *A = plateSteel*

Output	Qty	Recipe
`steel_armor_legs`	1	AAA A A A A *A = plateSteel*

18.0.2 blueprints

Output	Qty	Recipe
`blueprint`	1	K DDD PPP *D = dyeBlue, K = plateSteel, P = paper*
`blueprint`	1	GGG GDG GPG *D = dyeBlue, G = ingotHOPGraphite, P = paper*
`blueprint`	1	JKL DDD PPP *D = dyeBlue, J = gunpowder, K = ingotCopper, L = gunpowder, P = paper*
`blueprint`	1	JKL DDD PPP *D = dyeBlue, J = ingotCopper, K = ingotAluminum, L = ingotIron, P = paper*

18.0.3 cloth_devices

Output	Qty	Recipe
`cloth_device/0`	3	FFF F F FFF *F = fabricHemp*
`cloth_device/1`	2	F FTF S *F = fabricHemp, S = slabTreatedWood, T = torch*
`cloth_device/2`	3	SSS FFF FFF *F = fabricHemp, S = listMetalSticks*

18.0.4 compat

Output	Qty	Recipe
`plate/0`	1	ingotBrass, tool/0
`plate/2`	1	ingotThaumium, tool/0
`plate/3`	1	ingotVoid, tool/0

18.0.5 conduit

Output	Qty	Recipe
`conduit/0`	8	PPP WWW PPP *P = plateAluminum, W = wireCopper*
`conduit/1`	1	P PCP P *C = conduit/0, P = plateAluminum*
`conduit/2`	1	P C P *C = conduit/0, P = plateAluminum*

18.0.6 connectors

Output	Qty	Recipe
`connector/0`	4	I BIB BIB *B = hardened_clay, I = ingotCopper*
`connector/1`	8	I BIB *B = hardened_clay, I = ingotCopper*
`connector/10`	1	H H ICI IRI *C = repeater, H = connector/4, I = ingotIron, R = dustRedstone*
`connector/11`	1	M BCB ICI *B = hardened_clay, C = copperCoil, I = ingotIron, M = tool/2*
`connector/12`	4	III BRB *B = hardened_clay, I = nuggetElectrum, R = dustRedstone*
`connector/13`	1	C GPG Q *C = connector/12, G = paneGlass, P = material/27, Q = gemQuartz*
`connector/2`	4	I BIB BIB *B = hardened_clay, I = ingotIron*
`connector/3`	8	I BIB *B = hardened_clay, I = ingotIron*

Output	Qty	Recipe
`connector/4`	4	I BIB BIB *B = hardened_clay, I = ingotAluminum*
`connector/5`	8	I BIB BIB *B = stone_decoration/8, I = ingotAluminum*
`connector/6`	8	ISI I I *I = ingotSteel, S = stickSteel*
`connector/7`	1	L M IBI III *B = electrumCoil, I = ingotIron, L = connector/0, M = connector/2*
`connector/8`	1	M H IBI III *B = hvCoil, H = connector/4, I = ingotIron, M = connector/2*
`connector/9`	1	L CIC *C = hardened_clay, I = ingotCopper, L = lever*

18.0.7 conveyors

Output	Qty	Recipe
`conveyor`	1	C T *C = baseConveyor, T = iron_trapdoor*
`conveyor`	1	CIC C *C = baseConveyor, I = ingotIron*
`conveyor`	1	S C *C = baseConveyor, S = scaffoldingSteel*
`conveyor`	1	S C *C = conveyor, S = scaffoldingSteel*
`conveyor`	1	S C *C = conveyor, S = scaffoldingSteel*
`conveyor`	1	S C *C = conveyor, S = scaffoldingSteel*
`conveyor`	1	WS MC *C = baseConveyor, M = componentIron, S = cloth_device/2, W = plankTreatedWood*
`conveyor`	1	baseConveyor
`conveyor`	1	conveyor

Output	Qty	Recipe
`conveyor`	3	CI C CI *C = baseConveyor, I = ingotIron*
`conveyor`	8	LLL IRI *I = ingotIron, L = itemRubber, R = dustRedstone*
`conveyor`	8	LLL IRI *I = ingotIron, L = leather, R = dustRedstone*
`conveyor`	12	S S S S S S *S = blockSheetmetalAluminum*
`conveyor`	12	S S S S S S *S = blockSheetmetalCopper*
`conveyor`	12	S S S S S S *S = blockSheetmetalIron*
`conveyor`	12	S S S S S S *S = blockSheetmetalSteel*

18.0.8 fluidnet

Output	Qty	Recipe
`fluidnet_device/0`	1	SSS PTB SSS *B = componentSteel, P = metal_device1/6, S = plateSteel, T = elecTube*
`fluidnet_device/1`	1	SSS BTP SSS *B = componentSteel, P = metal_device1/6, S = plateSteel, T = elecTube*
`fluidnet_device/2`	1	SSS RTR SSS *R = dustRedstone, S = plateSteel, T = elecTube*
`fluidnet_device/3`	4	SGS WPW SCS *C = componentSteel, G = paneGlass, P = metal_device1/6, S = plateSteel, W = plankTreatedWood*
`network_linker/1`	1	P STS W *P = metal_device1/6, S = plateSteel, T = elecTube, W = stickTreatedWood*

18.0.9 grid

Output	Qty	Recipe
`grid_device/0`	1	SSS CTL SSS *C = copperCoil, L = componentSteel, S = plateSteel, T = elecTube*
`grid_device/1`	1	SSS LTC SSS *C = copperCoil, L = componentSteel, S = plateSteel, T = elecTube*
`grid_device/2`	4	SGS WRW SCS *C = componentSteel, G = paneGlass, R = metal_decoration0/3, S = plateSteel, W = plankTreatedWood*
`grid_device/3`	1	SSS RTR SSS *R = redstone, S = plateSteel, T = elecTube*
`grid_device/4`	4	SPS PRP SCS *C = componentSteel, P = plateAluminum, R = metal_decoration0/3, S = plateSteel*
`network_linker/0`	1	C STS W *C = copperCoil, S = plateSteel, T = elecTube, W = stickTreatedWood*
`network_terminal`	1	A SGS SCS *A = stickTreatedWood, C = componentSteel, G = paneGlass, S = plateSteel*

18.0.10 material

Output	Qty	Recipe
`bullet/0`	5	I I I I I *I = ingotCopper*
`bullet/1`	5	PDP PDP I *D = dyeRed, I = ingotCopper, P = paper*
`gunpowder`	4	dustSaltpeter, dustSaltpeter, dustSaltpeter, dustSaltpeter, dustSulfur, charcoal
`gunpowder`	4	dustSaltpeter, dustSaltpeter, dustSaltpeter, dustSaltpeter, dustSulfur, dustCharcoal

Output	Qty	Recipe
material/0	4	W W *W = plankTreatedWood*
material/1	4	I I *I = ingotIron*
material/10	1	S SBS BSB *B = plankTreatedWood, S = stickTreatedWood*
material/11	1	BB SSB SS *B = plankTreatedWood, S = stickTreatedWood*
material/12	1	CC CCC C *C = fabricHemp*
material/13	1	SS IS SS *I = ingotCopper, S = stickTreatedWood*
material/14	1	II *I = stickSteel*
material/15	1	I ICI I *C = componentIron, I = ingotSteel*
material/16	1	I II II *I = ingotSteel*
material/2	4	I I *I = ingotSteel*
material/20	1	plateCopper, listCutters
material/21	1	plateElectrum, listCutters
material/22	1	plateAluminum, listCutters
material/23	1	plateSteel, listCutters
material/3	4	I I *I = ingotAluminum*
material/5	1	HHH HSH HHH *H = fiberHemp, S = stickWood*
material/8	1	I I C I I *C = ingotCopper, I = plateIron*
material/9	1	I I C I I *C = ingotCopper, I = plateSteel*
metal/15	2	dustCopper, dustNickel
metal/16	2	dustSilver, dustGold
metal/30	1	ingotCopper, tool/0

Output	Qty	Recipe
metal/31	1	ingotAluminum, tool/0
metal/32	1	ingotLead, tool/0
metal/33	1	ingotSilver, tool/0
metal/34	1	ingotNickel, tool/0
metal/35	1	ingotUranium, tool/0
metal/36	1	ingotConstantan, tool/0
metal/37	1	ingotElectrum, tool/0
metal/38	1	ingotSteel, tool/0
metal/39	1	ingotIron, tool/0
metal/40	1	ingotGold, tool/0
string	1	fiberHemp, fiberHemp, fiberHemp
torch	12	WC SSS *C = ?, S = stickWood, W = wool/32767*

18.0.11 metal_decoration

Output	Qty	Recipe
aluminum_scaffolding_stairs0	1	aluminum_scaffolding_stairs2
aluminum_scaffolding_stairs0	4	S SS SSS *S = metal_decoration1/5*
aluminum_scaffolding_stairs1	1	aluminum_scaffolding_stairs0
aluminum_scaffolding_stairs1	4	S SS SSS *S = metal_decoration1/6*
aluminum_scaffolding_stairs2	1	aluminum_scaffolding_stairs1
aluminum_scaffolding_stairs2	4	S SS SSS *S = metal_decoration1/7*
metal_decoration0/0	1	WWW WIW WWW *I = ingotIron, W = wirecoil/0*
metal_decoration0/1	1	WWW WIW WWW *I = ingotIron, W = wirecoil/1*
metal_decoration0/2	1	WWW WIW WWW *I = ingotIron, W = wirecoil/2*
metal_decoration0/3	2	IRI RCR IRI *C = ingotCopper, I = ingotIron, R = dustRedstone*
metal_decoration0/4	2	IGI CCC IGI *C = ingotCopper, G = componentIron, I = ingotIron*

Output	Qty	Recipe
metal_decoration0/5	2	IGI PEP IGI *E = ingotElectrum, G = componentSteel, I = ingotSteel, P = piston*
metal_decoration0/6	2	III EDE III *D = metal_device1/2, E = ingotElectrum, I = ingotSteel*
metal_decoration0/7	2	ICI CBC ICI *B = ?, C = ingotCopper, I = ingotSteel*
metal_decoration1/0	3	IRI IRI *I = ingotSteel, R = stickSteel*
metal_decoration1/1	1	S S *S = metal_decoration1_slab/1*
metal_decoration1/1	1	metal_decoration1/3
metal_decoration1/1	6	III R R R *I = ingotSteel, R = stickSteel*
metal_decoration1/2	1	S S *S = metal_decoration1_slab/2*
metal_decoration1/2	1	metal_decoration1/1
metal_decoration1/3	1	S S *S = metal_decoration1_slab/3*
metal_decoration1/3	1	metal_decoration1/2
metal_decoration1/4	3	IRI IRI *I = ingotAluminum, R = stickAluminum*
metal_decoration1/5	1	S S *S = metal_decoration1_slab/5*
metal_decoration1/5	1	metal_decoration1/7
metal_decoration1/5	6	III R R R *I = ingotAluminum, R = stickAluminum*
metal_decoration1/6	1	S S *S = metal_decoration1_slab/6*
metal_decoration1/6	1	metal_decoration1/5
metal_decoration1/7	1	S S *S = metal_decoration1_slab/7*
metal_decoration1/7	1	metal_decoration1/6
metal_decoration1_slab/1	1	metal_decoration1_slab/3
metal_decoration1_slab/1	6	SSS *S = metal_decoration1/1*
metal_decoration1_slab/2	1	metal_decoration1_slab/1
metal_decoration1_slab/2	6	SSS *S = metal_decoration1/2*

Output	Qty	Recipe
`metal_decoration1_slab/3`	1	metal_decoration1_slab/2
`metal_decoration1_slab/3`	6	SSS *S = metal_decoration1/3*
`metal_decoration1_slab/5`	1	metal_decoration1_slab/7
`metal_decoration1_slab/5`	6	SSS *S = metal_decoration1/5*
`metal_decoration1_slab/6`	1	metal_decoration1_slab/5
`metal_decoration1_slab/6`	6	SSS *S = metal_decoration1/6*
`metal_decoration1_slab/7`	1	metal_decoration1_slab/6
`metal_decoration1_slab/7`	6	SSS *S = metal_decoration1/7*
`metal_decoration2/0`	1	F F S *F = fenceSteel, S = bricksStone*
`metal_decoration2/1`	4	II IS *I = ingotSteel, S = stickSteel*
`metal_decoration2/2`	1	F F S *F = fenceAluminum, S = bricksStone*
`metal_decoration2/3`	4	II IS *I = ingotAluminum, S = stickAluminum*
`metal_decoration2/4`	3	I PGP I *G = glowstone, I = plateIron, P = paneGlass*
`metal_decoration2/5`	3	PWP WWW PWP *P = plateSteel, W = wireSteel*
`metal_decoration2/7`	4	SSS SS S *S = scaffoldingSteel*
`metal_decoration2/8`	4	SSS SS S *S = scaffoldingAluminum*
`metal_ladder/0`	3	S S SSS S S *S = listMetalSticks*
`metal_ladder/1`	1	S L *L = metal_ladder/0, S = scaffoldingSteel*
`metal_ladder/2`	1	S L *L = metal_ladder/0, S = scaffoldingAluminum*
`steel_scaffolding_stairs0`	1	steel_scaffolding_stairs2
`steel_scaffolding_stairs0`	4	S SS SSS *S = metal_decoration1/1*

Output	Qty	Recipe
steel_scaffolding_stairs1	1	steel_scaffolding_stairs0
steel_scaffolding_stairs1	4	S SS SSS *S = metal_decoration1/2*
steel_scaffolding_stairs2	1	steel_scaffolding_stairs1
steel_scaffolding_stairs2	4	S SS SSS *S = metal_decoration1/3*

18.0.12 metal_devices

Output	Qty	Recipe
metal_device0/0	1	III CLC WRW *C = ingotCopper, I = ingotIron, L = ingotLead, R = dustRedstone, W = plankTreatedWood*
metal_device0/1	1	III ELE WRW *E = ingotElectrum, I = ingotIron, L = ingotLead, R = blockRedstone, W = plankTreatedWood*
metal_device0/2	1	III ALA WRW *A = ingotAluminum, I = ingotSteel, L = blockLead, R = blockRedstone, W = plankTreatedWood*
metal_device0/4	1	SSS B B BBB *B = blockSheetmetalIron, S = slabSheetmetalIron*
metal_device0/5	1	I ICI PPP *C = componentIron, I = plateIron, P = metal_device1/6*
metal_device0/6	1	IBI B B IBI *B = iron_bars, I = plateIron*
metal_device1/0	1	SSS S S SHS *H = metal_device1/1, S = blockSheetmetalIron*
metal_device1/1	1	ICI CBC IRI *B = copperCoil, C = ingotCopper, I = ingotIron, R = dustRedstone*

Output	Qty	Recipe
`metal_device1/10`	1	S GC XBR *B = wooden_device0/6, C = material/27, G = chemthrower, R = metal_decoration0/3, S = toolupgrade/8, X = metal_device0/4*
`metal_device1/11`	1	S GC XBR *B = wooden_device0/6, C = material/27, G = revolver/0, R = metal_decoration0/3, S = toolupgrade/8, X = toolupgrade/5*
`metal_device1/13`	1	GTG G G WCW *C = componentIron, G = blockGlass, T = elecTube, W = plankTreatedWood*
`metal_device1/2`	1	RCR III *C = copperCoil, I = ingotIron, R = dustRedstone*
`metal_device1/3`	1	III CBC CCC *B = copperCoil, C = plateConstantan, I = ingotSteel*
`metal_device1/4`	3	I PTP IRI *I = plateIron, P = paneGlass, R = dustRedstone, T = elecTube*
`metal_device1/5`	1	IMI GGG WCW *C = copperCoil, G = blockGlass, I = ingotIron, M = connector/2, W = plankTreatedWood*
`metal_device1/6`	8	PPP PPP *P = plateIron*
`metal_device1/7`	1	SFS SFS BFB *B = metal_decoration0/4, F = fenceSteel, S = scaffoldingSteel*
`metal_device1/8`	1	III C MCM *C = copperCoil, I = ingotAluminum, M = metal_device0/2*
`metal_device1/9`	1	III PTC ILI *C = copperCoil, I = ingotIron, L = componentIron, P = paneGlass, T = elecTube*

18.0.13 metal_storage

Output	Qty	Recipe
`iron_ingot`	1	NNN NNN NNN *N = metal/29*
`metal/0`	1	NNN NNN NNN *N = metal/20*
`metal/0`	9	storage/0
`metal/1`	1	NNN NNN NNN *N = metal/21*
`metal/1`	9	storage/1
`metal/2`	1	NNN NNN NNN *N = metal/22*
`metal/2`	9	storage/2
`metal/20`	9	metal/0
`metal/21`	9	metal/1
`metal/22`	9	metal/2
`metal/23`	9	metal/3
`metal/24`	9	metal/4
`metal/25`	9	metal/5
`metal/26`	9	metal/6
`metal/27`	9	metal/7
`metal/28`	9	metal/8
`metal/29`	9	iron_ingot
`metal/3`	1	NNN NNN NNN *N = metal/23*
`metal/3`	9	storage/3
`metal/4`	1	NNN NNN NNN *N = metal/24*
`metal/4`	9	storage/4
`metal/5`	1	NNN NNN NNN *N = metal/25*
`metal/5`	9	storage/5
`metal/6`	1	NNN NNN NNN *N = metal/26*
`metal/6`	9	storage/6
`metal/7`	1	NNN NNN NNN *N = metal/27*
`metal/7`	9	storage/7

Output	Qty	Recipe
`metal/8`	1	NNN NNN NNN *N = metal/28*
`metal/8`	9	storage/8
`storage/0`	1	III III III *I = metal/0*
`storage/0`	1	S S *S = storage_slab/0*
`storage/1`	1	III III III *I = metal/1*
`storage/1`	1	S S *S = storage_slab/1*
`storage/2`	1	III III III *I = metal/2*
`storage/2`	1	S S *S = storage_slab/2*
`storage/3`	1	III III III *I = metal/3*
`storage/3`	1	S S *S = storage_slab/3*
`storage/4`	1	III III III *I = metal/4*
`storage/4`	1	S S *S = storage_slab/4*
`storage/5`	1	III III III *I = metal/5*
`storage/5`	1	S S *S = storage_slab/5*
`storage/6`	1	III III III *I = metal/6*
`storage/6`	1	S S *S = storage_slab/6*

Output	Qty	Recipe
`storage/7`	1	III III III *I = metal/7*
`storage/7`	1	S S *S = storage_slab/7*
`storage/8`	1	III III III *I = metal/8*
`storage/8`	1	S S *S = storage_slab/8*
`storage_slab/0`	6	BBB *B = storage/0*
`storage_slab/1`	6	BBB *B = storage/1*
`storage_slab/2`	6	BBB *B = storage/2*
`storage_slab/3`	6	BBB *B = storage/3*
`storage_slab/4`	6	BBB *B = storage/4*
`storage_slab/5`	6	BBB *B = storage/5*
`storage_slab/6`	6	BBB *B = storage/6*
`storage_slab/7`	6	BBB *B = storage/7*
`storage_slab/8`	6	BBB *B = storage/8*

18.0.14 petroleum

Output	Qty	Recipe
`petroleum/0`	1	P C S *C = componentIron, P = metal_device1/6, S = plateSteel*
`petroleum/1`	4	S S S *S = plateSteel*
`petroleum/2`	1	SCS SPS SCS *C = componentSteel, P = metal_device1/6, S = plateSteel*
`petroleum/3`	8	WW WW *W = fiberHemp*

Output	Qty	Recipe
`petroleum/5`	1	SGS CRC SSS *C = componentIron, G = paneGlass, R = dustRedstone, S = plateIron*
`petroleum_decoration/1`	4	AA AA *A = petroleum_decoration/0*
`petroleum_decoration/2`	1	petroleum_decoration/0, dyeYellow
`petroleum_decoration/3`	4	AAA WWW *A = blockSheetmetalAluminum, W = plankTreatedWood*
`petroleum_device/1`	4	S S R S S *R = plateSteel, S = stickSteel*
`petroleum_device/10`	8	SPS P P SPS *P = plateIron, S = plateSteel*
`petroleum_device/11`	1	C SSS SSS *C = componentSteel, S = plateSteel*
`petroleum_device/12`	1	SCS SSS WSW *C = componentSteel, S = plateSteel, W = stickTreatedWood*
`petroleum_device/2`	1	T SCS SSS *C = componentSteel, S = plateSteel, T = torch*
`petroleum_device/3`	1	P PCP P *C = componentSteel, P = plateSteel*
`petroleum_device/4`	1	C S S SSS *C = componentSteel, S = plateSteel*
`petroleum_device/5`	2	G PCP P *C = componentIron, G = paneGlass, P = plateIron*
`petroleum_device/6`	1	SGS PCP SSS *C = componentSteel, G = paneGlass, P = metal_device1/6, S = plateSteel*
`petroleum_device/7`	1	SGS SGS R *G = paneGlass, R = stickTreatedWood, S = plateIron*

Output	Qty	Recipe
`petroleum_device/8`	1	SPS CEC SSS *C = copperCoil, E = componentSteel, P = metal_device1/6, S = plateSteel*
`petroleum_device/9`	1	SPS PCP SPS *C = componentSteel, P = metal_device1/6, S = plateSteel*

18.0.15 sheetmetal

Output	Qty	Recipe
`metal/30`	1	sheetmetal/0
`metal/31`	1	sheetmetal/1
`metal/32`	1	sheetmetal/2
`metal/33`	1	sheetmetal/3
`metal/34`	1	sheetmetal/4
`metal/35`	1	sheetmetal/5
`metal/36`	1	sheetmetal/6
`metal/37`	1	sheetmetal/7
`metal/38`	1	sheetmetal/8
`metal/39`	1	sheetmetal/9
`metal/40`	1	sheetmetal/10
`sheetmetal/0`	1	S S *S = sheetmetal_slab/0*
`sheetmetal/0`	4	P P P P *P = plateCopper*
`sheetmetal/1`	1	S S *S = sheetmetal_slab/1*
`sheetmetal/1`	4	P P P P *P = plateAluminum*
`sheetmetal/10`	1	S S *S = sheetmetal_slab/10*
`sheetmetal/10`	4	P P P P *P = plateGold*
`sheetmetal/2`	1	S S *S = sheetmetal_slab/2*
`sheetmetal/2`	4	P P P P *P = plateLead*

Output	Qty	Recipe
`sheetmetal/3`	1	S S *S = sheetmetal_slab/3*
`sheetmetal/3`	4	P P P P *P = plateSilver*
`sheetmetal/4`	1	S S *S = sheetmetal_slab/4*
`sheetmetal/4`	4	P P P P *P = plateNickel*
`sheetmetal/5`	1	S S *S = sheetmetal_slab/5*
`sheetmetal/5`	4	P P P P *P = plateUranium*
`sheetmetal/6`	1	S S *S = sheetmetal_slab/6*
`sheetmetal/6`	4	P P P P *P = plateConstantan*
`sheetmetal/7`	1	S S *S = sheetmetal_slab/7*
`sheetmetal/7`	4	P P P P *P = plateElectrum*
`sheetmetal/8`	1	S S *S = sheetmetal_slab/8*
`sheetmetal/8`	4	P P P P *P = plateSteel*
`sheetmetal/9`	1	S S *S = sheetmetal_slab/9*
`sheetmetal/9`	4	P P P P *P = plateIron*
`sheetmetal_slab/0`	6	BBB *B = sheetmetal/0*
`sheetmetal_slab/1`	6	BBB *B = sheetmetal/1*
`sheetmetal_slab/10`	6	BBB *B = sheetmetal/10*
`sheetmetal_slab/2`	6	BBB *B = sheetmetal/2*

Output	Qty	Recipe
sheetmetal_slab/3	6	BBB *B = sheetmetal/3*
sheetmetal_slab/4	6	BBB *B = sheetmetal/4*
sheetmetal_slab/5	6	BBB *B = sheetmetal/5*
sheetmetal_slab/6	6	BBB *B = sheetmetal/6*
sheetmetal_slab/7	6	BBB *B = sheetmetal/7*
sheetmetal_slab/8	6	BBB *B = sheetmetal/8*
sheetmetal_slab/9	6	BBB *B = sheetmetal/9*

18.0.16 signage

Output	Qty	Recipe
signage/0	4	PP *P = plateAluminum*

18.0.17 stone_decoration

Output	Qty	Recipe
material/6	9	stone_decoration/3
stone_decoration/0	1	S S *S = stone_decoration_slab/0*
stone_decoration/0	3	CBC BSB CBC *B = ingotBrick, C = clay_ball, S = sandstone*
stone_decoration/1	1	S S *S = stone_decoration_slab/1*
stone_decoration/1	3	NBN BDB NBN *B = ingotBrick, D = blaze_powder, N = ingotBrickNether*
stone_decoration/10	1	S S *S = stone_decoration_slab/10*
stone_decoration/10	2	SB BS *B = ingotBrick, S = sandstone*
stone_decoration/2	1	P B *B = stone_decoration/1, P = plateSteel*
stone_decoration/2	1	S S *S = stone_decoration_slab/2*

Output	Qty	Recipe
stone_decoration/3	1	CCC CCC CCC *C = material/6*
stone_decoration/4	1	S S *S = stone_decoration_slab/4*
stone_decoration/4	6	CCC HHH CCC *C = clay_ball, H = fiberHemp*
stone_decoration/5	1	S S *S = stone_decoration_slab/5*
stone_decoration/5	1	stone_decoration/6
stone_decoration/5	8	SCS GBG SCS *B = ?, C = clay_ball, G = gravel, S = sand*
stone_decoration/5	12	SCS GBG SCS *B = ?, C = clay_ball, G = gravel, S = itemSlag*
stone_decoration/6	1	S S *S = stone_decoration_slab/6*
stone_decoration/6	4	CC CC *C = stone_decoration/5*
stone_decoration/7	1	P B *B = stone_decoration/5, P = plateLead*
stone_decoration/7	1	S S *S = stone_decoration_slab/7*
stone_decoration/8	2	G IDI G *D = dyeGreen, G = blockGlass, I = dustIron*
stone_decoration_slab/0	6	BBB *B = stone_decoration/0*
stone_decoration_slab/1	6	BBB *B = stone_decoration/1*
stone_decoration_slab/10	6	BBB *B = stone_decoration/10*
stone_decoration_slab/2	6	BBB *B = stone_decoration/2*
stone_decoration_slab/4	6	BBB *B = stone_decoration/4*
stone_decoration_slab/5	6	BBB *B = stone_decoration/5*
stone_decoration_slab/6	6	BBB *B = stone_decoration/6*
stone_decoration_slab/7	6	BBB *B = stone_decoration/7*
stone_decoration_stairs_concrete	1	stone_decoration_stairs_concrete_tile

Output	Qty	Recipe
stone_decoration_stairs_concrete	4	C CC CCC *C = stone_decoration/5*
stone_decoration_stairs_concrete_leaded	4	C CC CCC *C = stone_decoration/7*
stone_decoration_stairs_concrete_tile	1	stone_decoration_stairs_concrete
stone_decoration_stairs_concrete_tile	4	C CC CCC *C = stone_decoration/6*
stone_decoration_stairs_hempcrete	4	C CC CCC *C = stone_decoration/4*

18.0.18 tool

Output	Qty	Recipe
axe_steel	1	SS ST T *S = ingotSteel, T = stickTreatedWood*
chemthrower	1	OG EG PB *B = bucket, E = metal_decoration0/5, G = woodenGrip, O = toolupgrade/0, P = metal_device1/6*
drill/0	1	G EG C *C = componentSteel, E = metal_decoration0/5, G = woodenGrip*
drillhead/0	1	SS BBS SS *B = blockSteel, S = ingotSteel*
drillhead/1	1	SS BBS SS *B = blockIron, S = ingotIron*
fluorescent_tube	1	GEG GgG GgG *E = graphite_electrode, G = blockGlass, g = dustGlowstone*
hoe_steel	1	SS T T *S = ingotSteel, T = stickTreatedWood*

Output	Qty	Recipe
`jerrycan`	1	II IBB IBB *B = bucket, I = plateIron*
`maintenance_kit`	1	RW HHH *H = fabricHemp, R = stickIron, W = tool/1*
`pickaxe_steel`	1	SSS T T *S = ingotSteel, T = stickTreatedWood*
`railgun`	1	HG CBH BC *B = material/14, C = metal_decoration0/1, G = woodenGrip, H = metal_device0/2*
`revolver/0`	1	I BDH GIG *B = material/14, D = material/15, G = woodenGrip, H = material/16, I = ingotIron*
`shield`	1	PTP PSP PTP *P = plateSteel, S = shield, T = plankTreatedWood*
`shovel_steel`	1	S T T *S = ingotSteel, T = stickTreatedWood*
`skyhook`	1	II IC GG *C = componentIron, G = woodenGrip, I = ingotSteel*
`speedloader`	1	I IIS I *I = ingotIron, S = ingotSteel*
`sword_steel`	1	S S T *S = ingotSteel, T = stickTreatedWood*
`tool/0`	1	IF SI S *F = string, I = ingotIron, S = listWoodenSticks*
`tool/1`	1	SI S *I = ingotIron, S = listWoodenSticks*
`tool/2`	1	P SCS *C = ingotCopper, P = compass, S = listWoodenSticks*
`tool/3`	1	book, lever
`toolbox`	1	PPP RCR *C = wooden_device0/0, P = plateAluminum, R = dyeRed*

18.0.19 toolupgrades

Output	Qty	Recipe
`toolupgrade/0`	1	BI IBI IC *B = bucket, C = componentIron, I = dyeBlue*
`toolupgrade/1`	1	BI IBI IC *B = ?, C = componentIron, I = ingotIron*
`toolupgrade/10`	1	AGA GTG *A = plateAluminum, G = paneGlass, T = elecTube*
`toolupgrade/11`	1	CRC CRC CRC *C = connector/0, R = stickIron*
`toolupgrade/12`	1	L LC LIL *C = metal_decoration0/0, I = ingotIron, L = leather*
`toolupgrade/13`	1	P TCT *C = componentSteel, P = metal_device1/6, T = toolupgrade/3*
`toolupgrade/2`	1	SSS C *C = componentSteel, S = ingotSteel*
`toolupgrade/3`	1	CS SBO OB *B = bucket, C = componentIron, O = dyeRed, S = ingotSteel*
`toolupgrade/4`	1	SI IW *I = ingotSteel, S = iron_sword, W = plankTreatedWood*
`toolupgrade/5`	1	CS C C IC *C = ingotCopper, I = componentIron, S = ingotSteel*
`toolupgrade/6`	1	TTT RWR *R = stickSteel, T = elecTube, W = wireCopper*
`toolupgrade/7`	1	SS PPH SS *H = hopper, P = metal_device1/6, S = ingotSteel*
`toolupgrade/8`	1	GC C C CG *C = ingotCopper, G = paneGlassColorless*
`toolupgrade/9`	1	WWW HHH *H = metal_device0/2, W = wirecoil/2*

18.0.20 treated_wood

Output	Qty	Recipe
treated_wood/0	1	W W *W = treated_wood_slab/0*
treated_wood/0	1	treated_wood/2
treated_wood/0	8	WWW WCW WWW *C = ?, W = plankWood*
treated_wood/1	1	W W *W = treated_wood_slab/1*
treated_wood/1	1	treated_wood/0
treated_wood/2	1	W W *W = treated_wood_slab/2*
treated_wood/2	1	treated_wood/1
treated_wood_slab/0	1	treated_wood_slab/2
treated_wood_slab/0	6	WWW *W = treated_wood/0*
treated_wood_slab/1	1	treated_wood_slab/0
treated_wood_slab/1	6	WWW *W = treated_wood/1*
treated_wood_slab/2	1	treated_wood_slab/1
treated_wood_slab/2	6	WWW *W = treated_wood/2*
treated_wood_stairs0	1	treated_wood_stairs2
treated_wood_stairs0	4	W WW WWW *W = treated_wood/0*
treated_wood_stairs1	1	treated_wood_stairs0
treated_wood_stairs1	4	W WW WWW *W = treated_wood/1*
treated_wood_stairs2	1	treated_wood_stairs1
treated_wood_stairs2	4	W WW WWW *W = treated_wood/2*
wooden_decoration/0	3	WSW WSW *S = stickTreatedWood, W = plankTreatedWood*
wooden_decoration/1	6	WWW S S S *S = stickTreatedWood, W = plankTreatedWood*

18.0.21 vehicles

Output	Qty	Recipe
`hydraulic_crawler`	1	SCS EHE BBB *B = blockSheetmetalSteel, C = componentSteel, E = metal_device1/6, H = blockSteel, S = plateSteel*

18.0.22 wirecoils

Output	Qty	Recipe
`wirecoil/0`	4	I ISI I *I = wireCopper, S = listWoodenSticks*
`wirecoil/1`	4	I ISI I *I = wireElectrum, S = listWoodenSticks*
`wirecoil/2`	4	A ISI A *A = wireAluminum, I = wireSteel, S = listWoodenSticks*
`wirecoil/2`	4	I ASA I *A = wireAluminum, I = wireSteel, S = listWoodenSticks*
`wirecoil/3`	4	I ISI I *I = fiberHemp, S = listWoodenSticks*
`wirecoil/4`	4	I ISI I *I = wireSteel, S = listWoodenSticks*
`wirecoil/5`	4	A ISI A *A = wireAluminum, I = dustRedstone, S = listWoodenSticks*
`wirecoil/5`	4	I ASA I *A = wireAluminum, I = dustRedstone, S = listWoodenSticks*
`wirecoil/6`	4	FCF CFC FCF *C = copperCoilItem, F = fabricHemp*
`wirecoil/7`	4	FCF COC FCF *C = electrumCoilItem, F = fabricHemp, O = ?*

18.0.23 wooden_devices

Output	Qty	Recipe
`wooden_device0/0`	1	WWW W W WWW *W = plankTreatedWood*
`wooden_device0/1`	1	SSS W W WWW *S = slabTreatedWood, W = plankTreatedWood*
`wooden_device0/2`	1	WWW B F *B = craftingTableWood, F = fenceTreatedWood, W = plankTreatedWood*
`wooden_device0/3`	1	WRW IBI WCW *B = baseConveyor, C = componentIron, I = ingotIron, R = dustRedstone, W = plankTreatedWood*
`wooden_device0/4`	1	F GBG GGG *B = wooden_device0/1, F = fiberHemp, G = gunpowder*
`wooden_device0/5`	1	WPW RCR WPW *C = wooden_device0/0, P = plateIron, R = stickIron, W = plankTreatedWood*
`wooden_device0/6`	1	WIW RCR *C = copperCoil, I = ingotIron, R = dustRedstone, W = plankTreatedWood*
`wooden_device0/7`	1	WRW IBI WCW *B = metal_device1/6, C = componentIron, I = ingotIron, R = dustRedstone, W = plankTreatedWood*
`wooden_device1/0`	1	P PIP P *I = ingotSteel, P = material/10*
`wooden_device1/1`	1	PPP PIP PPP *I = ingotIron, P = material/11*
`wooden_device1/3`	1	F F S *F = fenceTreatedWood, S = bricksStone*
`wooden_device1/4`	4	WW WS *S = stickTreatedWood, W = plankTreatedWood*

CHAPTER 19

Every configuration option

Generated from `Config.java`, including the comment each option ships with.
190 documented options.

`validateConnections` *boolean, default* `false`
Drop connections with non-existing endpoints when loading the world. Use with care and backups and only when suspecting corrupted data. This option will check and load all connection endpoints and may slow down the world loading process.

`wireTransferRate` *int[], default* `new int[]{2048, 8192, 32768, 0, 0, 0}`
The transfer rates in Flux/t for the wire tiers (copper, electrum, HV, Structural Rope, Cable & Redstone(no transfer))

`wireLossRatio` *double[], default* `new double[]{.05, .025, .025, 1, 1, 1}`
The percentage of power lost every 16 blocks of distance for the wire tiers (copper, electrum, HV, Structural Rope, Cable & Redstone(no transfer))

`wireColouration` *int[], default* `wireColourationDefault`
The RGB colourate of the wires.

`wireLength` *int[], default* `new int[]{16, 16, 32, 32, 32, 32}`
The maximum length wire can have. Copper and Electrum should be similar, Steel is meant for long range transport, Structural Rope & Cables are purely decorational

`enableWireDamage` *boolean, default* `true`
If this is enabled, wires connected to power sources will cause damage to entities touching them This shouldn't cause significant lag but possibly will. If it does, please report it at https://github.com/BluSunrize/ImmersiveEngin unless there is a report of it already.

`blocksBreakWires` *boolean, default* `true`
If this is enabled, placing a block in a wire will break it (drop the wire coil)

`cityMode` *boolean, default* `true`
"City mode": trades Immersive Engineering's simulation detail for server tick time. Intended for decorative / city packs that want the look of the mod without paying for its physics. This is the master switch. When it is off, nothing below applies and behaviour is identical to stock. When it is on, each subsystem listed below is simplified unless you turn that subsystem back off individually. DEFAULT: ON. This build exists for a city pack, so it ships with city mode enabled; set this to false for stock Immersive Engineering behaviour. A server pushes this setting and all the cityMode* flags below to every client on login, so the dedicated server, its clients and single-player worlds all agree without anyone editing a config. Editing a client's own copy therefore only affects the single-player worlds that client opens. Existing worlds are unaffected either way – city mode adds no saved data and changes no persisted state.

`cityModeWires` *boolean, default* `true`
City mode: wires. A powered connector pushes energy straight to the devices on its network with no per-wire loss and no distance/path weighting – the realistic-grid simulation (loss, proportional distribution, the double simulate/transfer pass, the network-wide energy broadcast) is skipped. Side effects worth knowing: wires can no longer burn out from overload, voltage tiers stop limiting throughput, and wire shock damage is sourced from the local connector rather than the whole network. Only applies when cityMode is enabled.

cityModePipes *boolean, default* ***true***
City mode: fluid pipes. A pipe hands its fluid to the endpoints on its network in order until the fluid runs out, instead of simulating a fill against every one of them and then splitting the resource between them in proportion to what each asked for. That halves the capability calls per fill and drops the per-fill list and map allocation. Transfer limits still apply and nothing is created or destroyed – the only difference a player could observe is which of several tanks fills first. Only applies when cityMode is enabled.

cityModeConduits *boolean, default* ***true***
City mode: conduits. A bundle stops accounting for individual units of flux and switches to presence: a conductor is either energised or it is not, an energised breakout delivers at full rate, and line loss is not charged. That replaces the per-channel gradient arithmetic on every junction box with a flag propagated along the run, and removes the buffering that makes a long run ramp up. This is the conduit's counterpart to the virtual grid's city mode – same trade, same reasoning. A conductor goes dark about a second after whatever was feeding it stops, so a switched circuit still visibly switches. Only applies when cityMode is enabled.

cityModeTanks *boolean, default* ***true***
City mode: tanks. A tank holding any fluid at all becomes an unlimited source of it – draw as much as you like and the level does not move. An empty tank still gives nothing, so filling one still matters and a dry tank still reads as dry. Filling is untouched: a tank still fills at its ordinary rate from whatever is feeding it. Only the draw side is free. Applies to the buried Domestic, Commercial and Bulk tanks and to the Sheetmetal Tank. Only applies when cityMode is enabled.

cityModeFloodlights *boolean, default* ***true***
City mode: floodlights. Skips the periodic re-scan of the light beams, recomputing only when the light actually switches or a neighbouring block changes, and caps how many light blocks a single floodlight may place. Floodlights are usually the most expensive block in a city build because each one places a number of individually ticking light blocks and periodically re-traces 13 beams and recalculates lighting. Only applies when cityMode is enabled.

cityModeGenerators *boolean, default* ***true***
City mode: generators. Fuel becomes cosmetic – a diesel generator checks that fuel is present and sips a token amount rather than computing and draining a per-tick burn rate, and produces a flat output. Generators still only run when something actually wants power, so an idle generator still costs nothing and still burns nothing. Only applies when cityMode is enabled.

cityModeMachines *boolean, default* ***true***
City mode: machines. Multiblocks scan for a new recipe far less often when they are already busy, and the stone furnaces stop broadcasting a block update to all 26 neighbours on every state change. A multiblock that has not managed to start anything for ten seconds is throttled too, so a decorative machine holding input it cannot use stops re-scanning the recipe list forever; a machine that is working is not throttled and does not stutter between batches. A machine that is switched on and holds power also animates and plays its looping sound steadily rather than only while mid-process, and its energy buffer follows its redstone switch: enabling the machine fills the buffer, disabling it empties it. That is the one place city mode creates energy. Machines still craft the same recipes at the same speed; an idle one may take slightly longer to notice newly inserted items. Only applies when cityMode is enabled.

cityModeVirtualGrid *boolean, default* ***true***
City mode: the virtual power grid. Grid segments stop accounting for flux and switch to presence semantics – a segment is either energized or it is not, and its Service Units then deliver freely with no pool, no buffer and no loss. Feed Units consume only a token 'sip' (see gridSipAmount / gridSipIntervalTicks) to prove their source is still live, so a city-scale grid costs almost nothing per tick. Segment on/off switches, failover links, the console GUI and chunk loading all behave identically in both modes. Only applies when cityMode is enabled.

cityModePetroleum *boolean, default* ***true***
City mode: petroleum. Reservoirs still exist, are still prospected and still drilled – only the bookkeeping is dropped. A reservoir never depletes and a well holds constant pressure, so extraction rate never decays and free flow never lapses. This removes the only per-tick cost the reservoir model has (tracking remaining capacity), which is cheap already, but the flag exists so a city build can have petroleum feel bottomless like everything else in city mode does. Only applies when cityMode is enabled.

disableFancyTESR *boolean, default* ***false***

By default all devices that accept cables have increased renderbounds to show cables even if the block itself is not in view. Disabling this reduces them to their minimum sizes, which might improve FPS on low-power PCs Disables most lighting code for certain models that are rendered dynamically (TESR). May improve FPS. Affects turrets and garden cloches

disableFancyBlueprints *boolean, default* ***true***
Disables the fancy rendering of blueprints on the Workbench and Autoworkbench. Set this to true if your game keeps freezing or crashing when looking at such a block.

colourblindSupport *boolean, default* ***false***
Support for colourblind people, gives a text-based output on capacitor sides

nixietubeFont *boolean, default* ***true***
Set this to false to disable the super awesome looking nixie tube front for the voltmeter and other things

adjustManualScale *boolean, default* ***false***
Set this to false to disable the manual's forced change of GUI scale

badEyesight *boolean, default* ***false***
Set this to true if you suffer from bad eyesight. The Engineer's manual will be switched to a bold and darker text to improve readability. Note that this may lead to a break of formatting and have text go off the page in some instances. This is unavoidable.

oreTooltips *boolean, default* ***true***
Controls if item tooltips should contain the OreDictionary names of items. These tooltips are only visible in advanced tooltip mod (F3+H)

increasedTileRenderdistance *double, default* ***1.5***
Increase the distance at which certain TileEntities (specifically windmills) are still visible. This is a modifier, so set it to 1 for default render distance, to 2 for doubled distance and so on.

preferredOres *String[], default* ***new String[]{ImmersiveEngineering.MODID}***
A list of preferred Mod IDs that results of IE processes should stem from, aka which mod you want the copper to come from. This affects the ores dug by the excavator, as well as those crushing recipes that don't have associated IE items. This list is in oreder of priority.

showUpdateNews *boolean, default* ***true***
Set this to false to hide the update news in the manual

villagerHouse *boolean, default* ***true***
Set this to false to stop the IE villager house from spawning

enableVillagers *boolean, default* ***true***
Set this to false to remove IE villagers from the game

hempSeedWeight *int, default* ***5***
The weight that hempseeds have when breaking tall grass. 5 by default, set to 0 to disable drops

fancyItemHolding *boolean, default* ***true***
Allows revolvers and other IE items to look properly held in 3rd person. This uses a coremod. Can be disabled in case of conflicts with other animation mods.

stencilBufferEnabled *boolean, default* ***true***
Set to false to disable the stencil buffer. This may be necessary on older GPUs.

coreSampleCoords *boolean, default* ***true***
Set to false to have Coresamples not show the coordinates of the chunk.

enableDebug *boolean, default* ***false***
A list of all mods that IE has integrated compatability for Setting any of these to false disables the respective compat A config setting to enable debug features. These features may vary between releases, may cause crashes, and are unsupported. Do not enable unless asked to by a developer of IE.

enableVirtualGrid *boolean, default* ***true***
Master switch for the virtual power grid. When false the tick engine does no work at all and Feed/Service Units stay inert (they are still placeable, so turning this off cannot destroy an existing build).

gridCrossDimension *boolean, default* ***true***
Whether a single grid segment may contain devices in more than one dimension. When false, a segment is pinned to the dimension of the first device assigned to it.

gridDefaultDeviceCap *int, default 4096*
Default throughput of a single Feed or Service Unit, in Flux/t. The default matches an HV connector.

gridMaxSegmentIO *int, default 131072*
The highest per-segment input/output rate that may be set from the Grid Management Console, in Flux/t. Also the ceiling for a single device's transfer cap. Times gridBufferTicks this lands exactly on gridBufferCapMax, so a new segment gets its full smoothing buffer. Raise both together or new segments quietly get less.

gridDefaultLossPct *double, default 0.0*
Fraction of energy lost in transmission through a new segment, 0.0 to 1.0. Ships at 0: by default the grid is a convenience feature, not a lossy one. Raise it if you want physical wire to keep an efficiency advantage.

gridFailoverTopUpDefault *boolean, default true*
Default state of a new segment's 'backups also cover shortfalls' toggle. When false, linked backup segments only step in if the segment is switched off or has tripped.

gridBufferTicks *int, default 2*
How many ticks of its own output rate a segment buffers by default. This buffer only exists so energy collected this tick can be delivered this tick; it is deliberately far too small to be used as storage.

gridBufferCapMax *int, default 262144*
Hard ceiling on a segment's buffer, in Flux. This is the anti-battery clamp – the grid is a conduit, not free storage.

gridConsoleRequiresPower *boolean, default true*
Whether the Grid Management Console needs standby power to light its screen. The GUI still opens without power, in read-only mode, so losing power can never lock you out of your own grid.

gridConsoleStandbyDraw *int, default 8*
Standby draw of a Grid Management Console, in Flux/t.

gridAllowChunkloading *boolean, default true*
Master switch for the per-device chunk loading toggle. When false, the toggle is ignored and no grid device keeps chunks loaded.

gridChunkloadBudget *int, default 25*
Server-wide maximum number of chunks that grid devices may hold loaded between them.

gridSipIntervalTicks *int, default 100*
City mode only: how often, in ticks, a Feed Unit checks that its power source is still live. 100 ticks is once every five seconds.

gridSipAmount *int, default 1*
City mode only: how much Flux a Feed Unit consumes per liveness check. This is the entire running cost of a city-mode grid.

gridMaxFailoverDepth *int, default 4*
How many failover links a shortfall may travel along before giving up. Cycles are always broken regardless of this value.

gridBreakersEnabled *boolean, default false*
Set to true to make segments trip like a real breaker: a segment held at its output ceiling for gridBreakerTripSeconds latches off and must be switched back on by hand. Off by default – with it off, demand above the ceiling is simply clamped.

gridBreakerTripSeconds *int, default 5*
Seconds of continuous output saturation before a segment's breaker trips. Only applies when gridBreakersEnabled is true.

enableFluidNetwork *boolean, default true*
Master switch for the virtual fluid network. When false the tick engine does no work at all and Inlets/Outlets stay inert (they are still placeable, so turning this off cannot destroy an existing build).

fluidNetCrossDimension *boolean, default true*
Whether a single main may contain fittings in more than one dimension. When false, a main is pinned to the dimension of the first fitting assigned to it.

fluidNetDefaultDeviceCap *int, default 1000*

Default throughput of a single Inlet or Outlet, in mB/t. The default is comfortably a pressurised Fluid Pipe's worth, and feeds three Steam Turbine Halls.

`fluidNetMaxMainIO` *int, default `32768`*
The highest per-main input/output rate that may be set from the Fluid Control Console, in mB/t. Also the ceiling for a single fitting's transfer cap. Times fluidNetPackTicks this lands exactly on fluidNetPackCapMax, so a new main gets its full smoothing pack. Raise both together or new mains quietly get less.

`fluidNetDefaultLeakPct` *double, default `0.0`*
Fraction of fluid lost in transit through a new main, 0.0 to 1.0. Ships at 0: by default the network is a convenience feature, not a lossy one. Raise it if you want physical pipe to keep an efficiency advantage.

`fluidNetFailoverTopUpDefault` *boolean, default `true`*
Default state of a new main's 'backups also cover shortfalls' toggle. When false, linked backup mains only step in if the main is closed or has tripped.

`fluidNetPackTicks` *int, default `2`*
How many ticks of its own output rate a main holds as line pack. This only exists so fluid collected this tick can be delivered this tick; it is deliberately far too small to be used as a tank.

`fluidNetPackCapMax` *int, default `65536`*
Hard ceiling on a main's line pack, in mB. This is the anti-tank clamp – a main is a conduit, and the buried tanks are what storage is for.

`fluidNetConsoleRequiresPower` *boolean, default `true`*
Whether the Fluid Control Console needs standby power to light its screen. The GUI still opens without power, in read-only mode, so losing power can never lock you out of your own network.

`fluidNetConsoleStandbyDraw` *int, default `8`*
Standby draw of a Fluid Control Console, in Flux/t.

`fluidNetAllowChunkloading` *boolean, default `true`*
Master switch for the per-fitting chunk loading toggle. When false, the toggle is ignored and no fluid network fitting keeps chunks loaded.

`fluidNetChunkloadBudget` *int, default `25`*
Server-wide maximum number of chunks that fluid network fittings may hold loaded between them.

`fluidNetSipIntervalTicks` *int, default `100`*
City mode only: how often, in ticks, an Inlet checks that its source is still live. 100 ticks is once every five seconds.

`fluidNetSipAmount` *int, default `1`*
City mode only: how much fluid an Inlet consumes per liveness check, in mB. This is the entire running cost of a city-mode network.

`fluidNetMaxFailoverDepth` *int, default `4`*
How many failover links a shortfall may travel along before giving up. Cycles are always broken regardless of this value.

`fluidNetTripsEnabled` *boolean, default `false`*
Set to true to make mains trip on overpressure: a main held at its output ceiling for fluidNetTripSeconds latches closed and must be opened by hand. Off by default – with it off, demand above the ceiling is simply clamped.

`fluidNetTripSeconds` *int, default `5`*
Seconds of continuous output saturation before a main's overpressure cut-out trips. Only applies when fluidNetTripsEnabled is true.

`enablePetroleum` *boolean, default `true`*
Master switch for petroleum reservoirs. When false no reservoir is ever generated and extraction blocks stay inert (they are still placeable, so turning this off cannot destroy an existing build).

`petroleumCellChunkSize` *int, default `8`*
Edge length, in chunks, of the square cell a single reservoir occupies. Deposits are rolled per cell rather than per chunk so that a field spans a believable area and neighbouring chunks agree about what is underneath them. Raising this makes fields rarer but larger; the per-cell chance below is unaffected.

petroleumCellChance *double, default 0.25*
Chance that any given cell contains a reservoir at all, 0.0 to 1.0.

petroleumMinCapacity *int, default 2000000*
Smallest and largest reservoir capacity in mB. Rolls are log-distributed between them, so modest fields are common and a giant is a genuine event.

petroleumFreeFlowThreshold *double, default 0.6*
Fraction of original capacity remaining above which a well flows without a pump, 0.0 to 1.0.

petroleumPeakFlowRate *int, default 30*
Peak extraction rate in mB/tick at full pressure, before any equipment multiplier.

petroleumResidualFlowRate *double, default 0.025*
Floor the flow rate decays to rather than reaching zero, in mB/tick. A depleted field becomes slow, never dead – a pumpjack must not strand a base that was built around it.

petroleumReinjectionCap *double, default 0.2*
Fraction of a deposit's original capacity that a Re-injection Well may ever put back, 0.0 to 1.0. A lifetime allowance per field, not a rate: enhanced recovery is meant to be the thinking player's alternative to flaring, not a way to farm one field forever. Set to 0 to switch enhanced recovery off entirely.

petroleumGushers *boolean, default true*
Whether drilling a still-pressurised field without a Blowout Preventer causes a gusher: a fountain of crude, a spill around the wellhead and a slice of the field's pressure gone. On by default. Only a field that still has its own pressure can blow out, which is exactly when the preventer is worth fitting – the mechanic teaches itself.

petroleumAssociatedGasRatio *double, default 0.25*
Millibuckets of associated (sour) gas a wellhead produces per millibucket of crude. Oil comes up with gas dissolved in it whether or not there is a use for it, which is what a flare stack is for. Set to 0 to switch the gas branch off entirely.

petroleumDimensionBlacklist *int[], default new int[]{1}*
Dimensions in which reservoirs are never generated.

wireConnectorInput *int[], default new int[]{256, 1024, 4096}*
In- and output rates of LV,MV and HV Wire Conenctors. This is independant of the transferrate of the wires.

capacitorLV_storage *int, default 100000*
The maximum amount of Flux that can be stored in a low-voltage capacitor

capacitorLV_input *int, default 256*
The maximum amount of Flux that can be input into a low-voltage capacitor (by IE net or other means)

capacitorLV_output *int, default 256*
The maximum amount of Flux that can be output from a low-voltage capacitor (by IE net or other means)

capacitorMV_storage *int, default 1000000*
The maximum amount of Flux that can be stored in a medium-voltage capacitor

capacitorMV_input *int, default 1024*
The maximum amount of Flux that can be input into a medium-voltage capacitor (by IE net or other means)

capacitorMV_output *int, default 1024*
The maximum amount of Flux that can be output from a medium-voltage capacitor (by IE net or other means)

capacitorHV_storage *int, default 4000000*
The maximum amount of Flux that can be stored in a high-voltage capacitor

capacitorHV_input *int, default 4096*
The maximum amount of Flux that can be input into a high-voltage capacitor (by IE net or other means)

capacitorHV_output *int, default 4096*
The maximum amount of Flux that can be output from a high-voltage capacitor (by IE net or other means)

dynamo_output *double, default 3d*
The base Flux that is output by the dynamo. This will be modified by the rotation modifier of the attached water- or windmill

thermoelectric_output *double, default 1d*
Output modifier for the energy created by the Thermoelectric Generator

lightning_output *int, default 4*4000000*
The Flux that will be output by the lightning rod when it is struck

dieselGen_output *int, default 4096*
The Flux per tick that the Diesel Generator will output. The burn time of the fuel determines the total output

heater_consumption *int, default 8*
The Flux per tick consumed to add one heat to a furnace. Creates up to 4 heat in the startup time and then 1 heat per tick to keep it running

heater_speedupConsumption *int, default 24*
The Flux per tick consumed to double the speed of the furnace. Only happens if furnace is at maximum heat.

preheater_consumption *int, default 32*
The Flux per tick the Blast Furnace Preheater will consume to speed up the Blast Furnace

coredrill_time *int, default 200*
The length in ticks it takes for the Core Sample Drill to figure out which mineral is found in a chunk

coredrill_consumption *int, default 40*
The Flux per tick consumed by the Core Sample Drill

pump_consumption *int, default 250*
The Flux the Fluid Pump will consume to pick up a fluid block in the world

pump_consumption_accelerate *int, default 5*
The Flux the Fluid Pump will consume pressurize+accelerate fluids, increasing the transferrate

pump_infiniteWater *boolean, default true*
Set this to false to disable the fluid pump being able to draw infinite water from sources

pump_placeCobble *boolean, default true*
If this is set to true (default) the pump will replace fluids it picks up with cobblestone in order to reduce lag caused by flowing fluids.

pipe_transferrate *int, default 50*
The basic transferrate of a fluid pipe, in mB/t

pipe_transferrate_pressurized *int, default 1000*
The transferrate of a fluid pipe when accelerated by a pump, in mB/t

charger_consumption *int, default 256*
The Flux per tick the Charging Station can insert into an item

teslacoil_consumption *int, default 256*
The Flux per tick the Tesla Coil will consume, simply by being active

teslacoil_consumption_active *int, default 512*
The amount of Flux the Tesla Coil will consume when shocking an entity

teslacoil_damage *float, default 6*
The amount of damage the Tesla Coil will do when shocking an entity

turret_consumption *int, default 64*
The Flux per tick any turret consumes to monitor the area

turret_chem_consumption *int, default 32*
The Flux per tick the chemthrower turret consumes to shoot

turret_gun_consumption *int, default 32*
The Flux per tick the gun turret consumes to shoot

belljar_consumption *int, default 8*

The Flux per tick the belljar consumes to grow plants

belljar_fertilizer *int, default 6000*
The amount of ticks one dose of fertilizer lasts in the belljar

belljar_fluid *int, default 250*
The amount of fluid the belljar uses per dose of fertilizer

belljar_growth_mod *float, default 1*
A modifier to apply to the belljars total growing speed

belljar_solid_fertilizer_mod *float, default 1f*
A base-modifier for all solid fertilizers in the belljar

belljar_fluid_fertilizer_mod *float, default 1f*
A base-modifier for all fluid fertilizers in the belljar

lantern_spawnPrevent *boolean, default true*
Set this to false to disable the mob-spawn prevention of the Powered Lantern

lantern_energyDraw *int, default 1*
How much Flux the powered lantern draws per tick

lantern_maximumStorage *int, default 10*
How much Flux the powered lantern can hold (should be greater than the power draw)

floodlight_spawnPrevent *boolean, default true*
Set this to false to disable the mob-spawn prevention of the Floodlight

floodlight_energyDraw *int, default 5*
How much Flux the floodlight draws per tick

floodlight_maximumStorage *int, default 80*
How much Flux the floodlight can hold (must be at least 10x the power draw)

metalPress_energyModifier *float, default 1*
A modifier to apply to the energy costs of every MetalPress recipe

metalPress_timeModifier *float, default 1*
A modifier to apply to the time of every MetalPress recipe

crusher_energyModifier *float, default 1*
A modifier to apply to the energy costs of every Crusher recipe

crusher_timeModifier *float, default 1*
A modifier to apply to the time of every Crusher recipe

squeezer_energyModifier *float, default 1*
A modifier to apply to the energy costs of every Squeezer recipe

squeezer_timeModifier *float, default 1*
A modifier to apply to the time of every Squeezer recipe

fermenter_energyModifier *float, default 1*
A modifier to apply to the energy costs of every Fermenter recipe

fermenter_timeModifier *float, default 1*
A modifier to apply to the time of every Fermenter recipe

refinery_energyModifier *float, default 1*
A modifier to apply to the energy costs of every Refinery recipe

refinery_timeModifier *float, default 1*
A modifier to apply to the time of every Refinery recipe. Can't be lower than 1

arcFurnace_energyModifier *float, default 1*
A modifier to apply to the energy costs of every Arc Furnace recipe

arcFurnace_timeModifier *float, default 1*
A modifier to apply to the time of every Arc Furnace recipe

arcfurnace_electrodeDamage *int, default 96000*
The maximum amount of damage Graphite Electrodes can take. While the furnace is working, electrodes sustain 1 damage per tick, so this is effectively the lifetime in ticks. The default value of 96000 makes

them last for 8 consecutive ingame days

arcfurnace_electrodeCrafting *boolean, default* `false`
Set this to true to make the blueprint for graphite electrodes craftable in addition to villager/dungeon loot

arcfurnace_recycle *boolean, default* `true`
Set this to false to disable the Arc Furnace's recycling of armors and tools

autoWorkbench_energyModifier *float, default* `1`
A modifier to apply to the energy costs of every Automatic Workbench recipe

autoWorkbench_timeModifier *float, default* `1`
A modifier to apply to the time of every Automatic Workbench recipe

bottlingMachine_energyModifier *float, default* `1`
A modifier to apply to the energy costs of every Bottling Machine's process

bottlingMachine_timeModifier *float, default* `1`
A modifier to apply to the time of every Bottling Machine's process

mixer_energyModifier *float, default* `1`
A modifier to apply to the energy costs of every Mixer's process

mixer_timeModifier *float, default* `1`
A modifier to apply to the time of every Mixer's process

assembler_consumption *int, default* `80`
The Flux the Assembler will consume to craft an item from a recipe

excavator_consumption *int, default* `4096`
The Flux the Bottling Machine will consume per tick, when filling items The Flux per tick the Excavator will consume to dig

excavator_speed *double, default* `1d`
The speed of the Excavator. Basically translates to how many degrees per tick it will turn.

excavator_particles *boolean, default* `true`
Set this to false to disable the ridiculous amounts of particles the Excavator spawns

excavator_chance *double, default* `.2d`
The chance that a given chunk will contain a mineral vein.

excavator_fail_chance *double, default* `.05d`
The chance that the Excavator will not dig up an ore with the currently downward-facing bucket.

excavator_depletion *int, default* `38400`
The maximum amount of yield one can get out of a chunk with the excavator. Set a number smaller than zero to make it infinite

excavator_dimBlacklist *int[], default* `new int[]{1}`
List of dimensions that can't contain minerals. Default: The End.

ore_copper *int[], default* `new int[]{8, 40, 72, 8, 100}`
Generation config for Copper Ore. Parameters: Vein size, lowest possible Y, highest possible Y, veins per chunk, chance for vein to spawn (out of 100). Set vein size to 0 to disable the generation

ore_bauxite *int[], default* `new int[]{4, 40, 85, 8, 100}`
Generation config for Bauxite Ore. Parameters: Vein size, lowest possible Y, highest possible Y, veins per chunk, chance for vein to spawn (out of 100). Set vein size to 0 to disable the generation

ore_lead *int[], default* `new int[]{6, 8, 36, 4, 100}`
Generation config for Lead Ore. Parameters: Vein size, lowest possible Y, highest possible Y, veins per chunk, chance for vein to spawn (out of 100). Set vein size to 0 to disable the generation

ore_silver *int[], default* `new int[]{8, 8, 40, 4, 80}`
Generation config for Silver Ore. Parameters: Vein size, lowest possible Y, highest possible Y, veins per chunk, chance for vein to spawn (out of 100). Set vein size to 0 to disable the generation

ore_nickel *int[], default* `new int[]{6, 8, 24, 2, 100}`
Generation config for Nickel Ore. Parameters: Vein size, lowest possible Y, highest possible Y, veins per chunk, chance for vein to spawn (out of 100). Set vein size to 0 to disable the generation

ore_uranium *int[], default* `new int[]{4, 8, 24, 2, 60}`
Generation config for Uranium Ore. Parameters: Vein size, lowest possible Y, highest possible Y, veins per chunk, chance for vein to spawn (out of 100). Set vein size to 0 to disable the generation

oreDimBlacklist *int[], default* `new int[]{-1, 1}`
A blacklist of dimensions in which IE ores won't spawn. By default this is Nether (-1) and End (1)

retrogen_log_flagChunk *boolean, default* `true`
Set this to false to disable the logging of the chunks that were flagged for retrogen.

retrogen_log_remaining *boolean, default* `true`
Set this to false to disable the logging of the chunks that are still left to retrogen.

retrogen_key *String, default* `"DEFAULT"`
The retrogeneration key. Basically IE checks if this key is saved in the chunks data. If it isn't, it will perform retrogen on all ores marked for retrogen. Change this in combination with the retrogen booleans to regen only some of the ores.

retrogen_copper *boolean, default* `false`
Set this to true to allow retro-generation of Copper Ore.

retrogen_bauxite *boolean, default* `false`
Set this to true to allow retro-generation of Bauxite Ore.

retrogen_lead *boolean, default* `false`
Set this to true to allow retro-generation of Lead Ore.

retrogen_silver *boolean, default* `false`
Set this to true to allow retro-generation of Silver Ore.

retrogen_nickel *boolean, default* `false`
Set this to true to allow retro-generation of Nickel Ore.

retrogen_uranium *boolean, default* `false`
Set this to true to allow retro-generation of Uranium Ore.

disableHammerCrushing *boolean, default* `false`
Set this to true to completely disable the ore-crushing recipes with the Engineers Hammer

hammerDurabiliy *int, default* `100`
The maximum durability of the Engineer's Hammer. Used up when hammering ingots into plates.

cutterDurabiliy *int, default* `250`
The maximum durability of the Wirecutter. Used up when cutting plates into wire.

bulletDamage_Casull *float, default* `10f`
Enable this to use the old, harder bullet recipes(require one ingot per bullet) The amount of base damage a Casull Cartridge inflicts

bulletDamage_AP *float, default* `10f`
The amount of base damage an ArmorPiercing Cartridge inflicts

bulletDamage_Buck *float, default* `2f`
The amount of base damage a single part of Buckshot inflicts

bulletDamage_Dragon *float, default* `3f`
The amount of base damage a DragonsBreath Cartridge inflicts

bulletDamage_Homing *float, default* `10f`
The amount of base damage a Homing Cartridge inflicts

bulletDamage_Wolfpack *float, default* `6f`
The amount of base damage a Wolfpack Cartridge inflicts

bulletDamage_WolfpackPart *float, default* `4f`
The amount of damage the sub-projectiles of the Wolfpack Cartridge inflict

bulletDamage_Silver *float, default* `10f`
The amount of damage a silver bullet inflicts

bulletDamage_Potion *float, default* `1f`

The amount of base damage a Phial Cartridge inflicts

earDefenders_SoundBlacklist *String[], default* `new String[]{}`
A list of sounds that should not be muffled by the Ear Defenders. Adding to this list requires knowledge of the correct sound resource names.

chemthrower_consumption *int, default* `10`
The mb of fluid the Chemical Thrower will consume per tick of usage

chemthrower_scroll *boolean, default* `true`
Set this to false to disable the use of Sneak+Scroll to switch Chemthrower tanks.

railgun_consumption *int, default* `800`
The base amount of Flux consumed per shot by the Railgun

railgun_damage *float, default* `1f`
A modifier for the damage of all projectiles fired by the Railgun

powerpack_whitelist *String[], default* `new String[]{}`
A whitelist of armor pieces to allow attaching the capacitor backpack, formatting: [mod id]:[item name]

powerpack_blacklist *String[], default* `new String[]{"embers:ashen_cloak_chest", "ic2:batpack", "ic2`
A blacklist of armor pieces to allow attaching the capacitor backpack, formatting: [mod id]:[item name]. Whitelist has priority over this

toolbox_tools *String[], default* `new String[]{}`
A whitelist of tools allowed in the toolbox, formatting: [mod id]:[item name]

toolbox_foods *String[], default* `new String[]{}`
A whitelist of foods allowed in the toolbox, formatting: [mod id]:[item name]

toolbox_wiring *String[], default* `new String[]{}`
A whitelist of wire-related items allowed in the toolbox, formatting: [mod id]:[item name]

CHAPTER 20

The in-game manual, in full

The Engineer's Manual as the book itself presents it, transcribed so the two cannot disagree.

81 chapters, transcribed from the in-game book.

20.0.1 Alloys

Mixed Metal Arts

Mixing dusts creates an alloy blend that can then be smelted into alloyed ingots. Constantan has thermo-electric properties and is used in creating the <link;generator;thermoelectric generator;1> and electrum is an excellent electric conductor used in <link;wiring;medium-voltage architecture>.

20.0.2 Alloy Kiln

Mixin' Metals

The Alloy Kiln is a simple furnace made out of heat resistant sandstone and bricks. Within it, temperatures are high enough to allow metals to be melted into a fluid state and mixed, creating an <link;alloys;alloy>.

This allows the creation of important alloys like Electrum and Constantan without requiring the tedious mixing of dusts.

To form the Kiln, arrange 8 of the Kiln Bricks in a 2x2x2 cube, as shown above, and rightclick one of the sides' blocks with an Engineer's Hammer.

20.0.3 Arc Furnace

Not a reactor

The Arc Furnace is a massive structure used to quickly smelt metals. It consists of a steel vat topped by electrodes which heat up the metal.

The furnace excels at smelting ores and creating steel due to its high speed and its ability to process multiple items at the same time, but it is also complex in its workings and will require maintenance in order to continue functioning.

Slag must be removed from the machine or it will cease to function, and degrading electrodes need to be replaced over time.

To form the structure, assemble it as shown on the first page, then click the cauldron with an Engineer's Hammer.

Energy is input through the three connectors at the back, and supplying the control panel at the front with a redstone signal will turn off the machine. Right-clicking the panel, 'bucket' or vat will open the interface.

Interface:

The interface may look confusing upon first seeing it, but it's quite straight forward.

The three slots at the top are for the <link;graphite;Graphite Electrodes;1>.

The twelve slots on the left are the input slots for metals and ores. Each slot has a temperature bar next to it, representing the progress of smelting.

The four slots on the right are for "additives", different materials added to the molten metal during the crafting process. An example of these would be adding Coke Dust to melting Iron in order to create Steel.

Inputs and additives are inserted through the two hatches at the top of the structure, inputs to the left and additives to the right of the electrodes.

The seven slots at the bottom are six output slots and a slot for slag. Outputs are retrieved from the 'bucket' at the front of the structure, slag through the port at the back of the machine. The Arc Furnace will automatically push output and slag to connected inventories (this includes Conveyor Belts).

20.0.4 Assembler

Some assembly required

The Assembler is a complex machine capable of crafting items, similar to how a player would.

To form the structure, build it like demonstrated above and use the hammer on one of the conveyors.

The conveyor you clicked will be the input side, indicated by the blue marker above it and the arrow on the side of the machine.

The Assembler allows for up to 3 recipes to be stored inside it that will be performed in order. The outputs of previous recipes can be used as inputs for the following.

The 3 crafting grids store the recipes, and the 18 slots below them are the inventory for materials.

The input side, in addition to an item input also features a fluid input to fill the three internal tanks of the machine. Fluid in these tanks can be used to replace fluid containers like water buckets in recipes.

Power for the machine is input through the top center block.

20.0.5 Automated Workbench

For those who can't drive stick

The Automatic Workbench is one of the first major successes of small-scale engineering.

The machine is able to follow a set of instructions described by a <link;blueprints;Blueprint> to assemble the items detailed by it.

Upon assembling the multiblocks, a Blueprint can be placed on its drawing table. After selecting the item that should be created, it will begin the assembly process. In order for the machine to work, it will need to be supplied with materials and power of course.

Item inputs are the two hatches marked with blue dots, the power input lies above them. Finished items are dropped at the end of the conveyor belt.

A redstone signal below the drawing table can be used for control.

20.0.6 Balloon

...just make 99 of them

Balloons are sacks of fabric with a heatsource below to keep them afloat. They will also emit light, but despite the torch, they are not a fire hazard.

Balloons can be placed like normal blocks, or, when no block is targeted, they will be placed in the air infront of you.

Similar to <link;metalconstruction;Structural Connectors;5>, you can attach Hemp Rope and Steel Wire to the balloon.

Sneak-clicking allows you to set a height offset for the balloon, allowing you to place it up to 5 blocks above the target.

You can also use dyes on a placed balloon; which part you dye depends on where you target it.

Rightclicking with an Engineer's Hammer will toggle the balloon's design between horizontal or vertical stripes.

20.0.7 Wooden Barrel

Doesn't roll

Wooden barrels are simple containers that can store fluid. They will store up to 12 buckets, but because they are wooden, they can't store any gasses or hot fluids.

Rightclicking the top or bottom side of the barrel with a Hammer will switch between in-, output or no connection.

20.0.8 Garden Cloche

Localized Rainforest

The Garden Cloche, or Belljar, is a device designed to hasten the growth of plants. It allows even the an agriculturally challenged engineer to grow plants and harvest their yield.

The device accepts power, fluid and items through its respective ports. Water and power are always required, various fertilizers are optional. The GUI features slots for soil and seed in the middle, fertilizer on the left and outputs on the right.

The hatch on the front of the glass dome is the only automatic output. Every seed needs to be paired with the correct soil to grow on, with dirt being automatically recognized as farmland.

20.0.9 Biodiesel

Green power!

Combustible fuels are an easy to use and efficient fuel source. But with oil being a depletable resource, something renewable would be much preferable. Biodiesel is the solution! By combining plant oil and ethanol, you can create a combustible fluid that will burn in the <link;dieselgen;Diesel Generator>.

Plant Oil is created by processing various seeds in the industrial squeezer. Create the structure pictured above and click one of the sides with an Engineer's Hammer to form it. Power is input at the top or bottom, fluids are output through any of the ports at the

Ethanol is gained by fermenting sugarcane or other fruit. The fermenter's structure is created in a similar manner to that of the squeezer, as such power in- and fluid outputs are identical.

Lastly, in order to refine plant oil and ethanol into Biodiesel, you will need to insert both fluids into a refinery and supply it with power. The structure is created as shown above and formed by clicking the center of the front with an

Engineer's Hammer.

The front of the refinery serves as fluid output, whereas the inputs are found on the smaller sides. Energy is supplied through the connector at the back. Applying a redstone signal to the control panel on the right will halt the refinery's process.

20.0.10 Crude Blast Furnace

Simple Steel

The Crude Blast Furnace is used to increase the carbon contents in iron, turning it into steel. To achieve this, only pure, high-carbon fuels may be used for the furnace, specifically coal coke and charcoal.

Along with steel, the furnace will also create slag, which you must remove or the furnace will cease to function. While technically a waste product, you can use slag to create concrete.

The Crude Blast Furnace is not automateable. All in- and outputs must be arranged manually through its GUI. If you want to automate the creation of steel, you should probably look into creating an <link;improvedBlastfurnace;Improved Blast Furnace;>.

20.0.11 Engineer's Blueprints

Back to the drawing board

This page features all Engineer's Blueprints that can be crafted by normal means. Other Blueprints may need to be acquired from villagers or found in the world. All of these can be used in an <link;workbench;Engineer's Workbench> to craft the items they describe.

20.0.12 Bottling Machine

Fill 'er up!

The Bottling Machine fills fluids into containers.

The structure is created as shown above, but can be mirrored. To form it, rightclick the central coveyor with an Engineer's Hammer. The direction of the conveyors is important.

Fluid is input through the ports on the lower block of the metal tank. Energy is input through the port at the top of the opposing side.

Generally, the Bottling Machine will be able to fill any container that has a filled version available (bucket, glass bottle). It will also accept other items that can store fluids, like the <link;drill;Mining Drill> or a <link;jerrycan;Jerrycan>.

Additionally, special recipes may be available.

20.0.13 Breaker Switch

Clap on - Clap off

Breaker Switches are simply switches that can stop the flow of power through wires. They can connect two wires of the same type and will only allow power to pass when activated. This can be used to separate power-hungry machinery from the network or hook up an additional set of generators when needed!

Due to the high risk, these switches will not accept HV wires.

They will also emit a redstone signal when they are switched on; this behaviour can be inverted by clicking the switch with an Engineer's Hammer.

Additionally they are an excellent way to control your <link;lighting;powered lights;1>!

The Redstone Breaker is similar to a normal breaker switch, but cannot be operated manually. Instead, it will require a redstone signal to break the connection. In return, this device is insulated well enough to allow the connection of HV wires as well!

20.0.14 Revolver Cartridges

Cartridges are created in the Engineer's Workbench, by the use of specific <link;blueprints;Engineer's Blueprints>. Blueprints for normal ammunition you can create yourself, but some more advanced types of ammunition will require you to buy adequate blueprints from someone else.

Empty Casings and Shells are the basis for every projectile. Empty Casings can also be crafted in the <link;metalPress;Metal Press>, using the appropriate mold, at a greatly reduced cost.

20.0.15 Charging Station

Charge your glasses!

The Charging Station is a device that allows you to charge tools and other items. Energy can be input through the bottom or the back and items can be placed in manually or inserted automatically.

The electron tubes at the front give a feedback about the current charge of the item, but the block will also output a comparator signal for the same purposes.

20.0.16 Chemical Thrower

BURN ALL THE THINGS!

The Chemical Thrower is a device that propels fluids from its internal tank. The device can be loaded with any fluid, liquid or gas, and, on right click, will spray it out.

It also features a function to ignite the fluid, if it is flammable. To toggle the pilot light, sneak and rightclick.

Like some other tools, the chemical thrower can be modified in the <link;workbench;Engineer's Workbench>. These upgrades affect the size of the tank, range and spread.

The Large Tank increases the fluid capacity of the Chemical Thrower by 2000 mb.

The Focused Nozzle decreases the spread of fluids sprayed out and also slightly increases their range, allowing you to hit further targets with more precision.

The Multitank allows the Chemthrower to store up to three different Fluids. You can select the active tank by <config;b;chemthrower_scroll;sneaking and using the scrollwheel, or through use of;using> the keybind <keybind;key.immersiveengineering.chemthrowerSwitch> if it has been set in the options.

This upgrade is mutually exclusive to the Large Tank.

20.0.17 Coke Oven

Hot topic!

The Coke Oven is the first important machine you will need to create in order to proceed with Immersive Engineering. Its functionality is simple, it will heat up coal or blocks of coal without supplying it with oxygen, creating Coal Coke, a carbon rich fuel. More importantly, this process creates Creosote Oil, which is used as a <link;treatedwood;preservative for wood>. Similarly you can also burn wooden logs into charcoal with this oven, albeit with a smaller yield of creosote.

20.0.18 Crafting Components

Integral Pieces

Many machines and tools are constructed from various sub-components. Some of these you will have to use on a regular basis, which is why they have their own entry in this manual. The Iron and Steel Mechanical Components are integral to the majority of Multiblocks and machinery.

As you will find yourself creating these components more frequently, it may be advisable to draft a <link;blueprints;Blueprint> for them. Using this blueprint in an <link;workbench;Engineer's Workbench> will allow you to construct the components at a reduced cost.

The workbench recipes are found on the following page.

Another, albeit less frequently used component are Vacuum Tubes. These glass tubes are used as elements in electronic machinery and used in <link;lighting;light emitting devices>.

A rarely used, but crucial component in advanced machinery that requires precise calculation is the Circuit Board. They are impractically large and clunky, but can perform basic mathematic operations in under a second!

20.0.19 Concrete

Fortress Material

Concrete is a very hard and resistant composite material. It's cheap to craft and makes for an excellent use for the excess slag produced by the <link;blastfurnace;Blast Furnace>.

20.0.20 Conduit

Wiring that stays on the wall

Wire sags. Strung across a valley that is exactly right, and strung along a ceiling it is exactly wrong.

Conduit is the indoor answer: thin tubing that clips flat to whatever face you put it against and runs in that face, turning in right angles. One length of it carries sixteen independent conductors, named after the sixteen dyes.

Conduit is cheap and comes eight at a time. Place it against a wall, a floor or a ceiling and it clips to whatever you clicked.

Lengths joined along a surface make a run, and a run wraps around corners: it climbs the wall it reaches and follows the edge of the beam it is on around onto the next face. Click whichever face makes sense - the wall ahead of the run, the far side of the edge, or the exposed face of the last length - and you get the piece that turns the corner.

An inner corner is turned inside the last length's own block, so it needs something in the corner to turn around; a doorway at the foot of the wall leaves nothing there and the run stops. An outer corner needs nothing at all.

A run has to end somewhere, and that somewhere is a Junction Box. A run arrives at a box face on rather than around a corner, so put the box at the corner: it sits in the plane of whichever run reaches it and both meet it flush.

Lay conduit between two boxes and the run connects itself. There is no coil to craft and no tool to click with: the run is the thing you built, and pulling a length out of the middle of it breaks the connection exactly as you would expect.

The box is where a bundle splits.

String an LV, MV or HV wire straight at a face of a junction box, or put a Connector of that tier - or a Grid Feed or Service Unit - against one, and the box breaks a conductor out to that face by itself. A breakout reaches the block edge wearing its conductor's colour at the mouth, so whatever is bolted against that face meets it flush and you can read the circuit from across the room.

Which colour it picks is fixed. Looking at a box: red on the left, blue in the middle - on top, for a box on a pole - and green on the right. Two boxes wired the same way come out the same, which is what makes a row of poles readable from the ground.

To change a breakout, sneak and hit the face with an Engineer's Hammer: it steps to the next conductor, and after the last one takes the breakout away entirely. Colours already spent on another face of the same box are stepped over - the same conductor arriving at two connectors is a short, not a circuit. A dye still does it in one hit when you know which colour you want.

A box passes the whole bundle through whether or not anything is patched, so a box dropped in to turn a corner or change surface needs no setting up at all.

Sneak and right-click a face empty-handed to unpatch it.

Right-click a patched face with Redstone Dust and it cycles: power, then reading redstone, then emitting it.

A conductor carrying a signal reaches every box on the run, so a lever at one end of a corridor throws a lamp at the other along the same bundle already carrying its power. A face does one of the three and never two - which is what stops a run from latching itself on and refusing to let go.

A junction box is the right way to leave a surface indoors, where it reads as hardware. Out in a field it is a steel box sitting in a lawn.

The Ground Feeder is the other answer.

Set a feeder into a floor, a wall or the ground and it puts on whatever is around it - grass in a lawn, stone in a cellar wall. It looks at what is visible rather than at what is merely nearby, so one buried at ground level wears the turf and not the dirt beneath it.

A run goes straight through it along one axis, which an Engineer's Hammer turns; the conduit on the far side may be clipped to a different surface, and crossing a feeder is what lets a run change to a surface with no corner between the two. It splits nothing - a breakout is a thing you should be able to see.

Right-click it with any full block to pin that block instead, or sneak and right-click it empty-handed to let it survey again.

A junction box is a wire endpoint in its own right. Point a coil at the face you want and the wire attaches there - no connector bolted to the wall beside it, and nothing to explain.

One wire per face, and the face you clicked is the circuit you get: six faces, six circuits, six conductors. The face the box is bolted to is not one of them, and a box that already has a wire everywhere it can take one will say so.

The tier of a circuit is the tier of the wire on it. An HV wire on one face and an LV wire on another are two circuits of two tiers coming out of the same bundle. Cut one off with Wirecutters and the face is free for another; the conductor stays patched where it was.

The connectors still work. Three of them round a box with the runs buried is what an underground feeder looks like, and nothing about it has changed.

20.0.21 Conveyor Belts

Who needs rails anyway?

Conveyor Belts are an easy way of transporting all sorts of things around your base. While they are strong enough to move living creatures like cows or sheep, their true strength lies in transporting items.

Throwing items atop them will have the items move along the belt. Should the belt lead into an inventory, the item will be inserted.

Blocks like hoppers will be able to drop items onto the belts. Use an Engineer's Hammer to rotate the belts (rightclick) or adjust whether they should run up- or downwards (sneak + rightclick).

Using a redstone signal on a conveyor will turn it off, this behavior can be disabled by crafting it into a redstone-ignoring version.

Rightclicking a conveyor with any dye will colour the walls of the conveyor to easily identify where it is going.

The Dropping Conveyor Belt is quite simply a conveyor with a hatch in the middle, allowing it to drop items that pass over it downwards.

Items will be inserted into inventories below or simply dropped if there is an empty space below the conveyor.

The Extracting Conveyor Belt possesses the ability to extract items from an inventory in a similar fashion to a Hopper. Unlike the latter however, it posesses no internal inventory and will place extracted items on the belt.

Sneak + rightlick with an Engineer's Hammer to change its input side. Use a wirecutter to adjust its tickrate.

The Vertical Conveyor Belt is a belt meant to be mounted on walls. Using the buckets attached to it, it will lift entities straight up. Despite the use of the term "bucket", it will not lift fluids.

This belt can insert into inventories above it or simply expel items over the edge at the top onto more conveyors.

The Splitting Conveyor Belt alternates between redirecting Items and Entities left and right, plain and simple. This feature can be used to achieve a more equal distribution in your factories.

The Covered Conveyor Belt works just like a normal conveyor, however it is covered by a piece of scaffolding, which prevents players from picking up items from it.

You can rightclick it with other types of scaffold or glass to change the cover's appearance.

Similarly, the Dropping, Extracting and Vertical Conveyors can also be covered in scaffolding to achieve the same benefits of "pickup-prevention" while still functioning as they normally do.

Chutes are simple constructions made from sheetmetal that allow entities to drop down straight. They can insert into inventories below them and be inserted into by conveyors or similar devices. Items within them won't be picked up by players and should be immune to most forms of attraction, magnetic or magical.

Sneak + rightclick with an Engineer's Hammer to turn the chute into a corner piece that directs an entity to the side.

20.0.22 Wooden Storage Crate

Move that gear up!

Wooden storage crates are blocks quite similar to a chest. Crafted with treated wood, they offer 27 slots of storage, however, when broken, they will keep that storage, allowing you to move them together with their contents.

Like chests, they will output a comparator signal based on the amount of items stored.

The reinforced storage crate offers the same capacity as a normal crate, but unlike them, these creates are impervious to explosions. No longer will pesky creepers or ill-placed mining charges destroy your valueables!

The upgrade recipe will keep the contents of the crate.

20.0.23 Hydraulic Crawler

A machine you climb into

The Excavator on the facing page is a building. This is not.

The Hydraulic Crawler is a tracked machine that arrives as an item, unfolds where you put it, and is driven. It needs five clear blocks across to unfold into. Right-click it to climb in; there is no GUI, and everything it does is done from the seat.

Driving. W and S drive the tracks, A and D steer them. It is heavy: about a second to reach full speed, rather less to stop, and it brakes harder than it accelerates so it does not coast. Reverse is slower, as it is on anything tracked, and it turns tightest standing still. A one-block ledge is climbed rather than bumped into.

Looking. The cab turns to wherever you look, at the speed a slew ring turns. The house follows your head, so turning to face what you are working on is the same gesture as slewing towards it. You cannot look around independently of the machine; that is the trade.

The arm. It holds its own aim rather than following your view, so you can watch what you are digging while you dig it.

R and C raise and lower it. X and Z reach out and draw back in. Those are two separate axes, and you need both: elevation alone would put the tool on a single arc, so anything nearer or further would be out of reach wherever you parked.

The panel in the corner shows both, and turns red at the stops.

The tools. G changes attachment, V works it.

The Bucket digs and keeps what it digs. Sneak and V tips the load out at the machine's feet - there is no other way to unload it by hand, though a pipe or a conveyor against the machine will empty it too. When the bucket fills it drops the rest on the floor rather than eating it.

The Breaker destroys what it touches and leaves it on the ground. The Grapple picks one block or one creature up and puts it down again.

Diesel, and nothing else. Right-click the machine with a container of Diesel to fuel it - the refined cut, not Biodiesel. The Gas Station Pump will fill it.

The engine burns while anyone is aboard, and the attachment burns more. Below 400mB the tool stops but the tracks keep turning: that reserve exists so the machine cannot strand you a long way from home, in a thing that cannot be pushed.

Taking it away. Sneak and right-click with an Engineer's Hammer. It comes back as an item, with its diesel still in it.

20.0.24 Crusher

Keep your hands out!

Crushing an ore is an effective way of increasing their yield, for each ore creates two dusts which you can then smelt. This industrial-size crusher is a quick way of doing so. The machine's processing speed depends on the power supplied to it.

Applying a redstone signal to the control panel at the front will halt the crushers process, this behavior can be inverted by use of the Engineer's Hammer.

The structure is formed by right clicking the central block of the front. The front is the wide side of the structure which has a light engineering block at the bottom.

20.0.25 Diesel Generator

Better than volume: More volume.

High-Voltage architecture might be fascinating, but it's not very useful unless you have a generator that can actually create the required output for it.

The Diesel Generator is one of the few ways of generating this sort of energy. It is created as a complex multiblock structure and will run off of <link;refinery;Biodiesel> or other types of combustible fuel. Due to its high output, it will work through fuel very quickly, so make sure you have a sufficient fuel production to run it.

The structure is built from Engineering, Generator and Radiator blocks as well as Fluid Pipes and Scaffolding. Arrange them in the shape shown above and rightclick the central generator with an Engineer's Hammer.

Fuel is to be input at the bottom corners of the generator, whereas energy will be output to up to three connections on the top. The <config;i;dieselGen_output>Flux/t the generator outputs will be split between all connected points.

The small terminal on the side will turn off the generator if supplied with a redstone signal, this behavior can be inverted by use of the Engineer's Hammer.

The following table lists all valid fuels for the Diesel Generator and for how long in seconds (ticks) they burn.

20.0.26 Mining Drill

My drill will pierce the Bedrock!

The handheld, diesel-powered drill is a big advancement in mining. With exchangeable drill heads and upgrades, its properties are almost entirely modifiable. To modify it, place it in an <link;workbench;Engineer's Workbench>.

The drill runs off of <link;refinery;Biodiesel> and can be filled directly in the refinery.

The Drill Head determines the base mining level, speed, and size, as well as the attack damage. As you dig blocks, the drill head takes damage, but it can be repaired in an anvil.

As the drill is powered by a combustion engine, it will not run underwater unless you equip it with the Pressurized Air Tank Upgrade. However, with this upgrade equipped, it will not only run while submerged, but also ignore the slowdown usually experienced when digging in water.

The Advanced Lubrication System applies additional lubricant to the drill at a regular basis, decreasing the damage done to the drillhead.

By adding Additional Augers to the drill, its destructive potential is increased, increasing its damage against living creatures and allowing it to run and dig faster.

The upgrade stacks up to three times.

The Large Fuel Tank allows the drill to store more fuel internally, extending the time you can use it before refilling.

20.0.27 Current Transformer

88 Flux per hour!

The Current Transformer is a measurement device for electrical power. It can be hooked up in any powerline and will keep track of energy transferred through it, returning an average Flux/t upon rightclick.

20.0.28 Ear Defenders

LOUD NOISES!

Ear Defenders are devices for protecting your hearing. They are quite important for engineers that work in close proximity to <link;dieselgen;Diesel Generators;> or other loud machinery.

While wearing Ear Defenders, all noises are reduced by 90%%.

Placing the Ear Defenders in the <link;workbench;Engineer's Workbench;> allows you to specify which types of noises you want cancelled and by how much their volume should be reduced.

You can dye them like you would leather armor.

Crafting them with any other helmet will attach the defenders to that piece of equipment, crafting the helmet alone will remove the defenders again, to allow further customizing.

20.0.29 Excavator

Does it have school bus mode!?

The Excavator is one of the pinnacles of modern engineering. It is able to dig up <link;minerals;minerals;0> from veins inaccessible by normal mining. In return however, the excavator is a complex structure which requires <config;i;excavator_consumption> Flux/t in order to operate.

The structure is comprised of two pieces: The engine shown on the first page and the bucket wheel displayed on the second, which fits into the 'braces' of the engine as displayed on the previous page. The energy input for the engine is on its side, the back features a redstone control panel and the item output.

As the excavator digs up ores, the yield of the vein slowly decreases, but a vein of minerals can last for <config;i;excavator_depletion_days> days of consecutive digging.

20.0.30 Fermenter

Skál!

The Fermenter is a multiblock structure that extracts ethanol from certain fruits, grains and vegetables through a fermenting process. The structure is built as shown above and formed by using the Engineer's Hammer on the central cauldron, from the side that has the Engineering Blocks.

Items can be input into the Fermenter via the two hatches at the back that are marked with blue dots. The hatch with the orange dot serves as an item output and the block below it to output fluids.

Applying a redstone signal to the control panel at the front will halt the machine's process, this behavior can be inverted by use of the Engineer's Hammer.

20.0.31 Fluid Networks

Mains without the trench

A gas main under every road of a town is thousands of pipe blocks, and every one of them is a thing the server has to think about. The Virtual Fluid Network is the same answer the Virtual Power Grid gives for Flux: named mains that move fluid between places with no pipe between them at all.

It is not only for oil. A main carries whatever you put in it - water, steam, heating oil - and does it at the same cost.

Fluid still has to enter and leave somewhere. It enters at a Fluid Inlet and leaves at a Fluid Outlet: service boxes that bolt to any solid surface, including engineering posts.

An Inlet draws from whatever it is bolted to - a tank's fill cap, a wellhead - and will also take a pipe run into any of its faces. An Outlet fills whatever it touches, trying the block it is bolted to first.

One Fluid Console Housing and one each of the three engineering blocks, in a 2x2 wall struck on the face that becomes the screen, make the Fluid Control Console.

Housing top left, Redstone Engineering beside it, Light Engineering under the housing and Heavy Engineering at the bottom right. Hammer any of the four.

The Fluid Control Console is where mains are made and fittings are assigned. Four tabs: an overview, the mains themselves, the fittings on them, and a minute of throughput history.

It runs on a trickle of Flux. Losing power darkens the screen and makes the window read-only rather than refusing to open - being locked out of your own network because the lights are off would be a trap.

A main carries one fluid. It takes its type from the first Inlet with something to offer, and after that an Inlet holding anything else is simply ignored - which is what stops a mis-plumbed fitting quietly re-typing a live main and feeding every machine on it the wrong thing.

To change it, drain the main first. The console will not offer the control until the line pack is empty, and says so.

Each main has caps on what may flow in and out per tick, a small line pack so fluid collected this tick can be delivered this tick, and optional leakage. The pack is smoothing, not storage - the buried tanks are what storage is for.

Outlets marked critical are served first when supply is short, and a main can name another as a backup - but only one carrying the same fluid will ever actually cover it.

A Main Valve is a shut-off one way round and a pressure lamp the other. As a shut-off, a redstone signal holds its main closed; inverted, it demands a keep-open signal instead, which makes it a dead-man's switch.

As an indicator it emits while the main is flowing, or - inverted - while it is not, which is a low-pressure alarm. On a real network the fitting that closes a branch and the one that tells you the branch is live are the same thing.

The Fluid Linker is the Grid Linker's twin on this side of the plant.

Rightclick any fitting with it and a short list of mains appears - name, colour, and what each is doing. Pick one and it links that fitting and loads the tool; every rightclick after that links the fitting you clicked.

Sneak-rightclick a fitting to choose a different main, and sneak-rightclick the air to empty the tool. A locked main stays locked.

20.0.32 Fluid Transport

Professional Plumber

Transporting fluids has become quite essential in the engineering business. Fluid Pipes and Fluid Pumps do exactly that. On their own, Fluid Pipes are quite slow at transferring fluids, but if a pump is used to insert the fluid, it'll move much faster.

You can rightclick the connection piece of a pipe with an Engineer's Hammer to prevent it from connecting to that side.

Fluid Pipes can be covered with wooden or steel scaffolding. Simply rightclick a piece of scaffolding onto a pipe to cover it. Break the pipe to retrieve the scaffolding.

Fluid Pumps have three essential functions. They can be used to insert into pipes at high pressure, to transfer fluids much faster, or they can pick up fluids from the world. Using your hammer on the base of the pump will switch the sides between in- and output.

In order for the pump to pick up fluid blocks, it will need a power input at the top and a redstone signal. Pumps can also be used to extract from tanks and other fluid containers that do not have an automatic output. This will also require a redstone signal, but won't consume power.

<config;b;pump_infiniteWater;Water blocks will serve as an infinite fluid source for the pump, as long as more than 3 blocks are connected.

;><config;b;pump_placeCobble;Fluid blocks that were picked up will be replaced with cobblestone to reduce the lag of flowing fluids. This behaviour can be toggled off and on by using an Engineer's Hammer on the pump.;>

The Fluid Outlet acts as an endpoint for a pipe system, allowing Fluid to be placed in the world. The sides can be configured between input, output (open gate) or closed, a redstone signal can be used to turn the outlet off.

The block will only ever output fluid up to its own heightlevel.

20.0.33 Fluid Router

Milk goes left, acid goes right

The Fluid Router works much akin to the <link;sorter;Item Router>. Fluids input through any side will be redistributed based on how the six side filters are set up. Clicking a bucket or any other type of fluid container into the slots of the GUI will configure the filter to that fluid.

When any fluids are piped into the router, they will be routed to an output whose filter matches the fluid, or, if no matching filter is found, the fluid routes to an unfiltered output. If there is no available or valid route, no fluid will be accepted.

The buttons above each filter can be used to have the router respect or ignore the fluid's NBT data.

20.0.34 External Heater

Now you're cooking with flux!

The External Heater does exactly what its name implies, it generates thermal energy from Flux. When placed next to a normal furnace, this thermal energy will power the furnace, allowing it to smelt without any fuel inside it.

Click a side of the heater with a hammer to make it an energy acceptor, sneak to target the opposing side.

The heater will consume <config;i;heater_consumption> Flux/t for each heat unit added. During the initial phase of the furnace, the heater will produce up to 4 units of heat per tick, but as soon as the furnace reaches maximum temperature, it will only require 1 unit of heat to keep it going.

Even better, while the furnace is at maximum temperature, the heater will consume an additional <config;i;heater_speedupConsumption> Flux/t to increase the furnace's processing speed.

The heater will only affect a furnace with a valid input and space in the output slot, in order to conserve energy. However, you can apply a redstone signal to the heater to disable that functionality and have it keep the furnace at full heat.

20.0.35 Power Generation

Transferring power is quite helpful provided you have power. To actually generate Flux, you can use the kinetic forces of windmills and waterwheels. Connecting these to a dynamo will generate power based on how fast they turn.

The Water Wheel is a fairly easy way of powering a dynamo. It's turning speed is based on the water flowing around it, so for optimal results, you'll want to direct the water in a semi-circle from the top down one side and along the bottom. Up to three water wheels can be placed against each other.

The Windmill is a free and easy way of generating power, albeit not a very powerful one. Its speed depends the space in front of it, with its kinetic output reducing greatly if the airflow is obscured. Additionally, weather conditions like rain or thunderstorms result in stronger winds and increased rotation speeds.

Your wind-based power generation can be improved by covering the blades of your windmill in cloth. Doing so increases the surface area and with it the rotation speed and energy output.

The required sails are crafted from <link;hemp;tough hemp fabric;1> and applied to the windmill by rightclicking it in world.

20.0.36 Graphite

Coal stuff

Highly Ordered Pyrolytic Graphite (HOP) is a complex, highly compressed, carbon material used in special engineering constructs.

HOP Graphite Dust is created by compressing eight pieces of Coke Dust in the <link;squeezer;Industrial Squeezer>. That dust can then be smelted into an ingot.

The most common use for HOP Graphite is the creation of electrodes to be used in the <link;arcfurnace;Arc Furnace>. These electrodes are created with an <link;blueprints;Engineer's Blueprint>, which you can<config;b;arcfurnace_electrodeCrafting; craft,> find in certain chests or trade with a villager. You can also create electrodes in the metal press, using the rod mold on 4 ingots, but the electrodes crafted this way have less durability.

20.0.37 Industrial Hemp

530 fabric it

Industrial Hemp is a remarkable plant! Not only are its seeds useful for the the creation of <link;refinery;Biodiesel>, the hemp fibers are also useful for the creation of Tough Fabric. The seeds can be gathered by breaking tall grass.

Tough Fabric is a resilient weave made from hemp fiber. It's used to create <link;generator;Improved Windmills;3> as well as <link;balloon;Balloons> and Jump Cushions (detailed on the next page).

Jump Cushions are simple blocks created from tough fabric and inflated with air. They are capable of absorbing impacts and prevent falldamage when landed upon, but, unlike slime blocks, will not bounce you back up.

Strip Curtains are pieces of fabric that are mounted to doorframes or other openings to serve in blocking the view.

When an entity passes through them, they will emit a redstone signal. Rightclick them with the hammer to enable strong signals.

They can be dyed in the same way leather armor can.

20.0.38 High-Voltage

You can't shake the shock

High-Voltage wires have the highest available transfer rate (<config;iA;wireTransferRate;2> Flux/t) and will need special connectors. Note that on High voltage connections, Connectors and Relays are different blocks, Relays will not be able to in- or output to the energy net. Furthermore, Relays can only be placed on the underside of the block.

To transition from HV to LV or MV connections, you will need High-Voltage Transformers, and keep in mind, the lowest tier of wire limits the transferrate of an entire connection.

20.0.39 Improved Blast Furnace

Industrialized Refinement

The Improved Blast Furnace is a much more professional way of <link;blastfurnace;creating steel;>. Not only does it allow the automation of in- and outputs, it can also be outfitted with preheaters to speed up the refinement process by addition of hot air.

The furnace is made from reinforced bricks, so you'd be advised to spend your first batches of steel on this upgrade.

The structure of the Furnace is similar to its crude predecessor, but the bricks have been reinforced and a hopper was added at the top to funnel in the inputs.

Steel is output at the front of the furnace, slag at the back, and the furnace will automatically output to connected inventories or conveyor belts. Iron and Coal Coke are fed in through the top.

Blast Furnace Preheaters are used to heat up air and then blow it into the furnace to speed up the refinement process of iron. Each furnace can have up to two preheaters connected to the ports on the each side, and each preheater requires <config;i;preheater_consumption> Flux/t to operate.

20.0.40 Introduction

Getting into Engineering

Greetings, fellow engineer, and welcome to Immersive Engineering. This journal shall serve as your guide for setting up all the new machines and functions this mod has to offer.

Let's start with the basics first: Immersive Engineering is built around the concept of retrofuturistic generation and transfer of energy. The energy used in this mod is Redstone Flux, a trait shared with many other mods currently available.

Energy is transmitted between connectors by the use of wires. Wires can only bridge a limited distance of blocks, so you will need to use more connectors as relays between the wires.

You have the choice between three different types of wire, which differ in transferrates, <config;iA;wireTransferRate;l3> Flux/tick. To switch between different types of wire, you will need to use transformers.

20.0.41 Jerrycan

Better than buckets

Jerrycans are simple metal containers used to carry fluids. They can store up to 10 buckets and can pour that fluid into the world as well as into machines. They can not pick up fluids. Additionally, they can be used to refill other fluid-consuming items like the <link;drill;Mining Drill> by crafting them together.

20.0.42 Engineered Lighting

Lighting up the Wasteland

Deep in the halls of the factories, it can be quite dark. In order to still find your way around the machines, you should probably light up the place with some Lanterns. These Lanterns provide almost as much light as a glowstone block would.

Powered Lanterns are one of the first ever invented uses for electric power. To run these, you need to connect them directly to your power grid using wires. They will allow power to be transferred through them, so you can link them in a row.

Using the Engineer's Hammer on them will flip them upside down.

As long as they are supplied with a little bit of power, they will light up the surrounding area like a normal Lantern, but additionally, they will also prevent hostile mobs to spawn in a 32 block radius!

Floodlights are strong lightsources. While they require more power than a simple Powered Lantern, they will create a cone of light up to 32 blocks long. Use an Engineer's Hammer on the top or bottom faces of the block to rotate the light or click its sides to change its pitch.

Generally, Floodlights require a redstone signal to run, however their control-behavior can be inverted by sneaking and right-clicking their base with an Engineer's Hammer.

20.0.43 Lightning Rod

Thunderstruck!

Lightning is a phenomenon of nature. It contains a huge amount of raw power, and this lightning rod is able to harness that power.

By stacking a lot of steel fences on this multiblock, you increase the chance of a lightning bolt striking it during rain or thunderstorms.

When the steel pole is struck, the huge amounts of energy created are stored in the base of the lightning rod and then slowly distributed to the energy connections at the sides.

You can improve the chances of the Lightning Rod getting struck by creating a "net" of steel fences connected to the topmost fence.

20.0.44 Maintenance Kit

On the go repairs

Among those Engineers that travel the world or spend much of their time in mines, one problem crystalizes itself. Their tools require constant maintenance, and carrying around an entire workbench just isn't viable. Fortunately, the majority of minor repairs and re-configurations can be performed with a maintenance kit.

A set of tools, wrapped in sturdy hemp cloth, the maintenance kit allows you to customize your tools much in the way the <link;workbench;Engineer's Workbench> does. No longer will you have to deal with resurfacing to change the head on your Mining Drill or head back to your workshop to configure your Ear Defenders. All of this can now be performed on the go with the Maintenance Kit!

20.0.45 3D Maneuver Gear

Attack on upscaled zombies

The 3D Maneuver Gear is a marvel of engineering. It consists of a harness of leather straps, two grappling hooks and a pressurized air propulsion system.

The gear is equipped in the <config;b;baubles;Baubles belt slot or the ;>leggings slot and powered with Flux charge.

Controls:

Holding the <keybind;key.immersiveengineering.3dgear> Key will switch you to the Maneuver Gear controls. In this controlmode, the mousebuttons will fire the hooks at the cost of some Flux charge. While the hooks are embedded in an object, holding the button will pull you to that object. A doubleclick will dislodge the hook and return it to you.

Double-tapping the <keybind;key.immersiveengineering.3dgear> Key will return both hooks.

Propulsion:

Pressing the jump key while the <keybind;key.immersiveengineering.3dgear> Key is held will expend some of the pressurized air to give you a quick burst of speed in the direction you are looking. After not using the propulsion system for few seconds, the internal compressor will quickly refill the tank at the cost of Flux charge.

Should you land from a height prone to damage you, the system will automatically expend pressurized air to cushion your fall.

20.0.46 Metal Press

Under Pressure!

The Metal Press is an automated way of shaping metal. It uses a high pressure piston to press a mold onto an ingot, forming it into a plate, rod or other metal components.

The structure is created as shown above, to form the multiblock, click the piston with an Engineer's Hammer.

Power is input through the top, and items through the conveyors, obviously in the direction of movement. The lower middle block is the redstone control port, rightclicking it with an Engineer's Hammer allows you to invert the redstone control.

To actually work, the metal press requires a mold to be attached to the piston. These molds are created in the <link;workbench;Engineer's Workbench> using the blueprint displayed above.

Molds are crafted as shown above, with the wirecutter taking some damage for every crafting operation. They are then inserted into the metal press by right-clicking the multiblock with a mold in hand.

The packing and unpacking molds can be used to compress 4 or 9 of the same item into their relevant storage form, as well as unpack them again. This can be utilized to store ingots as blocks, or automatically create simple blocks like Sandstone.

20.0.47 Metal Barrel

Doesn't roll

Metal barrels work exactly like <link;barrel;Wooden Barrels;>. They store the same amount of fluid and their in- and outputs are configured the same way, but unlike the wooden version, these can store gasses and hot fluids, and won't output automatically if receiving a redstone signal

20.0.48 Metal Constructions

Riveting

Apart from Treated Wood, processed metals make for common construction blocks. You can craft almost any metal into blocks and slabs, or hammer iron ingots into a sheetmetal block.

Steel is not only an integral part of high-voltage architectures, it is also quite useful in, well, architecture. Steel scaffolding can be used to quickly create climbable structures, steel fencing keeps baddies at bay and structural arms make for great attachment points for your HV relays.

Structural Connectors work similarly to normal connectors, but instead of transferring energy, they are purely decorative.

They won't accept normal wires, but Hemp Rope and Steel Cable (see next page).

You can also rotate them by rightclicking them with an Engineer's Hammer.

Metal ladders are...well, ladders made from metal. You can climb them, and they look cool.

You can also combine them with Scaffolding to make them into Covered Ladders.

20.0.49 Mineral Deposits

Oresome

The deposits of Ore found around your world are nothing new to you. What you have so far left out of consideration though are mineral veins, which contain a mix of different ores. Due to their wide and thin spread, the average miner can't find them, but a <link;excavator;special machine> might be able to assist with that.

To find and determine minerals in the world, you will want to craft a Core Sample Drill. Supply the machine with power and rightclick it or use redstone to activate it. It drills into the chunk and extracts a core sample which shows if and what mineral was found and its yield.

The Drill consumes <config;i;coredrill_consumption> Flux/t and takes <config;i;coredrill_time> ticks to fully retrieve and analyse a sample.

The sample can also be placed as a block by sneaking and rightclicking. Using a map on the sample block will set a marker for the chunk.

The following pages list every type of vein and the ores that can be retrieved from them.

%1$s

It consists of %2$s.

20.0.50 Mixer

Vats of Cookie Dough!

The Mixer is a multiblock structure that dissolves solids in fluids, mixing them together. The structure is built as shown above and formed by using the Engineer's Hammer on the central sheetmetal block.

The most common uses for this machine are the mixing of fluid concrete from water, 2 parts sand 1 part gravel and 1 part clay, or the creation of potions from water and their various ingredients.

Fluids are input into the mixer through the port at the bottom of its powersupply. Items can be input via the two hatches at the back. The fluid port at the front serves as the fluid output.

Applying a redstone signal to the control panel at the front will halt the machine's process, this behavior can be inverted by use of the Engineer's Hammer.

20.0.51 Multiblock Constructions

Where's my Allen Wrench?

Fitting complex machinery into a single 1x1x1 cube seems to be a task unmanageable without the use of magic or possibly shrink rays. So in order to create more complex machines, you will need component blocks. The most common ones have been listed on the following pages.

20.0.52 Ore Processing

Crush crush crush

Crafting ores with an Engineer's Hammer allows you to crush them into dusts. This dust can be smelted into ingots, but more importantly, is used to <link;alloys;create metallic alloys>.

20.0.53 Oilfield Engineering

What the ground is keeping from you

Some ground has oil under it and most of it does not. A deposit is not something you can dig up or even see - there is no ore, no black patch, nothing to find by mining. It is decided by the world seed, it covers a wide square of chunks so a whole area agrees about what lies beneath it, and it holds a fixed amount of crude that never comes back.

So you survey before you build. Put down a <link;minerals;core sample drill;1> and read the sample: it names the deposit, gives its size as a band rather than a number, and reports how much pressure is left and whether it will still flow unaided. A sample that says No Reservoir means exactly that, and no amount of building on top of it will change the answer.

Oilfield Frame is the structural part every rig on the field is built out of, and the stack a finished rig packs itself back into. Keep plenty of it.

None of the oilfield machinery is crafted as a block and placed. It is built in position out of frame, steel and sheetmetal, and then struck with an Engineer's Hammer.

The Drilling Derrick sinks the bore: a three by three lattice nine blocks tall, hammered together like any other structure.

Power goes into the drill floor - the whole bottom course takes a connector on any face - and it wants around 256 Flux a tick for the minute the bore takes.

If there is nothing under the rig it says so in chat the moment it forms, and keeps saying so on the overlay for as long as it stands. A dry hole is not a wiring fault, and a rig over one will accept power forever without ever drilling.

When the bore is finished the derrick hands back its frames and leaves a Wellhead standing over the hole. The rig is equipment rather than architecture - pick it up and go and drill the next site with it.

The Wellhead is the whole of the well from then on. It buffers a couple of seconds of production and pushes it into anything beside it that accepts fluid. Rightclick it for how much of the field is left and how it is flowing, and put a comparator against it to read how badly it is backing up.

A field is not a constant. Pressure - what is left of it - decides how the oil reaches the surface.

Above about three fifths remaining the deposit lifts itself, and a bare Wellhead runs at full rate with nothing else attached at all. Below that line the pressure can no longer carry oil up the bore and the well delivers nothing whatsoever until you put a pump on it. Pumped, the rate falls off steadily as the field empties, until at the bottom it settles to a slow seep that never quite stops. An old well is slow. It is never dead.

The Pumpjack is what a tired well needs. Build it so its well bay lands over the Wellhead - the bay is drawn here to show you where it goes, is not a part you supply, and forming the machine will never swallow the well it exists to work. A well sunk a block into a cellar is found as well.

Power connects across the motor blocks at the back, and it draws about 512 Flux a tick while it is nodding. The pump never touches the oil itself; it leans on the well, and the well does the producing.

A pumpjack over a field that still flows on its own does nothing and costs nothing. There is no extra rate to be had, so it idles rather than burning power for the look of the thing, and it starts nodding by itself once the field falls past the free-flow line.

It also has to keep asking. The pump re-asserts itself once a second, so one that is broken, unpowered or torn down leaves the well quiet again straight away, with nothing to reset by hand.

Oil does not come up alone. Sour gas comes up dissolved in it - a quarter of a bucket for every bucket of crude - into a second buffer on the Wellhead, and it fills whether you have any use for it or not.

So it needs somewhere to go. A full gas buffer stops the well, oil and all, exactly as a full oil buffer does: the gas is coming up regardless and the well cannot produce past it. This is the commonest reason a rig that worked yesterday is doing nothing today.

The Wellhead pushes both buffers into any face that will take them, and an ordinary tank or pipe accepts whichever arrives first - so plumb the gas somewhere deliberately rather than hoping it sorts itself out.

The cheap answer is a Flare Stack stood against the Wellhead. It takes gases and nothing else - it will not accept crude, so it is no way to make a spill disappear - destroys what it is given, and hands back nothing at all. It burns while it is being fed and lights the ground around it, and that light is the entire return.

That is deliberate. A flare is a loss you can stand and watch, and the point of watching it is to make you want the next machine.

The Gas Scrubber turns that loss into two products: twin vessels six blocks tall on a three by three skid, with an alley between them to stand and plumb in.

Height is the interface here, exactly as it is on the tower. Sour gas enters anywhere along the skid at the bottom, sweet natural gas leaves along the head course at the top, and sulfur drops out of the front of the skid. The four courses between them connect to nothing at all, so a pipe run climbing past the vessels can never pick up the wrong stream.

The scrubber does not run on Flux. It runs on heat, drawn from a lit Industrial Burner standing against its skid, and without one it does nothing - the overlay tells you which of the three it is: idle, no heat, or backed up.

Four fifths of the sour gas survives as natural gas and the missing fifth is what the sulfur is made from. Underfeed it and it scrubs slowly rather than stalling on some threshold nothing warns you about. A burner will run on the scrubber's own gas for about a quarter of what it makes, but feed it heavy fuel oil instead - the cut nothing else wants - and you keep the whole of it.

The Distillation Tower is where crude becomes worth something: a two by two steel column twelve blocks tall standing on a scaffolding deck, with frame nozzles out of the shell in pairs at seven heights.

Crude goes in at the deck and nowhere else. Power connects at any of the four deck corners - the column looks the same from every side, so there is no back to hunt for - and the redstone panel is down there with it. The bare shell between the nozzles neither takes nor gives anything.

The nozzle heights are the machine. Each pair of nozzles draws exactly one cut, in column order, lightest at the top and heaviest at the foot: natural gas, naphtha, gasoline, diesel, heavy fuel oil, lubricant, and bitumen at the bottom. Point a pipe at the wrong height and you get the wrong fluid, not a mixture; look at a course, or hammer it, to be told what that height carries.

A batch will not start unless every cut it would make has somewhere to go. A tower with one full output tank stalls with its crude intact rather than throwing a fraction away.

Distilling is mostly heat. The Industrial Burner is a three by three firebox of blast brick under a steel sheetmetal crown, with a frame burner head let into the front where the fuel line lands.

It makes no Flux at all, and that is the point of it. Stand one against the side of a Distillation Tower - beside it, never under it - and it fires the column, and while there is fire on it the tower runs for a fraction of the energy it otherwise costs.

The burner ranks fuels by what a millibucket of them is worth as heat, and heavy fuel oil beats the lot. Diesel, propane and natural gas all burn in it too, and every one of them is worth more somewhere else - which is exactly why the firebox exists.

It will also heat whatever is standing on its crown, a row of ordinary furnaces included, so it has a job from the day you build it. A burner nobody is drawing heat from stops burning fuel on its own; it stokes

to demand and needs no switch.

What comes off the column, and what each cut is for.

Diesel is the best thing a Diesel Generator can burn, and it runs a drill as well. Gasoline runs handheld tools but will not run a Diesel Generator at all - a compression engine cannot burn it, which is an engine-type split and not a fault. Naphtha burns in a generator. Heavy fuel oil burns only in the Industrial Burner. Lubricant does not burn at all, and bitumen is for paving.

Raw crude will run a generator, badly. Refining it first is always the better trade.

The Gas Turbine burns natural gas or propane for Flux: three blocks high and six long, filter house to nacelle to alternator to exhaust stack.

Gas goes in anywhere along the bottom course. The power leaves through the three cells standing open above the alternator, so put your connectors up there. A redstone signal stops it, and so does having nothing to deliver into - it will not sit there burning gas against an open breaker.

It does not switch on, it spools up: ten seconds from ignition to full output, drawing well over half its full-load gas from the first tick while producing nothing. Every cold start throws away about three seconds of fuel before it breaks even, so ten starts cost what half a minute of running does.

Left running it is the best thing you can do with a millibucket of gas - three Diesel Generators' worth of power on less gas than three of them would drink. Cycled against a wobbling demand it will eat a tank proving the point. It coasts down over twenty seconds and restarts cheaply from a warm rotor, so a brief interruption is not a cold start.

Lubricant is the one cut that does not burn, and the Lubrication Manifold is where it goes. Bolt it against any of the mod's larger machines, fill it, and that machine costs noticeably less to run.

It only draws while its host actually has something on the go, so an idle machine will not quietly empty it. A bucket is about eight minutes of continuous work - an occasional errand rather than a second supply chain.

Bitumen is the bottom of the barrel and the reason paving exists. Put 250 mB of it through a <link;mixer;mixer;0> with a shovelful of sand and gravel and you get 500 mB of wet asphalt.

Then pour it. It flows, finds its own level and sets a moment later into a full block of road - no forms, no depth to judge, because a road is flat by definition. Laid asphalt is a little slicker underfoot than the ground beside it, which is the whole point of building one. Cut it into tiles, or mark it out with yellow dye.

A Diesel Generator is a good machine and it never changes. The end of an oil chain deserves buildings instead - plants you walk into, fed by pipe, measured in tens of thousands of Flux.

They are built around one axis: response against efficiency. Nothing here is simply better than everything else. The engine bank answers instantly and wastes fuel; the turbine hall is brutally efficient and hates being cycled; the gas turbine sits between them.

The Fuel Oil Boiler is the furnace half of a power station, and it makes no Flux at all - it makes steam.

That is what finally gives heavy fuel oil a job at scale. Crude and diesel burn in it too, and both are worse trades: diesel is worth more in an engine and crude is worth more refined. Water goes in along the base course, fuel goes in with it, and steam leaves from the crown.

Steam is a real fluid here, not a number hidden inside one machine. It has a bucket, it has a pipe, and it moves between buildings.

That is the entire reason the boiler and the turbine hall are separate structures rather than one enormous one: you build a boiler house, you build a hall, and you plumb them together across your yard. It is also why the Heat Recovery Steam Generator can exist at all.

The Steam Turbine Hall is the largest thing in this chapter and the most efficient. Steam in at the condenser end, Flux out at the switchyard.

It is slow to start and it hates being cycled. Feed it steadily and nothing in the mod converts fuel to power better; switch it on and off against a wobbling demand and it will spend most of its life spinning up.

The Heat Recovery Steam Generator burns nothing whatsoever. Butt it against a Gas Turbine's exhaust end and it makes steam out of heat that was going up the stack regardless.

One HRSG claims one turbine's exhaust, so two of them on the same machine is one of them wasted. The face it needs is three wide and three tall, which is exactly the turbine's exhaust face - if it will not sit flush, it is not in line.

Turbine, HRSG, hall. That is the combined cycle, and it is the point of this whole section: the gas turbine's waste heat becomes steam, the steam becomes more Flux, and the same millibucket of gas does considerably more work than it does alone.

It is also the least forgiving thing to build. Three structures in a line, one fuel line, one steam line, and any of the three idle makes the other two worse.

The Reciprocating Engine Bank is the honest workhorse: diesel, biodiesel or gas, instant response, mediocre efficiency.

It is the only plant here that scales by building another one. Put a second bank alongside the first and they run as a set. Nothing about one bank changes - you simply have two - which makes it the plant to build when you are not sure how much power you will need.

A fuel supply should not have to be an eyesore. The Tank Fill Cap is the only part of a buried tank that shows above ground: a pipe connects to it, a comparator reads it, and its gauge is the whole of the tank's readout.

Dig the hole, line it, put the cap in the middle of the roof, backfill, and hammer it. A tank that is not covered will not form - and it will tell you so rather than simply refusing.

The Domestic Tank is two by two by two of iron sheetmetal under the garden, holding thirty-two buckets.

It is the cheap one on purpose: the first thing to build once you have a fuel worth keeping, rather than the reward for finishing a refinery. A couple of days of heating oil, or enough diesel to keep one generator honest overnight.

The Commercial Tank is a five by five hole four deep, lined in steel, holding exactly what a Sheetmetal Tank holds.

Matching that is the point. This is not a bigger store - it is the same store out of sight, and what it costs is a considerably larger hole.

The Bulk Depot is nine by nine and six deep, and holds four million millibuckets: a refinery's own stock, or a town's.

Nearly eight times the commercial tank for three and a half times the blocks, which is the reason to dig the bigger hole. Its middle is hollow - you are lining an excavation, not filling one.

The Gas Station Pump hands fuel to a person. Stack two of them and strike one with an Engineer's Hammer: they become one bowser two blocks tall, with the price on its own panel. Breaking either block takes it apart.

Right-click it holding anything that takes fluid - a bucket, a jerrycan, a Mining Drill - and it fills it.

Right-click it empty-handed for the panel: what is in the tank, the price per bucket, and how much this pump has ever sold. This fork has no currency; the price is a number the pump reports to whatever your server does about money.

Take the Fuel Nozzle off a pump and it remembers which pump. Then right-click anything within eight blocks that accepts fluid and the pump fills it - a generator across the yard, a tank, a machine. Sneak-right-click the air to rack it again.

The Forecourt Price Sign has no price of its own. It finds the nearest pump and shows that one's, which is the only arrangement that cannot go stale.

Gasoline runs handheld tools and a Diesel Generator will not touch it, which is correct - a compression engine cannot burn it. The Portable Generator is what it is for.

One block, 256 Flux a tick, four buckets of gasoline or ethanol, about five minutes of running. Carry it to the job, run your lights, walk it back to the pump. It refuses diesel, exactly as the Diesel Generator refuses gasoline.

Filling one jerrycan by hand is fine. Filling eleven is not.

The Fluid Loading Gantry takes empties out of the inventory on one side, fills them, and puts them into the inventory on the other. It has no interface at all - where you put the chests is the configuration.

The Cracking Unit breaks the heavy cuts into the light ones. Heavy fuel oil or naphtha goes in at the deck; gasoline leaves along the front of the head and diesel along the back, and the row between them connects to nothing at all.

The dial is the firebox. The cracker always asks for the heat of a maximum-severity run and cracks at whatever severity the heat it actually gets pays for. A burner on heavy fuel oil runs it hot and yields gasoline; one on natural gas runs it cool and yields diesel. Hammer the deck to read what the last pass ran at.

Heavy fuel oil also cokes out. Petroleum coke burns half again as long as coal, which is what stops the byproduct being a nuisance.

A field that still has its own pressure will blow out if you drill it without a Blowout Preventer: a fountain of crude, a spill around the wellhead, and a tenth of the field gone up the bore. Right-click the running rig holding one to fit it; the rig warns you on its overlay while there is still time.

An Absorbent Pad lifts spilt fuel three blocks at a time. Spilt crude is flammable and it flows, so do not leave it.

The Re-injection Well puts water or gas back downhole and gets a second tranche out of a tiring field. It is the thinking player's answer to a flare stack.

Water is cheap and poor; scrubbed gas is valuable and good; sour gas straight off the wellhead sits between them, which means a field can drive its own recovery on the waste it was already producing.

Each field has a lifetime allowance - about a fifth of what it originally held - and it does not come back.

A Core Sample Drill answers "is there oil here", one cell at a time, and a cell is a hundred and twenty-eight blocks across. The Seismic Survey Kit answers "which way do I walk".

Right-click the ground holding one - it needs a charge, so carry gunpowder - and it reads the nine cells around you and reports each deposit by bearing and rough range. It never gives you a coordinate. That is still the sample drill's job.

20.0.54 Metal Plates

Flat as a flounder

Metal Plates are a crafting material required for the creation of some machines or blocks. They are made by hammering an ingot with an Engineer's Hammer or in a <link;metalPress;Metal Press>.

20.0.55 Capacitor Backpack

Do....Ray....Egon!

Energy storage takes space and some heavy weight components. As such, storing large amounts of energy in each tool individually is inefficient and makes them akward to wield.

The Capacitor Backpack acts as a compromise, offering a centralized storage of energy for any powered tools to tap into.

The backpack charges any tools held in main- or offhand as well as any armorpieces you are wearing.

The capacitor can be worn on your back like a piece of chest armor, or you may attach it to most common sets of armor in order to gain the benefit of a power-supply as well as protection.

20.0.56 Railgun

Velocitas Eradico

The Railgun (or more accurately the Portable Induction Catapult) is a marvel of engineering. It consists of two rails through which an electric current flows and accelerates a sled, and by that, a projectile.

The process is energy intensive but will accelerate various projectiles to high speeds.

To launch a projectile from your inventory, click and hold the use button till the charge level reaches 99, then release.

Valid projectiles include, for instance, iron and steel rods<config;b;literalRailGun;, rebar and almost every rail from Railcraft;>, as well as graphite electrodes.

Due to the low internal energy storage, it is recommended to use the Railgun along with a <link;powerpack;Capacitor Backpack>.

As with other IE tools, the railgun can be modified in the <link;workbench;Engineer's Workbench> and be customized with <link;shader;Shaders>.

The Additional Capacitors greatly reduce the railgun's charge time.

The Precision Scope allows you to zoom in with the railgun. Sneak to use the scope and use the mouse wheel to switch between magnification levels.

20.0.57 Razorwire

Oversized eggslicer

Razorwire is a wire with sharpened pieces of metal attached. It will slow and cut any entity that tries to move through it.

Razorwire can only be removed with Wirecutters, any other attempts will damage the player.

Additionally, a LV wire can be connected to electrify the razorwire for extra damage.

20.0.58 Redstone Wires

Integration Takeover

Redstone Wires are aluminium coated in redstone dust. The resulting cables provide an easy way to transfer redstone signals over long distance without losing any signal strength.

The connectors work like any other wire connector in terms of connecting them, however, redstone wiring does not require relays; connectors and relays are the same block.

Using the Engineer's Hammer, the connectors can be switched between in- and output mode (rightclick) and configured between 16 coloured frequencies (sneak-rightclick).

Redstone Probe Connectors have a functionality partially comparable to Redstone Comparators. While they are not able to actively compare signals, they are able to read inventories for how full they are.

Additionally, these probes can operate on two channels, sending their read signal on one, and still receiving signals on another. These channels can be changed by using the Engineer's Hammer.

Note that many of IE's multiblocks feature comparator-functionality, as described on the following pages. Where available, the redstone control surface of the multiblock is the designated spot to connect a comparator.

Tank: As detailed in its <link;tank;manual entry;1>

Silo: As detailed in its <link;silo;manual entry;2>

Assembler: Outputs a signal relative to the fill of the inventory.

AutoWorkbench: Outputs a signal relative to the fill of the inventory.

Squeezer: Outputs a signal relative to the fill of the inventory.

Fermenter: Outputs a signal relative to the fill of the inventory.

Excavator: Outputs a signal relative to the remaining ore in the vein.

ArcFurnace: Outputs a signal relative to the fill of the inventory.

Additionally, connecting a comparator to the electrodes at the top will send a signal based on their integrity.

Mixer: Outputs a signal relative to the fill of the inventory.

20.0.59 Refinery

Fluid mixing!

The Refinery is a complex multiblock that can refine one or two fluids into another. The most common application of this is the refinement of <link;squeezer;Plant Oil> and <link;fermenter;Ethanol> into Biodiesel.

To form the structure, rightclick the Heavy Engineering Block on the middle layer.

Fluids are input through the two ports on either side of the block and output through the orange marked port at the front.

Applying a redstone signal to the control panel at the front will halt the machine's process, this behavior can be inverted by use of the Engineer's Hammer.

20.0.60 Revolver

Wanted: Dead or Alive

The Revolver, a more elegant weapon from a more civilized age, is an achievement of small mechanics. Small being relative here, since this weapon is chambered for 12-gauge.

Similar to the Mining Drill, the Revolver will allow for modification in the <link;workbench;Engineer's Workbench>, as detailed on the last pages.

There are different <link;bullets;types of ammunition;> the Revolver will accept. To load it, sneak+rightclick the weapon, then place cartridges into the slots.

A loaded Speedloader in your inventory will be used to refill the revolver after you've fired your last shot.

Note that firing the revolver is quite noisy, and will attract the attention of hostile mobs in the area!

By attaching a bayonet to the revolver, you outfit it with close-quarter-combat capabilities, allowing for melee damage as powerful as an Iron Sword.

The Extended Magazine allows loading 6 more cartridges into the revolver. This magazine does not get refilled by a speedloader.

The Amplifying Electron Tubes apply an electric shock to fired projectiles. This shock persists till it hits the target (don't ask how it works) and is passed on. The target will be stunned for a fraction of a second and all powered equipment they may be using will suffer a slight discharge.

Rarely, when exploring villages and trading with the local engineers, you may find special crafting components for the revolver. These components come with certain intrinsic perks and when used to craft a revolver, they will imbue those perks into it, affecting the firerate, the amount of noise it makes and augmenting the wielder's luck.

20.0.61 Shaders

Blue! No, yellow!

Shaders are items you can find in the world or aquire by trading with villagers. They don't have a purpose by themselves, but they can be equipped to <link;revolver;Revolvers>, <link;drill;Drills>, <link;chemthrower;Chemical Throwers>, <link;shield;Plated Shield> or <link;railgun;Railguns>(and possibly other things you still have to discover) to change the designs of the item. Simply put the item in an <link;workbench;Engineer's Workbench> and equip the shader in the appropriate slot.

You can also apply them to a Minecart, Banner or a <link;balloon;Balloon> by rightclicking!

For a full list of all available shaders, their looks, references and more details, <link;shaderList;check here>

Shader Grabbags are items containing a random shader. They can be found in crates, traded from villagers or dropped by bosses. The rarity of the contained shader is equal to, or better than the rarity of the bag. When using a bag, you are more likely to get a shader you do not yet own, than one you already posess.

Shaders bags can be crafted into two bags of a lower level. This will lower your chances of getting rarer shaders but provide you with more attempts.

20.0.62 Heavy Plated Shield

One Man Phalanx

The Heavy Plated Shield is a direct upgrade to the normal wooden shields you might be acquainted with. In similar fashion, this shield will block most incoming damage from the front, but its durability has been increased considerably, and it sports the ability to be upgraded.

The Flashbulb upgrade adds a bright lightbulb and reflector screen to the shield. Upon being attacked, the device will produce a bright, blinding flash, disorienting everyone in front of the user.

Adding Shock Emitters to the shield allows you to retaliate against attacking enemies. When the shield is struck, it will emit a shock, inflicting minor damage to the aggressor. Additionally, it will vapourize projectiles rather than just stopping them.

The Magnetic Glove serves as an easy way to access the shield when threatened. With this upgrade equipped, simply double-tapping the <keybind;key.immersiveengineering.magnetEquip> Key will have you retrieve the shield from your inventory and place it in your offhand slot.

20.0.63 Pole Signage

Somebody has to read it

A hundred poles across a map are a hundred identical poles.

The Utility Pole Sign is the tag that tells them apart: which station feeds this one, which feeder it is on, who last climbed it. Thirteen kinds, and every one of them is a sign that hangs on a real pole – the shapes and colours are the Los Angeles Department of Water and Power's and Southern California Edison's, and they are told apart the way the real ones are, by shape and colour long before you are close enough to read the number.

Hanging one. Place it against the side of whatever holds the wires up – a pole, a crossarm, a wall. It bolts flat to that face and reads from the outside; a tag has no top or bottom face to sit on, and if the pole comes down the tag comes down with it.

Choosing the kind. Hit it with an Engineer's Hammer and it steps to the next of the thirteen. Keep hitting to walk the whole set; it comes back round.

Writing on it. Sneak and hit it with the hammer and the plate opens: pick the kind with the arrows, type the lines, and watch the preview – what is in that window is what ends up on the pole.

Text is printed to fill the plate rather than typeset at a fixed size, which is what the real ones do: a short number comes out big and a long one comes out small. The vertical strips read downwards, because a strip six pixels wide holds a pole number no other way.

What the kinds mean. A redo horizontal strip says parallel generation feeds the station this pole hangs off. Yellow is general identification – the vertical strip is the pole number, the horizontal one the station, feeder and connection count. White is street lighting and silver is the older pattern of the same. Orange marks fibre and cable runs. The white oval is a series-wired light: series number over pole number. The round tag is who inspected the pole and when.

The diamonds belong to transmission towers: a plain one carries the tower number, and one with a cross on it marks a line crossing – another line, a valley, a ravine.

20.0.64 Silo

Full to the brim!

Silos offer space for a lot of items of the same type, making them excellent for mass storage of seeds, slag or cobblestone. They have space for 41472 items, equaling the capacity of 24 chests.

Items must be input through the top and extracted from the bottom. Applying a redstone signal to the bottom block will make it output automatically.

A comparator attached to the bottom center block will it emit a redstone signal proportional to how full the silo is. Attaching the comparator to one of the top 6 layers results in a signal proportional to the height of the comparator. For example, a comparator attached to the third layer of a silo would not emit any signal until the tank is 2/6 full and have a signal strength of 15 if the silo is half full or more.

20.0.65 Engineer's Skyhook

Booker, catch!

The Engineer's Skyhook is a motorized grabber system designed to hook onto wires, hemp ropes or steel cables.

With it, engineers can quickly traverse the length of a powerline to move between substations or other installations.

Simply hold the right mouse button while close to a wire and you will be attached to it.

The button can be released while riding the wire, but letting go of the skyhook or sneaking will cause you to dismount.

Gravity will be sufficient to move along decending wires at a decent speed, the standard movement keys can be used to move along ascending wires.

When the skyhook reaches an intersection it will continue in as close to the direction the player is looking as possible.

20.0.66 Item Router

Left foot yellow, right hand green

The Item Router is a block which will accept and redistribute items down filtered paths. Place it down and rightclick it to open the interface, which shows six different, colour and direction coordinated filters.

When an item enters the router, it will be routed to an output whose filter matches the item, or, if no matching filter is found, it routes to an unfiltered output. If the router cannot route the item in the first place, it will not allow the item to enter the router. Items also can not be output to the side it was inserted from.

The buttons above each filter can be used to have the router respect or ignore OreDictionary, NBT and Metadata.

20.0.67 Squeezer

Pressing matters

The Squeezer is a multiblock structure that presses juice or oil out of organic materials by use of a big piston. The structure is built as shown above and formed by using the Engineer's Hammer on the central wooden barrel, from the side that has the Engineering Blocks.

Items can be input into the Squeezer via the two hatches at the back that are marked with blue dots. The hatch with the orange dot serves as an item output and the block below it to output fluids.

Applying a redstone signal to the control panel at the front will halt the machine's process, this behavior can be inverted by use of the Engineer's Hammer.

20.0.68 Tank

Full to the brim!

The Tank is a multiblock that provides space for large amounts of fluid. It will store up to 512 buckets of any fluid, which can be piped in through the top or bottom block. Fluids can only be extracted from the bottom center block.

Applying a redstone signal to the bottom center block will make the tank output automatically, a comparator placed there will emit a redstone signal proportional to how full the tank is. Attaching the comparator to one of the top 4 layers results in a signal proportional to the height of the comparator. For example, a comparator attached to the second layer of a tank would not emit any signal until the tank is 1/4 full and have a signal strength of 15 if the tank is half full or more.

20.0.69 Tesla Coil

This one goes up to 11

The Tesla Coil started as a concept of wireless energy transfer. However, due to its potentially high loss rates and it's rather interesting property of frying every living being in a 6 meter radius, it was quickly converted into an Engineer's dream of home defense.

The upkeep cost of this machine is high, clocking in at a regular <config;i;teslacoil_consumption> Flux in passive consumption plus an additional <config;i;teslacoil_consumption_active> every time it applies damage.

The coil is disabled unless supplied with a redstone signal. Rightclicking it with an Engineer's Hammer will invert this behaviour.

In addition to damage, the shocks will also apply a lingering debuff, "Stunned", preventing the entity from teleporting.

Rightclicking the coil with an Engineer's Hammer while sneaking will toggle the coil between high- and low-power mode. To be able to do so safely the coil has to be turned off. It is in high-power mode by default. When the coil is in low-power mode it will use half the usual energy and have

half the range. If you are wearing a full Faraday suit, a coil in low-power mode won't harm you, but still shoot lightning at you. Coils in high-power mode will cause the suit to melt, thereby increasing the damage done to you and additionally damaging the suit.

Fluorescent tubes glow when placed inside a strong high-frequency electric field. Such a field is generated by any tesla coil, so placing a tube near an active tesla coil will cause it to glow. The range is about 1.5 times the range in

which you will be hit by lightning. They can be colored in the engineer's workbench and rotated by (shift) rightclicking them with an Engineer's Hammer

20.0.70 Thermoelectric Generator

Hot and Cold

Thermoelectric Generators are another option of power generation and work without the use of any mechanical parts, instead they use the temperature gradients between two sources.

Placing a hot fluid on one side and a colder fluid on the opposing side generates energy based on the difference between the two temperatures.

The list on the following page contains other, non-fluid blocks, which can be used as sources of temperature.

20.0.71 Engineer's Toolbox

The Engineer's Handbag

The toolbox is a simple, carried container, used to store all the essentials of an engineer! Hovering over the relevant slots tells you what they will accept.

'Food' and 'Anything' are quite self-explanatory, 'Wiring' slots are for coils, connectors and similar items, and 'Tools' will store any IE tool as well as picks, shovels and axes!

Sneaking and right-clicking allows the toolbox to be placed as a block. While placed, it's inventory can still be accessed with a rightclick. Sneaking and right-clicking again will pick the toolbox up.

20.0.72 Treated Wood

Highly Durable

Treated Wood is, quite simply, wooden planks covered in creosote oil. It is however, one of the most important materials you will be using in Immersive Engineering, so you'd better be prepared to create a lot of it.

20.0.73 treatedwoodPost

Posts made from treated wood make for excellent attachment points for your Wire Connectors. They are 4 blocks high, so you won't be able to place them without the proper clearance.

Using the hammer on the sides of the topmost block will attach an arm to the post.

You can also place <link;wiring;transformers;8> on the pole, which makes it less obtrusive.

Placing blocks below the arms of the wooden post will flip the arms upside down.

20.0.74 Turntable

I can't keep up

The Turntable will rotate blocks placed against it.

Holding an Engineer's Hammer while looking at the machine displays its active side and turn direction. Rightclicking with the Hammer will reorient it, sneaking while doing so will invert the Turntable's rotation direction.

20.0.75 Turrets

Are you still there?

Turrets are the next step in automated defense of your factory. They feature a complex optical system combined with mechanical "brain" to effectively distinguish and acquire targets and use the inbuilt weaponry to neutralize them.

All turrets feature energy ports at their base, and require a redstone signal to be active, although this behaviour can be inverted by sneaking and rightclicking it with an Engineer's Hammer.

Upon being placed, Turrets will remember their owner and will not allow other users to break or modify them. Rightclicking the turret brings up the GUI which features a list of names with an entry field, as well as multiple targeting options.

By the default the list is a blacklist of targets that may not be attacked. This list utilizes creatures names, so whitelisting animals is possible.

The Chemthrower Turret utilizes the fluid propelling capabilities of the <link;chemthrower;eponymous tool> to cover targets in anything, ranging from water to douse flames, to diesel oil to start them.

The turret features a fluid port in the bottom and back of the base to allow pipes to connect to it. Additionally, the GUI has a button on the left which dis- or enables the pilot flame, igniting the dispensed fluid, provided that it is flammable.

The providers of this blueprint are not liable for any damage to user, neighbours or various farm animals that may incur.

The Gun Turret features a heavily modified version of the <link;revolver;Revolver's> internals in a far bigger and more imposing casing. The construction also involves an extended magazine to allow for increased internal storage of ammunition.

A port for the automatic insertion of ammo and extraction of spent casing is located within the back and bottom of the base. The turret can store up to a full stack of cartridges and the GUI features a button to toggle the automatic expulsion of spent casings.

Note that certain ammunitions may cause collateral damage to the user or their property.

20.0.76 updateNews_

Dear Engineer!

Since you last worked with Immersive Engineering, many things have changed! Among the new things are: Villager houses, <link;lighting;new lights>, <link;barrel;wooden barrels>, <link;tanksilo;tanks>, <link;tanksilo;silos;1> and <link;fluidPipes;fluid pipes>, achievements, <link;shader;shaders> and the <link;assembler;Assembler> and <link;bottlingMachine;Bottling Machine>!

Most importantly however, the way energy transmission is handled has changed a lot. In order to avoid mistakes, please make sure to read the entry on <link;wiring;Wiring>, especially page 5, detailing the new distinction between a wire's transfer rate and a connector's in- and output.

Dear Engineer!

The past update has brought some new features and changes to Immersive Engineering:

<link;metalbarrel;Metal Barrels> offer a more stable, yet portable fluid storage solution, the Arc Furnace has been improved to alloy certain metals, as well as recycle old tools and armor, and the scaffolding used to cover a <link;fluidPipes;Fluid Pipe> can now be climbed.

Dear Engineer!

The past update has brought some new features and changes to Immersive Engineering:

A new variation of scaffolding was invented, that replaces the unsafe, open steel struts on the top with some wooden planks. Aluminium has turned out to be quite useful for decorative purposes, as such, fences and scaffolding have now been made available in your local Engineer's shop.

A new kind of <link;breaker;breakerswitch;2> was invented to be controlled with redstone signals and accept HV lines. Subsequently, HV lines are now forbidden from being attached to normal breakers, due to lifethreatening voltages. We don't want trouble with the union.

Dear Engineer!

The past update has brought some new features and changes to Immersive Engineering:

After suffering a severe concussion, Chief Engineer Adams drafted plans for the <link;chemthrower;Chemical Thrower>, a device that allows you to spray any fluid anywhere. It also features a built-in pilot light to ignite the fluid spray. We think Adams might be insane.

Another new invention of the research department is the <link;chargingStation;Charging Station>, which allows you to insert energy into items and charge them up.

Assistant Engineer "Cobra" has presented his newest invention, the <link;conveyor;Dropping Conveyor;2>, which drops items passing over it into inventories below.

To top it all off the <link;sorter;Item Router> has had its interface designed to allow for OreDictionary-, NBT-based and Fuzzy filtering of items!

Other noteable additions all around the topic of transport and decoration include:

- <link;shader;Shaders> can be applied to Minecarts

- <link;skyhook;The Engineer's Skyhook>, a tool to allow you to zip along your powerlines and overtake those Minecarts

- <link;balloon;Balloons>, as a convenient way to suspend connectors in the air and form the skylines for the afore mentioned hook

- <link;concrete;Concrete> that will make for a hard landing when you fall off the skyline

Finally, the financial department has been hassling us for not keeping track of our power usage. After a heated argument, the have presented us with the blueprints for the <link;eMeter;Current Transformer>, a device that can measure the energy flowing through it. Needless to say they are even less happy now that they know how much energy an Arc Furnace really consumes.

Dear Engineer!

The past update has brought only little changes.

The energy transmission is fixed again, the engineer it that broke has been dunked into cold water as punishment.

The <link;furnaceHeater;Furnace Heater> has seen some improvements that now allow it to heat up Alchemical Furnaces from Thaumcraft or Evaporators from Cutting Edge.

That's all for now, let's hope it stays sufficiently stable.

20.0.77 Grid Engineering

Power without the poles

Running catenary wire across a city is honest work, but it is not always practical. The Virtual Power Grid moves Flux between places that are not connected by any wire at all - across a mountain, across a chunk border, across dimensions.

Power still has to enter and leave somewhere. It enters at a Grid Feed Unit and leaves at a Grid Service Unit: sheet-metal service boxes that bolt to any solid surface, including engineering posts.

The grid is not one pool. It is divided into segments - named groups of boxes, each with its own switch, its own transfer limits, and its own statistics. A segment is the thing you switch off when you want the east side of town dark; the boxes on it are just the sockets.

A box that has not been assigned to a segment sits there with an unlit lamp doing nothing. Assigning it is the whole configuration step.

These boxes exchange power with the block they are bolted to, exactly as a machine does - and they take a wire directly as well. String a coil at the terminal post on the front of a Feed or Service Unit and it connects like any connector would; no relay block in between, and any of LV, MV or HV.

The wire sets the tier and the loss, exactly as it does everywhere else.

A Feed Unit takes power out of the world; a Service Unit puts it back.

One Console Housing and one each of the three engineering blocks, in a 2x2 wall struck on the front face with an Engineer's Hammer, make the Grid Management Console.

Housing top left, Redstone Engineering beside it, Light Engineering under the housing and Heavy Engineering at the bottom right. Hammer any of the four.

The console is where the grid is run from. Six tabs: an overview of every segment, the segment editor, the device list, the failover editor, live statistics, and settings.

Right-clicking any box also opens its own small panel, so a freshly placed unit can be assigned without walking back here.

Each segment carries a priority order and a transfer cap per device, so you can decide what gets served first when there is not enough to go round. Anything marked critical is served before everything else - that is your hospital, your pumps, your blast furnace.

If breakers are enabled in the config, a segment held at its output ceiling long enough will trip and latch off until somebody flips it back on. That is a real switch on the Segments tab, not a formality.

Segments can back each other up. On the Failover tab, give a segment an ordered list of other segments to fall back on: if it is switched off, tripped, or held down, its loads are served from the first backup that can manage it.

Backups can also cover ordinary shortfalls rather than only outright outages - that is the top-up toggle on the same tab. Loops are harmless; a segment is asked at most once per chain.

A segment can also keep its own hours. Set an on and an off time on the Segments tab in day-time ticks - 12000 is dusk, 23000 is dawn - and it will run only inside that window. Street lighting that switches itself on is the obvious use.

The schedule can only hold a segment down. It will never switch on one you switched off, so the console switch always has the last word.

The Grid Signal Unit bridges a segment to redstone in either direction.

As an output it emits a full signal while its segment is up, which drives a lamp, an alarm, or anything else you want tied to the state of the grid. Inverted, it emits while the segment is down instead - that is a fault light.

As an input it is an external kill switch: a redstone signal holds the segment off. Inverted, it demands a keep-alive signal instead, so the segment stops the moment its controller does.

Two conveniences worth knowing. Sneak-rightclick any grid box with an Engineer's Voltmeter and it picks up that box's segment; every sneak-rightclick after that assigns the box you clicked to it, which saves a great deal of walking when wiring a long run. Sneak-rightclick the air to empty the tool again.

Sneak-rightclick a box bare-handed for a status readout, and use /ie grid for the same information from a chat window.

The Grid Linker is the tool for wiring a street rather than a box.

Rightclick any grid box with it and a short list of segments appears - name, colour and what each is doing. Pick one and it links that box and loads the tool; every rightclick after that links the box you clicked, with no window in the way.

Sneak-rightclick a box to choose a different segment, and sneak-rightclick the air to empty the tool. A locked segment stays locked: a tool in hand is not a way around one.

20.0.78 Basic Wiring

Bzzzt!

The energy-net of Immersive Engineering has four important blocks: Capacitors to store energy, connectors as inputs and outputs for the net, relays to connect wires together and transformers to switch between different levels of wire.

To connect two blocks, simply click the first one with a wire coil, then use the same coil on the second block. The total transfer rate between two points depends on the weakest type of wire between them.

Note that restrictions apply based on blocks: Connectors and relays will only take wires of the same type that is already connected, transformers will only take two wires of a different type,

[cont]
and wires can only connect to blocks that match the voltage.

There are two important values to note about wiring:

Each connector has an input value (<config;iA;wireConnectorInput>) and each wire has a maximum transfer rate (<config;iA;wireTransferRate;l3>).

The connector limits how much power can be in- and output, but multiple connectors can unify into a single wire, allowing you to combine the transfer over longer ranges. Note however that the maximum transfer rate of the wire must not be exceeded, or else the wire will burn up.

Wires will transfer energy even through unloaded chunks as long as the input and output chunks are loaded.

Uninsulated wires connected to an energy source will cause damage to players and mobs too close to them. Insulated wires are available for low and medium voltage.

Sneaking when using the "Pick Block" function on a connector will pick the connected wire from your inventory.

Wire connections will break if a block obstructing the wire is placed.

20.0.79 wiringFeedthrough

Feedthrough Insulators allow connecting wiring from one side of a wall to wiring on the other side without a hole. They can be created with any "normal" solid block in the middle. All common types of connectors (power and redstone) can be used to create feedthrough insulators for the

corresponding wire type. Right-click one of the connectors with a hammer to form the multiblock.

20.0.80 wiringTransformer

Transformers allow you to transition between different types of wire. The weakest wire restricts the transfer rate. Note that HV Transformers can step down to LV or MV, so you do not need to chain two transformers.

You can attach <link;wiring;transformers to wooden poles;3> as shown on the second image on page 4, however only non-HV Transformers can be placed like this.

20.0.81 Workbench & Blueprints

Always cluttered

The Engineer's Workbench is a professional worktable for the educated engineer. It is widely used to construct the items described in <link;blueprints;Engineer's Blueprints>.

A wide variety of blueprints can be crafted, as detailed in the respective manual entry, whereas a few select ones can only be discovered in the world or purchased from villagers.

To use a blueprint in the Workbench, it in the slot on the left of the GUI and the necessary ingredients in the six slots in the middle. The resulting items can then be retrieved from the right side. Hovering your mouse over these output slots will inform you exactly which materials are needed for each item.

A secondary function of the Workbench is the modification and configuration of more complex tools and gadgets. Examples like the <link;drill;Mining Drill>, <link;shield;Heavy Plated Shield> or <link;revolver;Revolver> are examples of such items. When placed in the main slot of the workbench, it's use becomes that of modifying these tools by inserting applicable upgrades into the new slots that appear.